
[Embedfire] Practical Guide to Python Application Development —Based on LubanCat-RK Series Boards

EmbedFire
野火电子

2024 年 09 月 21 日



Contents

About this project	1
LubanCat	1
Documentation	1
Other supporting online documents and tutorials for the board	2
About Embedfire	3
Open source sharing, common progress	3
Contact information	3
Chapter1 LubanCat series board and python	4
1.1 Introduction to Python	4
1.2 Introduction to LubanCat	4
1.3 LubanCat+Python =?	5
1.4 Reference information	6
Chapter2 Install Python	7
2.1 Install Python, PIP through APT	7
2.2 About python version	8
2.3 Set the default version of Python and PIP	9
2.4 Software library installation method	10
2.4.1 pip download acceleration	10
2.4.2 Use APT instead of PIP installation software package	10
2.4.3 Install the software package with the SetupTools tool	11
2.5 Reference information	13
Chapter3 Run python	14
3.1 Use the python terminal interactive environment	14
3.2 Use python script	16
3.3 Use python IDE	18
3.4 Import other software packages	20
3.5 Reference information	23

Chapter4	Python basic quick view	24
4.1	Note	24
4.2	Indentation	24
4.3	Variables and Data Types	25
4.3.1	String	26
4.3.2	List	27
4.3.3	Tuple	27
4.4	Process control related	28
4.4.1	conditional control	28
4.4.2	Loop statement	29
4.4.3	Iteration	29
4.5	Function	30
4.6	Modules and Packages	30
4.6.1	Module	30
4.6.2	Package	31
4.7	virtual environment	31
4.8	Reference documents	33
Chapter5	Control GPIOs	34
5.1	libgpiod basic concept	34
5.2	Experiment preparation	36
5.3	Method 1: Use python3-libgpiod	36
5.3.1	Install python3-libgpiod	36
5.3.2	libgpiod output	37
5.3.3	libgpiod input and output	39
5.4	Method 2: Use python-periphery	40
5.4.1	Install python-periphery	40
5.4.2	peripheral input and output	41
5.5	Method 3: Using Adafruit Blinka	43
5.5.1	Installing Adafruit Blinka	43
5.5.2	Read before use	43
5.5.3	Adafruit-Blinka input and output	45

5.6	References	46
Chapter6	PWM output	48
6.1	PWM experiment	48
6.2	Experiment preparation	49
6.3	Use python-periphery	50
6.3.1	Install python-periphery	51
6.3.2	peripheral output PWM	51
Chapter7	UART communication	54
7.1	Experiment preparation	54
7.1.1	Add UART resource	54
7.1.2	Hardware connection	56
7.2	Method 1: Use the pyserial library	57
7.2.1	Install pyserial	57
7.2.2	Use pyserial	57
7.3	Method 2: Use python-periphery	61
7.3.1	Install python-periphery	61
7.3.2	The periphery uses the UART function	61
Chapter8	I2C communication	65
8.1	I2C experiment	65
8.2	Experiment preparation	66
8.2.1	Add I2C resources to the board	66
8.2.2	Hardware connection	68
8.3	Use python-periphery	68
8.3.1	Install python-periphery	68
8.3.2	The periphery uses the I2C function	69
8.4	Method 2: Using Adafruit Blinka	75
8.4.1	Installing Adafruit Blinka	75
8.4.2	Adafruit - Blinka using I2C master	75
8.4.3	Adafruit_CircuitPython_SSD1306	80
Chapter9	SPI communication	83
9.1	SPI experiment	83

9.2	Experiment preparation	83
9.2.1	Add SPI resource	83
9.2.2	Hardware connection	85
9.3	Use python-periphery	86
9.3.1	Install python-periphery	86
9.3.2	The periphery uses the SPI function	86
9.4	Method 2: Using Adafruit Blinka	88
9.4.1	Installing Adafruit Blinka	88
9.4.2	Adafruit - Blinka using SPI master	89
Chapter10	Screen Display - Pygame	92
10.1	Introduction to the Pygame library	92
10.2	Experiment preparation	92
10.2.1	Add screen resources	92
10.2.2	Hardware connection	93
10.2.3	Pygame library installation	94
10.2.4	The Pygame library uses	94
10.3	Test code one	94
10.3.1	Experimental procedure	96
10.4	Test code two	97
10.4.1	Experimental procedure	99
10.5	Reference	99
Chapter11	Audio playback - Pygame.	100
11.1	Introduction to the Pygame library	100
11.2	Experiment preparation	100
11.2.1	Sound resources on the board	100
11.2.2	Hardware connection	101
11.2.3	Pygame library installation	101
11.2.4	The Pygame library uses	101
11.3	Test code	101
11.3.1	Experimental procedure	104
11.3.2	Reference	105



Chapter12	Camera use-Opencv	106
12.1	Introduction to opencv-python library	106
12.2	Add camera resource to Lubancat board	107
12.3	Related tools and library installation	107
12.4	Camera to take pictures	109
12.5	Camera capture, record video	111
12.6	Reference	114
Chapter13	Jupyter Notebook	115
13.1	Introduction to Jupyter Notebook	115
13.2	Jupyter Notebook installation	116
13.2.1	Software Installation	116
13.2.2	Software configuration	116
13.2.3	Security Login Configuration	117
13.3	Precautions before using Jupyter Notebook	118
13.4	Jupyter Notebook uses	119
13.4.1	start software	119
13.4.2	Upload code	121
Chapter14	PIL library	123
14.1	Introduction to the PIL library	123
14.2	PIL library installation	123
14.3	PIL library usage	123
Chapter15	Numpy library	127
15.1	Introduction to the Numpy library	127
15.2	NumPy library installation	127
15.3	NumPy library used	127
Chapter16	Pandas library	129
16.1	Introduction to the Pandas library	129
16.2	Pandas library installation	129
16.3	Pandas library usage	129
Chapter17	Web library - Flask	131
17.1	Introduction to the Flask library	131

17.2	Flask library installation	131
17.3	Flask library usage	132
17.3.1	New project directory	132
17.3.2	Add code file	132
17.3.3	Run webapp	135
17.3.4	Reference	136
Chapter18	Web library - Django	137
18.1	Introduction to Django Libraries	137
18.2	Django library installation	137
18.3	Django library uses	137
18.3.1	New project directory	138
18.3.2	Add code files and configuration	138
18.3.3	Start webapp	139
18.3.4	Django Version Notes	140
Chapter19	Modbus protocol - pymodbus library	142
19.1	Modbus protocol experiment	142
19.2	python3 - pymodbus library	142
19.3	Experiment preparation	143
19.3.1	Add board uart resources	143
19.3.2	Hardware connection	145
19.3.3	pymodbus library installation	146
19.4	Test code	146
19.4.1	RTU slave slave_server.py	146
19.4.2	Experimental procedure	152
19.4.3	Analysis of the phenomenon	160
19.4.4	Reference	161
Chapter20	GUI library - PyQt5	162
20.1	Introduction to PyQt5 library	162
20.2	PyQt5 library installation	163
20.2.1	Qt library installation	163
20.3	Basic use of PyQt5 library	164

20.3.1	Import required modules	164
20.3.2	Create a class that will act as the main window	165
20.3.3	Create application	166
20.3.4	Overall program	167
20.4	Qt Designer uses	169
20.4.1	Create an interface UI file	171
20.4.2	Interface construction	171
20.4.3	Call ui file	175
20.5	Reference	178
Chapter21	Artificial Intelligence	179
21.1	Introduction	179
21.2	Artificial intelligence and Lubancat RK series board	181
21.3	Reference link	181
Chapter22	NPU	182
22.1	RKNN-Toolkit2	183
22.1.1	RKNN-Toolkit2 installation	183
22.1.2	Model Transformation and Model Inference	185
22.1.3	Performance and Memory Evaluation	185
22.2	RKNN Toolkit Lite2	190
22.2.1	Install RKNN Toolkit Lite2 on the board	190
22.2.2	Board side deployment reasoning	192
22.3	Related frequency settings (rk356X)	195
22.4	Reference	196
Chapter23	Handwritten digit recognition –PaddlePaddle	197
23.1	PaddlePaddle Introduction	197
23.2	Install PaddlePaddle	198
23.3	Handwritten digit recognition task	199
23.3.1	Prepare training and test sets	199
23.3.2	Model networking	200
23.3.3	Model Training and Evaluation	201
23.3.4	Model saving and exporting onnx model	202

23.3.5	Complete program	203
23.3.6	Simulation reasoning and export RKNN model	206
23.4	Board deployment reasoning	209
23.4.1	Install RKNN Toolkit Lite2 and related libraries	209
23.4.2	Reasoning test	209
23.5	Summarize	211
23.6	Reference link	211
Chapter24	PaddlePaddle FastDeploy	212
24.1	FastDeploy	212
24.2	Environment configuration	213
24.2.1	PC-side model conversion and inference environment construction	213
24.2.2	Build FastDeploy RKNPU2 inference environment on board	214
24.3	Example deployment inference	215
24.3.1	Lightweight detection network PicoDet	215
24.4	Reference link	220
Chapter25	Flower image classification–TensorFlow	222
25.1	Install TensorFlow environment	222
25.2	Image classification	223
25.2.1	Prepare data set	223
25.2.2	Construction and training model	225
25.2.3	Test model	231
25.2.4	TensorFlow Lite model	232
25.2.5	Model conversion and simulation test	233
25.3	Deployment reasoning test	236
25.4	Summarize	238
25.5	Reference link	239
Chapter26	Resnet18 network –PyTorch	240
26.1	PyTorch and ResNet18	240
26.1.1	Pytorch installation	240
26.1.2	ResNet18 Structure Introduction	242
26.1.3	Resnet18 in pytorch implementation	243

26.2	Resnet18 implementation	243
26.2.1	Data set preparation and data pre -processing	243
26.2.2	Build a model	245
26.2.3	Training and test model	252
26.2.4	Save as onnx model	254
26.2.5	Export the RKNN model and simulation test	254
26.2.6	Board deployment test	257
26.3	Reference	261
Chapter27	YOLOv5	262
27.1	YOLOv5 environment installation	262
27.2	YOLOv5 is easy to use	263
27.2.1	Get the pre-trained weights file	263
27.2.2	YOLOv5 simple test	264
27.2.3	Convert to rknn model	265
27.2.4	Deploy to LubanCat Card	274
27.3	airockchip/yolov5 Simple test	277
27.3.1	Converted to the RKNN model and deployed to LubanCat RK356X board	280
27.4	Reference link	283
Chapter28	Lubancat monitoring and detection	284
28.1	Dependent tools and library installation	284
28.2	Video streaming server and camera get frames	285
28.2.1	Deploy a video streaming server	285
28.2.2	Get frame from camera	288
28.3	NPU processing image	290
28.4	Adapt to the specific environment	292
28.5	Test experiment	294
28.6	Reference link	298
Chapter29	feedback	299
	Copyright statement	300

About this project

LubanCat

1. Lubancat is a brand of single-board computers computers running Linux and Android launched by Embedfire. This series of single-board computers has rich hardware models, high operating system adaptability, numerous open source teaching materials, and extremely simple application;
2. Lubancat' s excellent performance and rich hardware models cover the fields of education, commercial applications, industrial control, etc. It has a wide range of application scenarios: single-board computers, Linux server, home intelligent hub, industrial board;
3. Lubancat supports Ubuntu, Debian, Android and other systems, and provides multiple sets of teaching materials. Covering pure application layer users and system development users, even embedded beginners who are new to the industry. It can also complete the development according to our tutorials, and for experienced embedded veterans, it can speed up the secondary development process of the product.

Documentation

Click the link on the right to read this project document online: “[LubanCat Python Application Development Practice–Based on LubanCat_RK Series Board](#)”

This tutorial aims to guide developers to use Python to develop applications, control peripheral hardware, etc. on LubanCat.

The development environment used in this tutorial is described as follows:

- python version: Python3 or above
- Development board system: LubanCat system image (ubunutu, debian, etc. in extboot partition) customized by Embedfire.



Other supporting online documents and tutorials for the board

- ” [LubanCat-RK Series Board Quick Start Manual](#) “
- ” [Embedded Linux image construction and deployment - based on LubanCat-RK series board](#) “
- ” [LubanCat-RK Series Board Application Development Manual](#) “
- ” [Practical Guide to Embedded Linux Driver Development - Based on LubanCat_RK Series Boards](#) “
- ” [LubanCat-RK Series Board Android User Manual](#) “

About Embedfire

Open source sharing, common progress

At the beginning of the release of the first STM32 development board, Embedfire shouted the slogan of **Open source sharing, common progress**, The code and documentation tutorials are provided for users to download for free, and we have always carried this concept throughout.

Currently our products include *STM32*, *i.MX RT series*, *GD32V*, *FPGA*, *Linux*, *emXGUI*, *Operating system*, *network*, *downloader* and other branches, covering various commonly used technologies in the field of electronic engineering applications, Among them, the codes and documents of teaching products have been published on the Internet in an open-source manner. It is our greatest wish to solve problems for electronic engineers and make embedded technology free of difficult-to-use technologies.

Contact information

- Official website: <http://www.embedfire.com>
- Forum: <http://www.firebbs.cn>
- github homepage: <https://github.com/Embedfire>
- gitee homepage: <https://gitee.com/Embedfire>
- Taobao: <https://yehuosm.tmall.com>
- Email: embedfire@embedfire.com
- Tel: 0769-33894118

Chapter1 LubanCat series board and python

1.1 Introduction to Python

What is Python, the brief summary here is as follows:

- Python is a cross -platform parsing programming language.
- Python is easy to learn, easy to use, and powerful, and has been applied to AI, data processing, programming education.
- A large number of developers provide a variety of Python code libraries. Others can use these libraries to easily develop their programs.

For more detailed tutorials of Python, please refer to:

- 《廖雪峰-Python 教程》
- 《和孩子一起学编程》
- 《趣学 Python》

1.2 Introduction to LubanCat

LubanCat is a high-performance single-board computer brand launched by Embedfire. It takes inspiration from Luban, encouraging engineers to inherit the innovative spirit of Luban the craftsman and strive to become contemporary Luban. The cat-shaped design symbolizes the curiosity and explorative spirit of both children and cats, urging everyone to stay curious and never stop exploring, while keeping a youthful heart.

The LubanCat series of single-board computers offer a wide range of hardware models, high compatibility

with operating systems, abundant open-source teaching materials, and extremely simple application. It provides convenient examples and applications for various scenarios including AI, industrial control, Internet of Things, robotics, and programming education.

LubanCat System is a Linux distribution developed by Embedfire based on Ubuntu, Debian, Android, and other systems. It supports direct installation and maintenance of software using APT package management tools. Popular software such as Python, OpenCV, Nginx, and Docker can be easily deployed with it, making it effortless to deploy various applications.

1.3 LubanCat+Python =?

It is also very convenient to use traditional PC development and Python applications that run pure software types. However, when you want to control external hardware or electrical equipment, you need to expand special hardware such as IO cards and motion control cards, which are expensive and complicated.

The LubanCat series board with mobile phone size integrates various hardware control and communication interfaces such as IO, PWM, I2C, USB, and networks. It can easily interact with external equipment, and the characteristics of low power consumption, industrial stability, and high cost performance are very suitable for maker DIY and industrial control products.

Embedfire will provide the following Python application development example based on the Lubancat board:

- Create a personal website
- Develop a smart home data center
- Build an AI-powered smart speaker
- Design and develop a robot
- Set up a programming teaching environment
- ...

1.4 Reference information

- [python official website](#)
- 《廖雪峰-Python 教程》
- 《和孩子一起学编程》
- 《趣学 Python》

Chapter2 Install Python

This chapter explains how to install the Python environment on the Lubancat board.

重要: If it is not specially mentioned, this tutorial is based on Python 3.8.10 (mirror system is Ubuntu20.04) for experiments and explanations.

2.1 Install Python, PIP through APT

The LubanCat system supports the APT tool, and can be installed with the APT command directly. We use Python3 version directly:

```
# Execute the following commands on the board, you need to connect the
↪network.
# You need to update the first time using APT.
sudo apt update

# Install Python3 (the default system has been installed to not execute)
sudo apt -y install python3

# Install PIP tools
sudo apt -y install python3-pip
```

提示: PIP tools are used to install other Python libraries, and you can choose whether to install according to your needs. For the development platform of the test, it is recommended to install it for easy use.

2.2 About python version

Lubancat Mirror is based on Linux distributions such as Debian and Ubuntu. These distribution versions are attached with Python by default. Generally, Python2*older versions and Python3*latest versions. You can use the following command to view the default installation version:

```
# View python3 version
cat@lubancat:~$ python3 --version

# The following is output
Python 3.8.10

# View python version
cat@lubancat:~$ python2 --version

#Output
Python 2.7.18

# View pip3 version
cat@lubancat:~$ pip3 --version

# The following is output
pip 20.0.2 from /usr/lib/python3/dist-packages/pip (python 3.8)
```

提示: If you use the Debian10 mirror, the default is Python3.7.3 and Python2.7

2.3 Set the default version of Python and PIP

It can be noted that the Python3 and PIP3 in the above commands have the version number, mainly due to the historical reasons of Python, so that the use can be better distinguished from Python2. Python2 has stopped maintenance, and we don't recommend that everyone continue to use it. The mirror that we provide is installed with Python2 and Python3 by default, and you can set the default version of the current system.

You can set the Python and PIP commands by default by the following commands: Python3:

```
#Set up soft links, Python uses Python3 by default
sudo ln -sf /usr/bin/python3 /usr/bin/python

#Set the soft link, PIP defaults to use PIP3
sudo ln -sf /usr/bin/pip3 /usr/bin/pip
```

After setting, you can directly use the Python or PIP command to check the current system Python version:

```
#View python
cat@lubancat:~$ which python
/usr/bin/python

# View python version
cat@lubancat:~$ python --version
# The following is output
Python 3.8.10

# View pip version
cat@lubancat:~$ pip --version
# The following is output
pip 20.0.2 from /usr/lib/python3/dist-packages/pip (python 3.8)
```

In order to be more clear, the Python3 or PIP3 commands are still explained directly behind this book.

2.4 Software library installation method

2.4.1 pip download acceleration

When using Python later, you may need to use the PIP tool to download and install the dependent software package, and the official download station *pypi* <<https://pypi.org/>> _ The visit in China is not fast.

You can refer to the following webpage description Settings acceleration: [pypi image use help](#) .

For example, use PIP installation software package adafruit-circuitpython-ssd1306:

```
# Use APT to install Python's Adafruit-Circuitpython-SSD1306 package
sudo pip3 install adafruit-circuitpython-ssd1306

# Use pip list to view the installed package
pip3 list | grep adafruit-circuitpython-ssd1306

# Uninstall the package with PIP Uninstall
sudo pip3 uninstall adafruit-circuitpython-ssd1306
```

2.4.2 Use APT instead of PIP installation software package

When installing a software package with a PIP tool, it is usually compiled in this machine. The performance of some Lubancat board cards has caused a long compile time and may fail due to the lack of certain library files.

Therefore, when using a panel card with a low performance, we recommend searching for whether to use the APT tool to install. It will download the pre -compiled software package from the software library, and the installation time basically depends only on the network speed. You need to install the updated version or the package that you cannot find.

For example, the NUMPY data science library commonly used by Python is used as the following APT command installation, and it will be completed soon:

```
# Use APT to install python Numpy package  
sudo apt -y install python3-numpy
```

As for the names of other specific software wrapped in the APT tools, you can search for content such as “APT installation XXX (such as Numpy)” in the network. If you are using the Debian system, find keywords such as “Numpy” in the software list library of Debian: [Find the official software package of Debian](#) .

2.4.3 Install the software package with the SetupTools tool

In some cases, some Python libraries that our users need to use. For some reasons, we only got the source code of these library bags. Then we can't install it with PIP tools or APT tools.

At this time, we can use the Python's Setuptools tool to install the Python library package through the source code of the library.

SetupTools tool installation method is as follows:

```
# Enter the following command in the terminal:  
sudo apt -y install python3-setuptools
```

Let's use the SetupTools tool.

Taking the Jieba (stutter) library under Python as an example, the library can be used for Chinese words, which is a very popular open source Python project in Github.

Its github warehouse is:[jieba](#)

We can pull the source code from its warehouse or download its Releases compressed package and install it.

The source code directory is as follows:

```
cat@lubancat:~/jieba$ tree -L 1 ../jieba/
../jieba/
├── Changelog
├── LICENSE
├── MANIFEST.in
├── README.md
├── extra_dict
├── jieba
├── setup.py
└── test

3 directories, 5 files
```

Enter the source code directory and use the Setuptools tool to install the library through the source code:

```
#Pull source code
git clone https://github.com/fxsjy/jieba.git

# Enter jieba/directory and enter the following commands in the terminal:
sudo python3 setup.py install
```

Wait for the installation of the software package to complete.

Enter the command below. You can see that the Jieba library can be used normally.

```
#Log in to the system terminal, enter python3 or ipython, enter the Python_
↪interactive mode, and then use the command:
import jieba
set_list = jieba.cut("Welcome to the Python Practice Tutorial", cut_
↪all=False)
print("Default Mode:" + "/" .join(set_list))
```

```
cat@lubancat:~$ ipython
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.6.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: import jieba

In [2]: set_list = jieba.cut("欢迎来到野火Python实践教程", cut_all=False)

In [3]: print("Default Mode:" + "/".join(set_list))
Building prefix dict from the default dictionary ...
Loading model from cache /tmp/jieba.cache
Loading model cost 3.286 seconds.
Prefix dict has been built successfully.
Default Mode:欢迎/来到/野火/Python/实践/教程

In [4]: █
```

2.5 Reference information

- [Pypi mirror use help](#)
- [Find the official software package of Debian](#)

Chapter3 Run python

This chapter explains how to use Python on the Lubancat board, and its usage is no different from that on the PC.

3.1 Use the python terminal interactive environment

We can directly run python commands to enter Python interactive programming. Interactive programming does not require the creation of script files, and codes are written through the interactive mode of the Python interpreter.

You can do some simple operations in the interactive environment, such as outputting “hello world!” in the terminal, doing some arithmetic operations, etc., and finally exit through `exit()`:

```
# Type in terminal
python3
# Output content
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>

# Enter and press enter
>>> print("hello world!")
# Output
hello world!
>>>

# Output
```

(下页继续)

(续上页)

```
>>> 8+2
# Output
10

# To exit the interactive mode, you can also use quit() or the key_
↪combination Ctrl+D to exit
>>> exit()
cat@lubancat:~$
```

Alternatively, to use python in the terminal, you can also install ipython (enhanced interactive Python shell):

```
# Install ipython
sudo pip3 install ipython

#Use
cat@lubancat:~$ ipython
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.6.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]:

# Enter 8+2
In [1]: 8+2
Out[1]: 10

In [2]:

#Use exit to exit the interaction
```

3.2 Use python script

We can also write a simple Python code, save it as `hello.py` file and execute it.

You can use the serial port to log in to the board system and use the vim editor to write, or vscode ssh to log in to the board for writing:

The content of the `hello.py` code file is as follows:

列表 1: base/hello.py file content

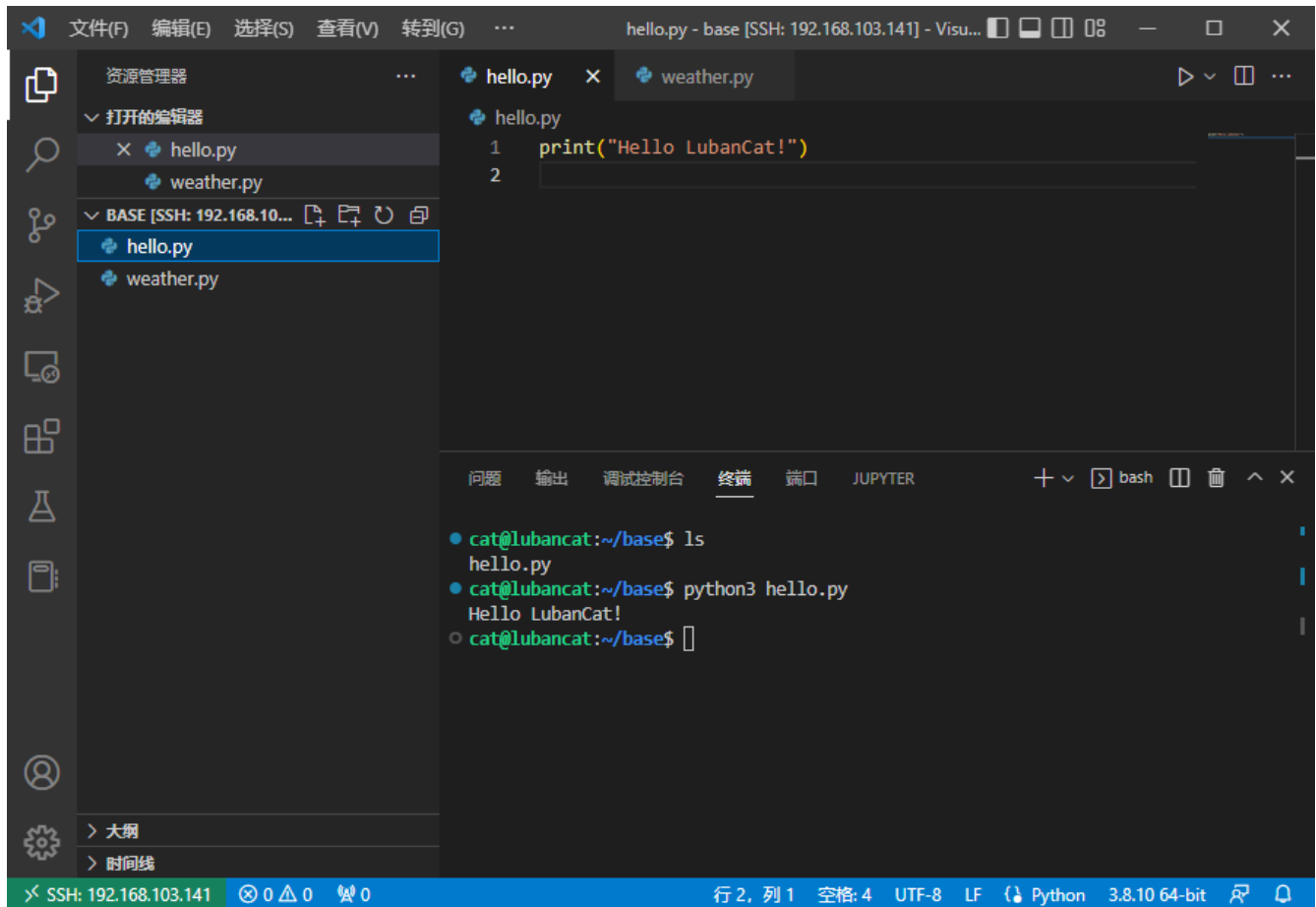
```
print("Hello LubanCat!")
```

Then run:

```
# Write hello.py
# Use the ls command to confirm that the file exists in the directory
ls
# The following is output
hello.py

# Run
python3 hello.py
# Output
Hello LubanCat!
```

The following is the scene of writing and running using VS Code Remote:



Use script, you can also specify the path of the interpreter in the script, such as:

```
.. code-block:: python
```

caption base/hello.py file content

```
#!/usr/bin/python print( "Hello LubanCat!" )
```

Then run:

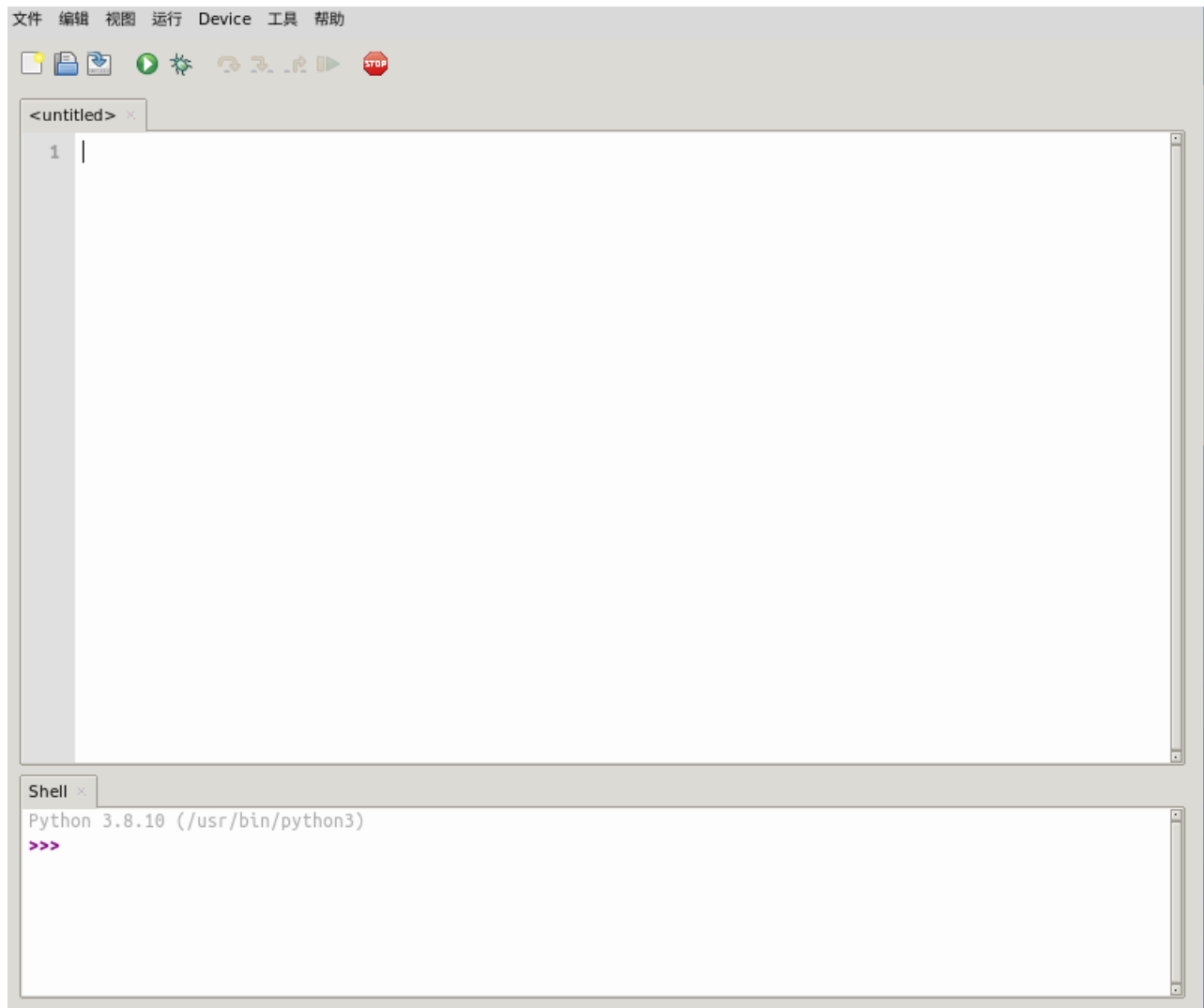
```
# Run
cat@lubancat:~$ ./hello.py
Hello LubanCat!
cat@lubancat:~$
```

3.3 Use python IDE

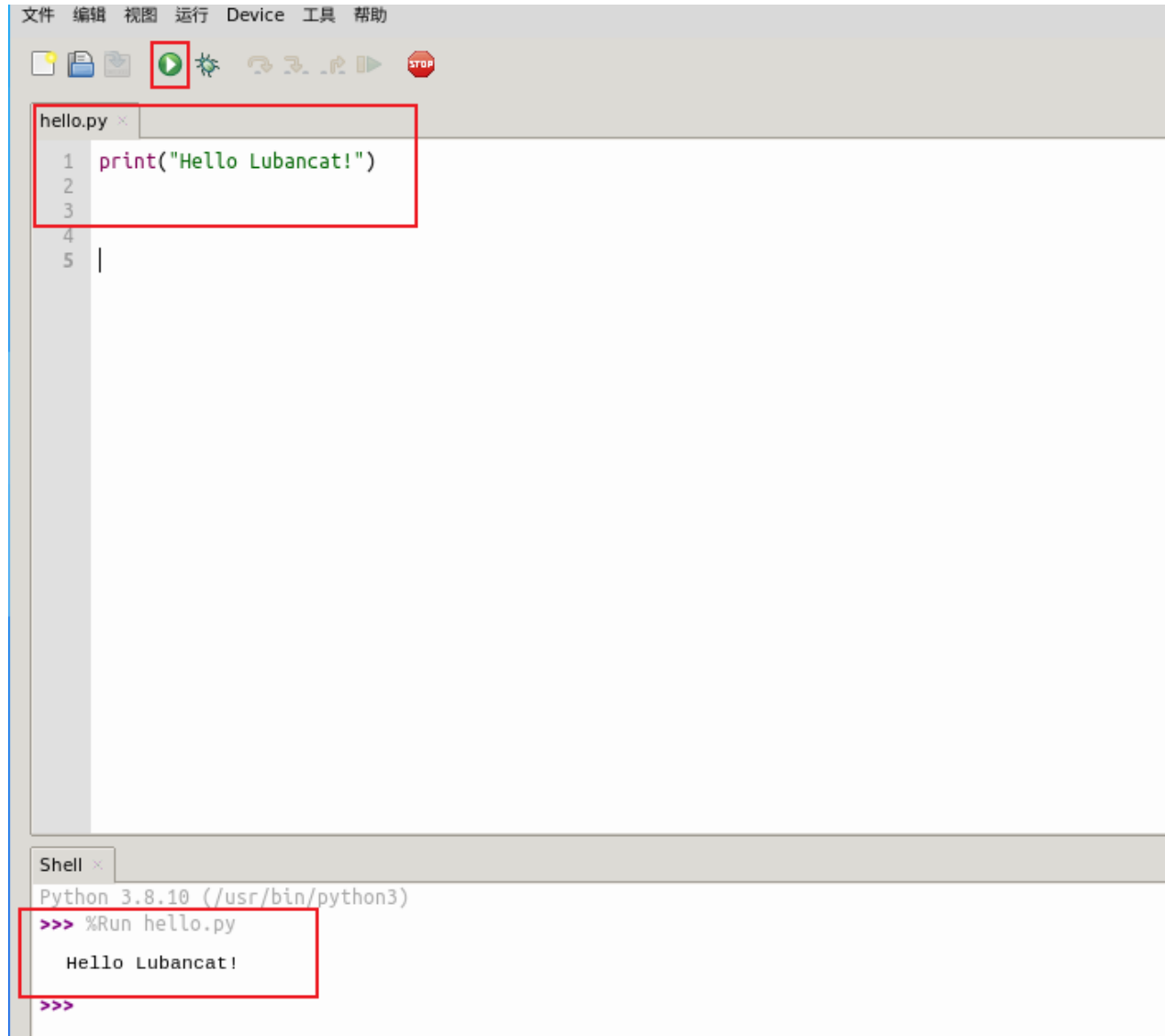
If you use a desktop system and screen display, you can choose to install the Python IDE for learning. Take the THONNY IDE as an example:

```
#Use the APT package manager to install  
sudo apt install thonny
```

Open the display after installation (take the XFCE4 desktop as an example):



Test, click Run:



Thonny is an integrated development environment (IDE) for Python beginners. It supports display grammatical errors and interpretation of scope, built-in debugger and gradual execution evaluation, and GUI interface to manage Python software packages.

3.4 Import other software packages

In addition to the Python code library of running foundation, it can also import third-party software packages in the code to run more complex applications.

To explain the example program, we create a new “Get_info.py”, write the following code:

列表 2: Supporting code base/get_info.py file content

```
#Import psutil package
import psutil
import datetime

#Get the current user information, the output name username, Terminal_
↪terminal, host host address, Started login time, PID process ID.
print("Current user:", psutil.users())

#Get the current system time and convert it to a natural time format
print("Current System Time:", datetime.datetime.fromtimestamp(psutil.boot_
↪time()).strftime("%Y-%m-%d %H: %M: %S"))

print("\n-----Memory information-----")
mem = psutil.virtual_memory()
print("System memory all information:", mem)
mem_total = float(mem.total)
mem_used = float(mem.used)
mem_free = float(mem.free)
mem_percent = float(mem.percent)
print(f"Total memory:{mem_total}")
print(f"The system has used memory:{mem_used}")
print(f"System free memory:{mem_free}")
print(f"System memory usage rate:{mem_percent}")
```

(下页继续)

(续上页)

```
print("\n-----CPU information-----")
print("CPU summary information:", psutil.cpu_times())
print("CPU logic number:", psutil.cpu_count())
print("CPU frequency:", psutil.cpu_freq())
```

注解: Python' s PSUTIL (Process and System Utilities) library is a cross -platform third -party library that can easily realize the process and system utilization of system operation (including CPU, memory, disk, network, process, etc.). It is mainly used for system monitoring, analysis, and restricting the management of system resources and processes.

Save the file and try to run the get_info.py program:

```
# Runtime
python3 get_info.py

# Output
Current user:[suser(name='cat', terminal='ttyFIQ0', host='',
↳started=1662631296.0, pid=742)]
Current System Time: 2022-11-01 16: 28: 20

-----Memory information-----
System memory all information:svmem(total=2059280384, available=892481536,
↳percent=56.7, used=1123770368, free=30068736, active=1374724096,
↳inactive=373829632, buffers=83066880, cached=822374400, shared=17391616,
↳slab=232140800)
Total memory:2059280384.0
The system has used memory:1123770368.0
System free memory:30068736.0
System memory usage rate:56.7
```

(下页继续)

(续上页)

```
-----CPU information-----  
CPU summary information: scputimes(user=2365.82, nice=3.25, system=765.33, ↵  
↵idle=335360.41, iowait=79.58, irq=0.0, softirq=50.22, steal=0.0, guest=0.  
↵0, guest_nice=0.0)  
CPU logic number: 4  
CPU frequency: scpuufreq(current=1992.0, min=408.0, max=1992.0)
```

For some packaged packages, we can install in the following way (take the requests software package as an example):

```
# Method 1: Use PIP to install directly  
sudo pip3 install requests  
  
# Method 2: Use APT to install  
sudo apt -y install python3-requests
```

Some dependencies are also supported by Debian, so you can use APT to install directly. For standard dependencies, we recommend using APT installation. The installation method of PIP is mainly used to install a third-party software package that cannot be found in APT.

If you use Thonny, you can click the tools of the interface-> management package, query and install the software package:



3.5 Reference information

- “Python Basic Tutorial”

Chapter4 Python basic quick view

Python is a simple and easy -to -learn and powerful programming language. This chapter will browse the basic grammar of Python. For details, you can see the reference document.

4.1 Note

Single-line comments in Python start with #, and multiple lines can use multiple # signs, as well as ' ' ' and " " ", for example:

```
1  # This is the comment
2
3  '''
4  This is also the annotation
5  '''
6
7  """
8  This is still a comment
9  """
10 print ("Hello!")
```

4.2 Indentation

Python uses indentation to represent code blocks, and the number of indented spaces is variable, but the statements in the same code block must contain the same number of indented spaces, otherwise it will cause a running error. Examples are as follows:

```
1  #For loop
2  for i in range(5):
3      print("in")
4  print("out")
5
6  # Output result
7  in
8  in
9  in
10 in
11 in
12 out
```

4.3 Variables and Data Types

Variables and variable types in Python do not need to be declared and are dynamic. Each variable must be assigned a value before use. The variable will be created after the variable is assigned, and the type of the variable is determined by the value assigned to it. For example:

```
1  a = 10                # Integer variable
2  print(type(a))
3  a = 10.0              # Floating point variable
4  print(type(a))
5  a = "test"            # String
6  print(type(a))
```

```
1  # Output
2  <class 'int'>
3  <class 'float'>
4  <class 'str'>
```

Several standard data types are common in Python: Immutable data: Number, String, Tuple; Variable data: List, Dictionary, Set. Here are a few:

4.3.1 String

Strings in Python (non-character type) are enclosed in single quotes `' '` or double quotes `" "`, and some special characters are escaped with backslashes `\`. For example:

```
1  #!/usr/bin/python3
2  str0 = 'Python'
3
4  '''
5  字符串索引和截取
6  +---+---+---+---+---+---+
7  | P | y | t | h | o | n |
8  +---+---+---+---+---+---+
9      0   1   2   3   4   5   (index from front)
10     -6  -5  -4  -3  -2  -1   (index from back)
11     :   1   2   3   4   5   : (taken from the front)
12     :  -5  -4  -3  -2  -1   : (taken from the back)
13  '''
14  print (str0) # output the entire str0 string
15  print (str0[0]) # output the first character of the string
16  print (str0[-6]) # output the first character of the string
17  print (str0[:]) # entire str0 string
18  print (str0[2:]) # Output all characters starting from the third
19  print (str0[:-2]) # output all characters from the first to the fourth
20  print (str0 * 2) # output the string twice, it can also be written as
21  print (2 * str0)
22  print (str0 + "TEST") # connection string
```

Strings can be indexed or truncated, but the index cannot be modified, reassigned, etc.

4.3.2 List

Lists can complete the data structure implementation of most collection classes. The types of elements in the list can be different, it supports numbers, strings can even contain lists (so-called nesting). A list is a comma-separated list of elements written between square brackets []. Like string indexing, list indexing starts at 0, the second index is 1, and so on.

```
1  #!/usr/bin/python3
2
3  list0 = [ 'python', "test", 10, 10.0 ]
4  list1 = [ 123, 'Python' ]
5
6  print (list0) # output the complete list
7  print (list0[0]) # output the first element of the list
8  print (list[1:3]) # output from the second to the third element
9  print (list[2:]) # output all elements starting from the third element
10 print (tinylist * 2) # output the list twice
11 print (list0 + list1) # link list
```

4.3.3 Tuple

Python's tuples are similar to lists, except that the elements of the tuple cannot be modified. Use parentheses () for tuples and square brackets [] for lists. Tuple creation is very simple, just add elements in parentheses and separate them with commas.

Example of use:

```
1  #!/usr/bin/python3
2
3  tup1 = ('Pyrhon', 'Test', 1997, 2000)
4  tup2 = (1, 2, 3, 4, 5, 6, 7 )
5
```

(下页继续)

(续上页)

```
6 print ("tup1[0]: ", tup1[0])
7 print ("tup2[1:4]: ", tup2[1:5])
```

4.4 Process control related

4.4.1 conditional control

A Python conditional statement is a block of code that is determined by the execution result (True or False) of one or more statements.

```
1  #!/usr/bin/python3
2
3  # Number guessing game
4  number = 20
5  guess = 0
6  print("Number Guessing Game!")
7  while guess != number:
8      guess = int(input("Please enter the number you guessed (0-100):"))
9
10     if guess == number:
11         print("Congratulations, you guessed it!")
12     elif guess < number:
13         print("Guess the number is small...")
14     elif guess > number:
15         print("Guess the number is big...")
```

4.4.2 Loop statement

The loop statements in Python are for and while. For example:

```
1  #!/usr/bin/env python3
2
3  # Use while to sum
4  n = 100
5  sum = 0
6  counter = 1
7  while counter <= n:
8      sum = sum + counter
9      counter += 1
10
11 print("The sum of 1 to %d is: %d" % (n, sum))
12
13 # Built-in range() function. it will generate the array
14 for i in range(4):
15     print(i)
```

4.4.3 Iteration

An iterator is an object that remembers where to traverse. Iterator objects are accessed from the first element of the collection until all elements have been accessed. Iterators can only go forward and not backward.

For example:

```
1  #!/usr/bin/python3
2
3  list=[1,2,3,4]
4  it = iter(list)    # Create an iterator object and use the for statement
  ↪to traverse
```

(下页继续)

(续上页)

```
5  for x in it:
6      print (x, end=" ")
```

4.5 Function

Functions can improve the modularity of the application and the reuse rate of the code. Python provides many built-in functions, such as `print()`, as well as user-defined functions.

In Python, a function is defined using the `def` keyword, and the general format is as follows:

def function name (parameter list): function body

Example of use:

```
1  #!/usr/bin/python3
2  #define function my_function
3  def my_function(test):
4      print("Hello!" + test)
5
6  #use function my_function
7  my_function(' Python')
```

4.6 Modules and Packages

4.6.1 Module

Python provides a way to get definitions from a file for use in a script or an interactive instance of the interpreter. Such a file is called a module; definitions in a module can be imported into another module or into the main module, using the `import` statement.

For example:


```
1  #!/usr/bin/python3
2
3  #Use the python standard module: sys, use the import statement
4  import sys
5  print('\nPython environment path:', sys.path)
6
7  #Use the from ... import statement
8  from periphery import I2C
```

4.6.2 Package

Packages are a form of managing Python module namespaces, using “dotted module names” .

For example, the name of a module is A.B, then it represents a submodule B in a package A. Just like when using a module, you don’ t have to worry about the mutual influence of global variables between different modules, and you don’ t have to worry about the duplication of module names between different libraries when you use the form of dot module names.

For the installation and download of the package, please refer to the previous section `Installing python`.

4.7 virtual environment

Python applications often use packages and modules that are not part of the standard library, and sometimes require a specific version of a package or module, because the application may need to fix a specific bug, or the application uses a previous version of the library interface. For example: If application A requires version 1.0 of a module, and application B requires version 2.0, these requirements conflict, installing version 1.0 or 2.0 will cause one application to not run, this problem can be solved by creating a virtual environment.

A virtual environment creates a self-contained directory tree containing the Python installation for a particular version of Python, plus a number of additional packages. Different applications can use different virtual environments, with different versions of packages and modules installed.

Starting from version 3.3, Python comes with a virtual environment `venv`. To install `venv`, use:

```
1  # Install the virtual environment
2  cat@lubancat:~$ sudo apt-get install python3-venv
3
4  # Create and enter the project-test directory, and then create an
↳ independent Python operating environment: test_env
5  cat@lubancat:~$ mkdir project-test && cd project-test
6  cat@lubancat:~/project-test$ python3 -m venv .test_env
7
8  cat@lubancat:~/project-test$ ls -al
9  total 12
10 drwxr-xr-x  3 cat cat 4096 Dec  7 10:38 .
11 drwxr-xr-x 17 cat cat 4096 Dec  7 10:38 ..
12 drwxr-xr-x  6 cat cat 4096 Dec  7 10:38 .test_env
13 cat@lubancat:~/project-test$
14
15 # Activation into the environment
16 cat@lubancat:~/project-test$ source .test_env/bin/activate
17 (.test_env) cat@lubancat:~/project-test$
18 (.test_env) cat@lubancat:~/project-test$ pip list
19 Package          Version
20 -----
21 pip               18.1
22 pkg-resources     0.0.0
23 setuptools       40.8.0
24 (.test_env) cat@lubancat:~/project-test$
25
26 # Export the package and module version numbers of the current virtual
↳ environment
27 (.test_env) cat@lubancat:~/project-test$ pip3 freeze > requirements.txt
28 # Package and module version numbers for installation requirements
29 (.test_env) cat@lubancat:~/project-test$ pip3 install -r requirements.txt
```

(下页继续)

(续上页)

```
30
31 # Exit environment
32 (.test_env) cat@lubancat:~/project-test$ deactivate
33 cat@lubancat:~/project-test$
```

For more knowledge about virtual environments, you can search for the keywords virtualenv, venv and pipenv.

4.8 Reference documents

For more basic and detailed reference: <https://docs.python.org/3.8/tutorial/index.html>

<https://www.runoob.com/python3/python3-basic-syntax.html>

Chapter5 Control GPIOs

重要: If there is no special mention, the tutorials in this book are based on the Python 3.8.10 version (extboot partition Ubuntu20.04 image) for experiments and explanations.

There is no difference between developing pure software Python applications on Lubancat boards and PCs. The following chapters focus on how to use Python to control peripheral interfaces such as GPIO, PWM, ADC, I2C, and SPI that are rarely used in pure PC programming.

When controlling these peripheral interfaces, the following Python packages are mainly used:

- [python3-libgpiod](#): The python version of the standard GPIO libgpiod library, which only supports controlling IO input and output.
- [python-periphery](#): Support basic control of GPIO, PWM, I2C, SPI, UART and other interfaces.
- [Adafruit Blinka](#): It supports GPIO, PWM, I2C, SPI, UART, etc., and also has some application examples of commonly used sensors and OLED screens.

They have their own characteristics, you can try them all.

5.1 libgpiod basic concept

GPIO is mainly used to output high and low levels externally. When controlling GPIO, it will basically involve the control of libgpiod. We mainly need to know the naming method of the board pins.

The GPIO pins of the CPU are named using `(chip, line)`, and you can view them using the following command:

```
# Execute the following command on the board
sudo gpioinfo

# If it prompts that the command cannot be found, use the following method_
↳to install
sudo apt -y install gpiod libgpiod-dev

# The following is the output of gpioinfo
gpiochip0 - 32 lines:
    line   0:      unnamed      unused   input   active-high
    line   1:      unnamed      unused   input   active-high
    line   2:      unnamed      unused   input   active-high
    line   3:      unnamed      unused   input   active-high
    line   4:      unnamed      unused   input   active-high
    line   5:      unnamed "headset_gpio" input active-high [used]
# ...

    line  31:      unnamed      unused   input   active-high
gpiochip1 - 32 lines:
    line   0:      unnamed      unused   input   active-high
    line   1:      unnamed      unused   input   active-high
# ...

    line  31:      unnamed      unused   input   active-high
```

For the correspondence between the pin headers and GPIO (chip, line) drawn from the board, please refer to the specific instructions of the board.

For a more detailed description of libgpiod, please refer to the description on the right: [“Controlling IO with libgpiod”](#)

5.2 Experiment preparation

Some GPIOs on the board may be used for other functions, you can comment out the loading of some device tree nodes or device tree plug-ins, restart the system, and release the corresponding GPIO pins. Of course, you can also change the pin test. The pins of LubanCat_RK series boards are as follows: “[LubanCat-RK Series-40pin Pin Mapping](#)”

Most of the functions of operating hardware peripherals almost require the root user authority. The simple solution is to add sudo before executing the statement or run the program as the root user.

5.3 Method 1: Use python3-libgpiod

5.3.1 Install python3-libgpiod

The [python3-libgpiod](#) package makes it easy to control GPIO pins using Python. Currently [python3-libgpiod](#) has no official documentation, you can use help to view the help after importing the package. Its installation and viewing help are as follows:

```
# Use the following command to install on the board
sudo apt -y install python3-libgpiod

# Enter python exchange mode, test and view help
python3
import gpiod
help(gpiod)

# The following is the help description of the output
NAME
    gpiod - Python bindings for libgpiod.

DESCRIPTION
```

(下页继续)

(续上页)

```
This module wraps the native C API of libgpiod in a set of python_
↪classes.
# ...
```

5.3.2 libgpiod output

The sample code for controlling GPIO lighting (output) using this [python3-libgpiod](#) package is as follows:

列表 1: Companion ‘code io/gpio/libgpiod_io0.py’ file content

```
1 import time
2 import gpiod
3
4 # Modify the Chip and Line used according to the LED light connection of_
↪the specific board. If there is no LED, you can connect it externally
5 LINE_OFFSET = 8
6
7 chip0 = gpiod.Chip("0", gpiod.Chip.OPEN_BY_NUMBER)
8
9 gpio0_b0 = chip0.get_line(LINE_OFFSET)
10 gpio0_b0.request(consumer="gpio", type=gpiod.LINE_REQ_DIR_OUT, default_
↪vals=[0])
11
12 print(gpio0_b0.consumer())
13
14 try:
15     while True:
16         gpio0_b0.set_value(1)
17         time.sleep(0.5)
18         gpio0_b0.set_value(0)
```

(下页继续)

(续上页)

```
19         time.sleep(0.5)
20 finally:
21     gpio0_b0.set_value(1)
22     gpio0_b0.release()
```

Code description:

- On line 7, a `gpiod.Chip` object `chip0` with a chip ID of 0 is created
- Line 9, set to use line8 of the `chip0` object as a pin
- Line 10, apply for `gpio`, set it as output, and output low level by default

The following code directly uses the `gpio0_b0` object to control the selected GPIO output high and low levels, so as to control the LED light on and off.

This example and the following content all use the LubanCat 2 board as the experimental object, so (chip, line)=(0, 8) in the code is selected according to the pin definition of the board. If you use other boards, you only need to modify the corresponding number.

5.3.2.1 Experimental operation

The sample code uses the LubanCat 2 board, the source code can refer to the supporting routine, the operation is as follows:

```
# Confirm that the python3-libgpiod package is installed
# Execute the following command in the directory where the board blink.py
→is located, and root permission is required
sudo python3 blink.py

# If an LED is connected to this pin, the light will blink
```

Use another terminal, use the `gpioinfo` command, you can see that the `gpio (0, 8)` pin has been applied for use:


```
cat@lubancat:~$ sudo gpioinfo
gpiochip0 - 32 lines:
  line 0:      unnamed      unused      input      active-high
  line 1:      unnamed      unused      input      active-high
  line 2:      unnamed      unused      input      active-high
  line 3:      unnamed      unused      input      active-high
  line 4:      unnamed      unused      input      active-high
  line 5:      unnamed      "headset_gpio" input      active-high [used]
  line 6:      unnamed      unused      input      active-high
  line 7:      unnamed      unused      input      active-high
  line 8:      unnamed      "gpio"      output      active-high [used]
  line 9:      unnamed      unused      input      active-high
  line 10:     unnamed      unused      input      active-high
  line 11:     unnamed      unused      input      active-high
  line 12:     unnamed      unused      input      active-high
  line 13:     unnamed      unused      input      active-high
  line 14:     unnamed      unused      input      active-high
```

5.3.3 libgpiod input and output

Similarly, the sample code for detecting GPIO input (keypress) using this [python3-libgpiod](#) package is as follows:

列表 2: Companion code ‘io/gpio/libgpiod_io1.py’ file content

```
1 import gpiod
2
3 # Modify the Chip and Line used according to the LED light and button_
  ↳ connection of the specific board
4
5 # Here take LubanCat 2 as an example, use GPIO0_B0 to connect to LED, and_
  ↳ GPIO0_C2 to connect to button
6 LED_LINE_OFFSET = 8
7 BUTTON_LINE_OFFSET = 18
8
9 chip0_led = gpiod.Chip("0", gpiod.Chip.OPEN_BY_NUMBER)
10 chip0_button = gpiod.Chip("0", gpiod.Chip.OPEN_BY_NUMBER)
```

(下页继续)

(续上页)

```
11
12 led = chip0_led.get_line(LED_LINE_OFFSET)
13 led.request(consumer="LED", type=gpio.LINE_REQ_DIR_OUT, default_vals=[0])
14
15 button = chip0_button.get_line(BUTTON_LINE_OFFSET)
16 button.request(consumer="BUTTON", type=gpio.LINE_REQ_DIR_IN)
17
18 print(led.consumer())
19 print(button.consumer())
20
21 try:
22     while True:
23         led.set_value(button.get_value())
24 finally:
25     led.set_value(1)
26     led.release()
27     button.release()
```

Code description:

- Line 12, set the GPIO control direction of the LED to output
- Line 15, set the GPIO control direction of the button as input
- Line 23, read the input value of the button pin to control the LED

5.4 Method 2: Use python-periphery

5.4.1 Install python-periphery

`python-periphery` is functionally similar to `python3-libgpiod`. However, in addition to supporting GPIO input and output control, the periphery also supports bus protocols such as I2C and SPI.

`python-periphery` is installed as follows:

```
# Use the following command to install on the board
sudo pip3 install python-periphery
```

5.4.2 peripheral input and output

The sample code for input and output using this `python-periphery` is as follows:

列表 3: Companion code ‘io/gpio/periphery_io.py’ file content

```
1 from periphery import GPIO
2
3 # Modify the Chip and Line used according to the LED light and button_
  ↳ connection of the specific board
4 # Here take LubanCat 2 as an example, use GPIO0_B0 to connect to LED, and_
  ↳ GPIO0_C2 to connect to button
5 LED_CHIP = "/dev/gpiochip0"
6 LED_LINE_OFFSET = 8
7
8 BUTTON_CHIP = "/dev/gpiochip0"
9 BUTTON_LINE_OFFSET = 18
10
11 led = GPIO(LED_CHIP, LED_LINE_OFFSET, "out")
12 button = GPIO(BUTTON_CHIP, BUTTON_LINE_OFFSET, "in")
13
14 try:
15     while True:
16         led.write(button.read())
17 finally:
18     led.write(True)
```

(下页继续)

(续上页)

```
19 led.close()  
20 button.close()
```

Code description:

- Lines 5 to 9 define the chip and line numbers of LEDs and buttons
- Lines 11~12 create the GPIO output and input objects of led and button respectively
- Line 16, read the input value of the button pin to control the LED

The experimental operation is the same as the previous section. Use another terminal, use the `gpioinfo` command, you can see that two pins have been applied for use:

```
cat@lubancat:~$ sudo gpioinfo  
gpiochip0 - 32 lines:  
  line 0:      unnamed      unused   input   active-high  
  line 1:      unnamed      unused   input   active-high  
  line 2:      unnamed      unused   input   active-high  
  line 3:      unnamed      unused   input   active-high  
  line 4:      unnamed      unused   input   active-high  
  line 5:      unnamed "headset_gpio" input   active-high [used]  
  line 6:      unnamed      unused   input   active-high  
  line 7:      unnamed      unused   input   active-high  
  line 8:      unnamed "periphery" output  active-high [used]  
  line 9:      unnamed      unused   input   active-high  
  line 10:     unnamed      unused   input   active-high  
  line 11:     unnamed      unused   input   active-high  
  line 12:     unnamed      unused   input   active-high  
  line 13:     unnamed      unused   input   active-high  
  line 14:     unnamed      unused   input   active-high  
  line 15:     unnamed      unused   input   active-high  
  line 16:     unnamed      unused   output  active-high  
  line 17:     unnamed      unused   output  active-high  
  line 18:     unnamed "periphery" input   active-high [used]  
  line 19:     unnamed      unused   input   active-high  
  line 20:     unnamed      unused   input   active-high  
  line 21:     unnamed      unused   input   active-high  
  line 22:     unnamed      unused   input   active-high
```

5.5 Method 3: Using Adafruit Blinka

5.5.1 Installing Adafruit Blinka

Adafruit Blinka is similar to [python-periphery](#) in that it not only supports GPIO input and output control, but also supports I2C, SPI and other bus protocols. But Adafruit also provides richer application demos based on blinka, such as controlling the screen and so on.

Adafruit Blinka is installed as follows:

```
# Use the following command to install on the board
sudo apt -y install python3-libgpiod # Note: If it has been installed in
↪method 1, ignore this step
sudo pip3 install Adafruit-Blinka
```

5.5.2 Read before use

The Lubancat RK series boards have been adapted to the Adafruit-Blinka library, but the available resources of the Adafruit-Blinka library on different boards are different. According to the Lubancat board in your hand, you can modify the `resource name` used in the corresponding sample program to complete the corresponding experiment.

It should be noted that some boards have limited resources, so the Adafruit-Blinka library function cannot support all boards. If you want to know whether your board supports some functions, you can query the board resource definition. The reference is as follows:

Make sure the Adafruit-Blinka library is installed and the board can be detected before querying.

1. Enter `python3` to enter the python3 terminal.
2. Output `from adafruit_platformdetect import Detector` to add `adafruit_platformdetect` package;

3. Enter `detector = Detector()` followed by `print("Chip id: ", detector.chip.id)` and `print("Board id: ", detector.board.id)` Detect board and chip id;
4. In the python3 terminal, enter `import board` to import the board package.
5. Enter `board.` and double-click the Tab key to display the completion information, which contains the resource content defined by the board.

lubancat-zero series:

```
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3566
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT_ZERO
>>>
```

LubanCat 1 series:

```
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3566
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT1
>>>
```

LubanCat 2 series:

```
cat@lubancat:~$ python
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3568
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT2
>>>
>>> import board
>>> board.
board.CS0      board.GPIO12  board.GPIO18  board.GPIO23  board.GPIO28  board.GPIO32  board.GPIO37  b
board.CS1      board.GPIO13  board.GPIO19  board.GPIO24  board.GPIO29  board.GPIO33  board.GPIO38  b
board.GPIO10   board.GPIO15  board.GPIO21  board.GPIO26  board.GPIO3   board.GPIO35  board.GPIO40  b
board.GPIO11   board.GPIO16  board.GPIO22  board.GPIO27  board.GPIO31  board.GPIO36  board.GPIO5   b
>>> board.
```

Note that the resources queried here are only the resources we added to adapt the Adafruit-Blinka library. Whether the specific resources can be used depends on the board situation, and you can also try it yourself.

提示: The added board resource is the 40pin lead out, and the setting rule is which pin corresponds to which GPIO. For example, GPIO3 corresponds to the third physical pin of 40pin. If the physical pin is grounded or there is no power supply, for example, there is no GPIO1.

The following formally enters the experimental part.

5.5.3 Adafruit-Blinka input and output

The sample code for using this [Adafruit Blinka](#) for input and output is as follows (Here, LubanCat 1 is used as an example, other series need to modify the pins by themselves, etc.):

列表 4: Companion code ‘io/gpio/digital_io.py’ file content

```
1 import board
2 import digitalio
3
4 # Modify the GPIO used according to the specific board
5 # Take LubanCat 1 board as an example, board.GPIO11 is the 11th physical_
  ↳ pin of 40Ppin, board.GPIO11 = GPIO3_A5 = Pin 101
6 led = digitalio.DigitalInOut(board.GPIO11)
7 led.direction = digitalio.Direction.OUTPUT
8
9 try:
10     while True:
11         led.value=1
12         time.sleep(0.5)
13         led.value=0
14         time.sleep(0.5)
```

(下页继续)

(续上页)

```
15
16 finally:
17     led.value = True
18     led.deinit()
```

Code description:

- Lines 1~2, board and digitalio are libraries provided by the Blinka package.
- Lines 6~7 define the pin board.GPIO11 used and set it as the output direction.
- Lines 9-14 make the pin output high and low levels.

The board.GPIO11 pin in the code is the LubanCat 1 pin defined in the Blinka library, use the command:

```
sudo python digital_io.py
```

If an LED light is connected to the corresponding pin, you can see the LED flashing.

When we apply for the corresponding board resource, it will detect the board model, and then load the corresponding resource definition according to the board information. So we can directly use the defined resource name, which is more intuitive to use than [python-periphery](#).

More detailed pin definitions can be viewed directly in [Source code for Adafruit Blinka](#) :

- [Pin definition of LubanCat series board in Adafruit Blinka](#)
- [Pin definition of LubanCat RK series chip in Adafruit Blinka](#)

5.6 References

In fact, there are more Python libraries that support peripheral hardware control. If you are interested, you can find them in [github](#) or [pypi](#).

- [《CircuitPython》](#)



- [python-periphery source code](#)
- [Adafruit Blinka source code](#)

Chapter6 PWM output

In the previous chapters, we wrote Python applications for operating GPIO peripherals through the Python libraries [python3-libgpiod](#) and [python-periphery](#). Finally, on the Lubancat board, the LED light on the board and the button resources are used through the Python application program. Then in this section, we will then introduce another function provided by the above library, PWM waveform output.

In this experiment, GPIO pins will be used to control LED lights for PWM output. At that time, you can view the experimental phenomenon through the light and dark changes of the LED light (the board may need an external LED).

6.1 PWM experiment

By controlling the high and low level changes introduced by GPIO, we can do some simple controls, such as controlling the on and off of LED lights. If we further adjust the frequency and duration of GPIO pin level changes, then based on this, the control we can do can become more complicated, which is PWM (Pulse Width Modulation).

Usually we can use PWM waveform output to achieve many functions, such as controlling the brightness of LED lights, the loudness of passive buzzers, and even controlling small servos. In this section of the experiment, we control the LED lights to achieve the effect of LED breathing lights by outputting PWM (take the LubanCat 2 board as an example, with external LEDs).

Regarding how PWM works, we will not do too much research here, and you can search for the principle of PWM by yourself. The focus of this section is to provide examples of PWM output, of course, this is inseparable from the Python library we introduced earlier.

重要: If there is no special mention, the tutorials in this book are based on the Python 3.8.10 version

(extboot partition Ubuntu20.04 image) for experiments and explanations.

6.2 Experiment preparation

Some GPIOs on the board may be occupied by the system, you can comment out the loading of some device tree nodes or device tree plug-ins, restart the system, and release the corresponding GPIO pins. Of course, you can also change the pin test. The pins of LubanCat_RK series boards are as follows: [“LubanCat-RK Series-40pin Pin Mapping”](#)

If `Permission denied` or similar words appear, please pay attention to user permissions. Most of the functions of operating hardware peripherals almost require root user permissions. The simple solution is to add `sudo` or run the program as root before executing the statement.

How to enable the pwm device tree plugin (take LubanCat 2 as an example):

```
# The configuration files of different boards are different from pwm,
↳ taking LubanCat 2 as an example, the configuration file is uEnvLubanCat 2.
↳ txt,
# Can turn on pwm8, pwm9, pwm10, pwm14
# Log in to the system terminal and open the '/boot/uEnv/uEnvLubanCat 2.txt
↳ ' file

sudo vim /boot/uEnv/uEnvLubanCat 2.txt
#Enter edit mode, cancel the comment in front of pwm, save and exit the
↳ file, restart the system.
```

```
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

#dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo

#overlay_end
```

取消前面的#注释，
开启pwm

```
# Enter the following command in the terminal to view the PWM resources of
the current board:
ls /sys/class/pwm/
```

The PWM resource example on the board is for reference only:

```
cat@lubancat:~$ ls /sys/class/pwm/
pwmchip0 pwmchip1 pwmchip2 pwmchip3 pwmchip4
cat@lubancat:~$
cat@lubancat:~$
cat@lubancat:~$
```

pwm8 pwm9 pwm10

pwmchip0 is pwm4, which is used for backlight adjustment. The pwmchip1 in the back of the picture is added when we open the device tree, corresponding to pwm8 on the hardware, and so on.

6.3 Use python-periphery

The PWM output supported by the [python-periphery](#) library is implemented based on the PWM subsystem of Linux, so if you want to use this library for PWM output, you need the board to provide support. Like the Lubancat board, you can perfectly use the [python-periphery](#) library to achieve PWM output. In this way,

we don't need to use GPIO to simulate PWM output at the software level.

For the content related to the PWM subsystem, refer to “PWM Control”。

6.3.1 Install python-periphery

重要: Be sure to skip this if you installed the library in the previous section.

`python-periphery` is installed as follows:

```
# Use the following command to install on the board
sudo pip3 install python-periphery
```

6.3.2 peripheral output PWM

The sample code for PWM output using `python-periphery` library is as follows:

列表 1: Supporting code ‘io/pwm/pwm_test_periphery.py’
file content

```
1 from periphery import PWM
2 import time
3
4 try:
5     # Define the duty cycle increment step size
6     step = 0.05
7     # Define the maximum range of range
8     rangeMax = int(1/0.05)
9     # Turn on PWM 8, channel 0, which corresponds to the PWM8 peripheral on
    ↳ the development board
10    pwm = PWM(1, 0)
```

(下页继续)

(续上页)

```
11     # Set the PWM output frequency to 1 kHz
12     pwm.frequency = 1e3
13     # Set the duty cycle to 0%, the ratio of the high level time in one_
↪cycle to the entire cycle time
14     pwm.duty_cycle = 0.00
15     # Turn on the PWM output
16     pwm.enable()
17     while True:
18         for i in range(0, rangeMax):
19             # sleep step seconds
20             time.sleep(step)
21             # Set the duty cycle to add step% each time, use round to avoid_
↪floating-point calculation errors
22             pwm.duty_cycle = round(pwm.duty_cycle+step, 2)
23             # Always off for 1 second
24             if pwm.duty_cycle == 0.0:
25                 time.sleep(1)
26             for i in range(0, rangeMax):
27                 time.sleep(step)
28                 pwm.duty_cycle = round(pwm.duty_cycle-step, 2)
29 except:
30     print("Some errors occur!\n")
31 finally:
32     # Turn off the LED on exit
33     pwm.duty_cycle = 0.0
34     # Release resources
35     pwm.close()
```

Code description:

- Line 10, a PWM object is created
- Lines 12 and 14 initialize the parameters of the PWM object respectively, which are the frequency

and duty cycle of the PWM output waveform

- Line 16, enable PWM waveform output according to the corresponding parameters

提示: If you are not familiar with the use of some python modules of PWM, you can enter the python terminal interactive mode and use the commands **from periphery import PWM** and **help(PWM)** to view the help about PWM modules.

6.3.2.1 Experimental operation

The sample code uses the LubanCat 2 board, and the operation is as follows:

```
# Confirm that the python-periphery package is installed
# Enable the PWM device tree plug-in, confirm the enabled pwm

# Execute the following command in the directory where the board pwm_test.
# py is located
sudo python3 pwm_test.py
```

If you are prompted about permission issues, you can switch to the root user and run:

```
#Switch to root, is the password root
su root
```

If you connect an external LED, you can see that the onboard LED lights will change in brightness, or you can view the waveform through an oscilloscope.

Chapter7 UART communication

In this chapter, we begin to introduce a common communication protocol UART. On our Lubancat board, we often use the terminal to interact with the board. Among the many ways to connect the terminal to the board, the UART serial port is one of the most commonly used ways.

Regarding the specific content of the UART protocol, we will not explore too much here, and you can search for the relevant knowledge of the UART communication protocol to learn.

In this chapter, our focus is to use the previous Python library to call the serial port resources on the board. In terms of hardware, it is connected to our computer through the board and USB to TTL module, and the demonstration and operation of the experimental phenomenon are performed through the serial port host computer.

重要: If there is no special mention, the tutorials in this book are based on the Python 3.8.10 version (extboot partition Ubuntu20.04 image) for experiments and explanations.

7.1 Experiment preparation

7.1.1 Add UART resource

Some GPIOs on the board may not be enabled, you can comment out the loading of some device tree nodes or device tree plug-ins, and restart the system to enable them. The pin reference of LubanCat_RK series boards is as follows: “[LubanCat-RK Series-40pin Pin Comparison Diagram](#)”

For example, for the LubanCat 2 board used by the author, according to the 40 pin comparison diagrams, the serial port resources can use UART3. Let's open the UART3 device tree plug-in:


```
# The configuration files of different boards are different from pwm,
↳taking LubanCat 2 as an example, the configuration file is uEnvLubanCat 2.
↳txt
# Use the following command:
sudo vim /boot/uEnv/uEnvLubanCat 2.txt
#Enter edit mode, cancel the comment in front of pwm, save and exit the
↳file, restart the system
```

```
root@lubancat:/boot/uEnv# sudo vim uEnvLubanCat2.txt
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

#dtoverlay=dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-spi3-m1-overlay.dtbo
[dtoverlay=dtb/overlay/lubancat-uart3-m1-overlay.dtbo] 取消#注释, 保存重启, 开启串口
#overlay_end

~
~
~
~
"uEnvLubanCat2.txt" 19L, 610C                               16,1      All
```

If you are using other Lubancat RK series boards, the operation is similar.

```
# Enter the following command in the terminal to view the UART resource:
ls /dev/ttyS*
```

The UART resource example on the board is for reference only:

```
cat@lubancat:~$ ls /dev/ttyS*
/dev/ttyS3
cat@lubancat:~$ █
```

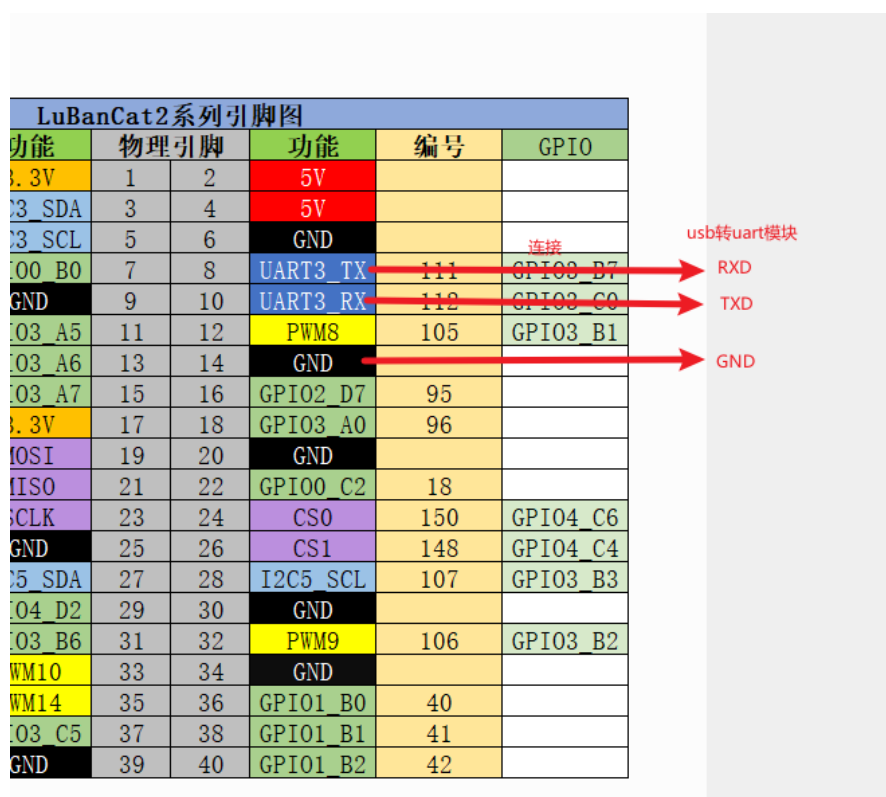
Among them, /dev/ttyS3 corresponds to the uart3 resource on the board.

7.1.2 Hardware connection

Note: In the hardware connection section, you need to make reference adjustments to the content of this chapter according to your own development environment and other conditions.

In this chapter, we use the above Python library to write the code to use the UART3 resource on the LubanCat 2 board. And through the USB to TTL module, the serial port host computer to test the correctness of the code.

The author's hardware connection is shown in the figure, for reference only:



The USB to TTL module is connected as shown above.

7.2 Method 1: Use the pyserial library

The [pyserial](#) library encapsulates the access methods to serial port resources, and this library is compatible with the use of serial port resources on various platforms. There are many methods related to platform features, official usage instructions refer to: [pyserial](#)

On the Linux system, for the content related to the UART subsystem, refer to “[Serial Port Communication](#)”

。

7.2.1 Install pyserial

重要： In the experiment in the previous section, this library may have been installed, if it is installed, skip this operation.

[pyserial](#) is installed as follows:

```
# Use the following command to install on the board
sudo pip3 install pyserial
```

7.2.2 Use pyserial

列表 1: Supporting code ‘io/uart/uart_test0.py’ file content

```
1 """ pyserial uart test """
2 import serial
3
4 # Open uart3, set the serial port baud rate to 115200, data bits to 8, no-
  ↳ parity bit, stop bit to 1, no flow control, open the serial port in non-
  ↳ blocking mode, and wait for 3s
```

(下页继续)

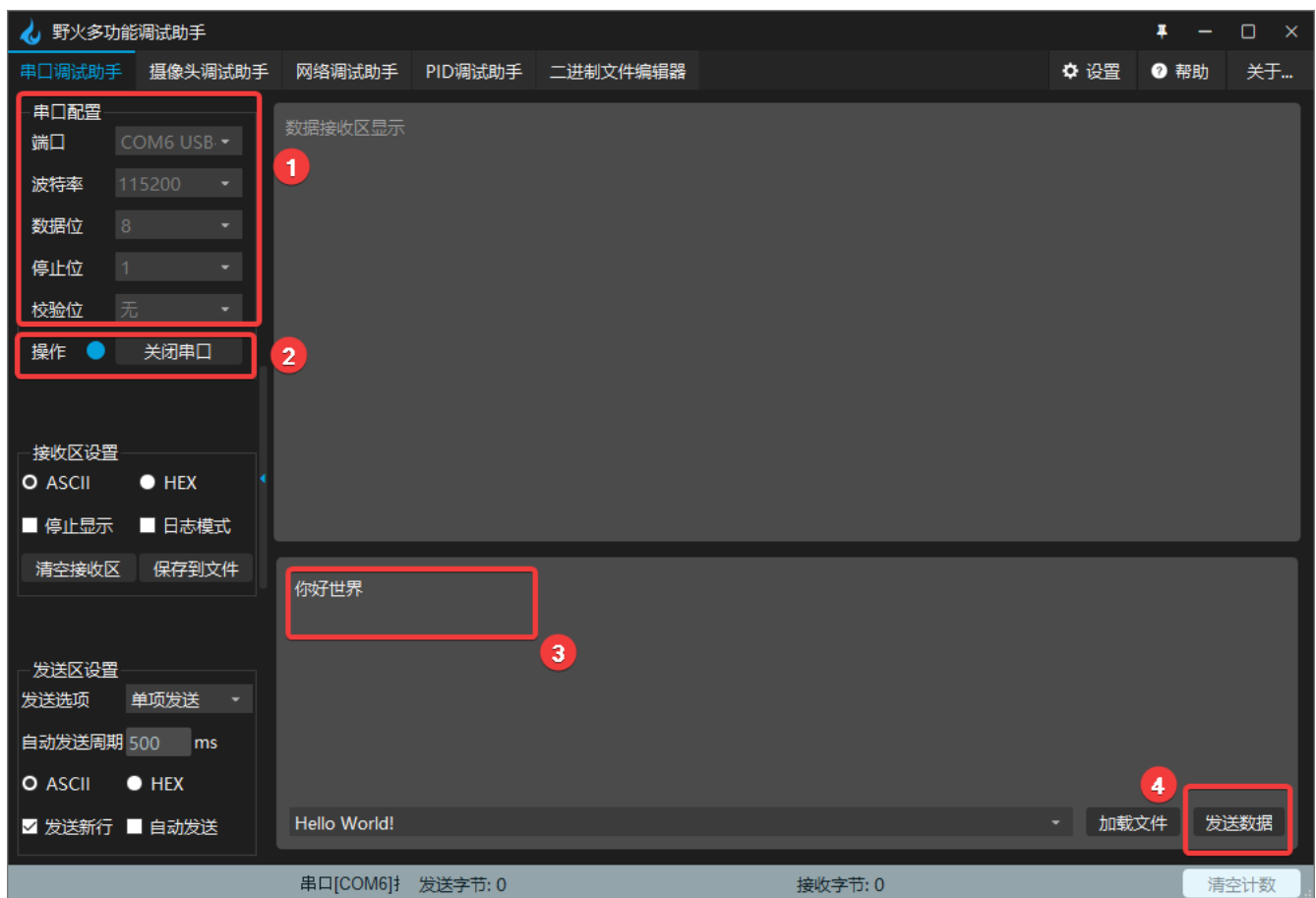
(续上页)

```
5 with serial.Serial(  
6     "/dev/ttyS3",  
7     baudrate=115200,  
8     bytesize=serial.EIGHTBITS,  
9     stopbits=serial.STOPBITS_ONE,  
10    parity=serial.PARITY_NONE,  
11    timeout=3,  
12 ) as uart3:  
13     # Use the requested serial port to send byte stream data "Hello World!\n  
↪ "  
14     uart3.write(b"Hello World!\n")  
15  
16     # The serial port opened in a non-blocking way, when reading the data  
↪ received by the serial port, the function returns the condition that one  
↪ of the two conditions is satisfied. First, read 128 bytes. and second,  
↪ the reading time exceeds 1 second  
17     buf = uart3.read(128)  
18  
19     # Note: The data type read by Python is: bytes  
20     # Print raw data  
21     print("Raw data:\n", buf)  
22     # Transcode to gbk string, can display Chinese  
23     data_strings = buf.decode("gbk")  
24     # Print the read data volume and data content  
25     print("{:d} bytes read, print as string:\n {}".format(len(buf), data_  
↪ strings))
```

7.2.2.1 Experimental operation

Since the TX\RX interface corresponding to uart3 has been connected to the RXD\TXD interface of the USB to TTL module during the hardware connection. Therefore, to view the experimental phenomenon, it is necessary to use the serial port host computer to display the data to facilitate the observation of the experimental phenomenon.

When using the Embedfire multi-function debugging assistant, it comes with a serial port host computer.



As shown in the figure, we set the working mode of the serial port in the serial port assistant to be consistent with our program configuration, and open the corresponding port, that is, steps 1 and 2 in the figure. Next we run the program.

The sample code uses the LubanCat i.MX6ULL MINI board, the operation is as follows:

```
# Confirm that the pyserial package is installed
# Confirm that the UART device tree plugin is enabled

# Execute the following command in the directory where the board uart_test.
  ↳ py is located
sudo python3 uart_test.py

# You can see the programmed data in the program printed by the serial port.
  ↳ assistant: Hello World!\n
```

After entering the command in the terminal, we edit the data to be sent in the serial port assistant and send it, that is, steps 3 and 4 in the figure.

After waiting for a timeout, you can see the received data printed out by the terminal: hello world



The screenshot shows the '野火多功能调试助手V1.3' (Fire Multi-functional Debug Assistant V1.3) interface. The '串口调试功能' (Serial Port Debug Function) tab is active. The configuration panel on the left shows: 端口 (COM6), 波特率 (115200), 校验位 (None), 数据位 (8), 停止位 (1). The '显示串口接收到的数据' (Display serial port received data) tab is selected on the right, showing 'Hello World!' in a red box. The terminal window on the left shows the command 'sudo python3 uart_test.py' and the output '原始数据: b'\xc4\xe3\xba\xc3\r\n', 读取到 6 个字节, 以字符串形式打印: 你好' (Original data: b'\xc4\xe3\xba\xc3\r\n', read 6 bytes, printed as string: 你好). The '你好' (Hello) is also shown in a red box in the terminal output.

For more usage of this library, refer to: [pyserial API](#).

7.3 Method 2: Use python-periphery

The UART function supported by the `python-periphery` library is implemented based on the Linux UART system, so if you want to use this library to use the UART function, you need the board to provide support. Like the Lubancat board, you can perfectly use the `python-periphery` library UART communication function.

7.3.1 Install python-periphery

重要: Be sure to skip this if you installed the library in the previous section.

`python-periphery` is installed as follows:

```
# Use the following command to install on the board
sudo pip3 install python-periphery
```

7.3.2 The periphery uses the UART function

The sample code for using the `python-periphery` library for the UART function is as follows:

列表 2: Supporting code ‘io/uart/uart_test1.py’ file content

```
1  """ python-periphery uart test """
2  from periphery import Serial
3
4  try:
5      # Apply for the serial port resource /dev/ttyS3, set the serial port
6      ↪ baud rate to 115200, data bits to 8, no parity bit, stop bit to 1, no
7      ↪ flow control
8      serial = Serial(
```

(下页继续)

(续上页)

```
7         "/dev/ttyS3",
8         baudrate=115200,
9         databits=8,
10        parity="none",
11        stopbits=1,
12        xonxoff=False,
13        rtscts=False,
14    )
15    # Use the requested serial port to send byte stream data "python-
    ↳periphery!\n"
16    serial.write(b"python-periphery!\n")
17
18    # To read the data received by the serial port, the function returns
    ↳the condition that one of the two conditions is satisfied, first, 128
    ↳bytes are read, and second, the reading time exceeds 1 second
19    buf = serial.read(128, 1)
20
21    # Note: The data type read by Python is: bytes
22    # Print raw data
23    print("Received raw data:\n", buf)
24
25    # Transcode to gbk string, can display Chinese
26    data_strings = buf.decode("gbk")
27
28    # Print the read data volume and data content
29    print("Read up to {:d} bytes, print as string:\n {:s}".format(len(buf),
    ↳data_strings))
30    finally:
31        # Release the requested serial port resources
32        serial.close()
```

Code description:

- Line 6: Apply for UART resources, occupy uart3, and configure the corresponding working mode
- Line 16: Use the applied serial port resources to send data
- Line 19: Use the applied serial port resources to receive data, and the receiving method is blocking reception
- Lines 23~29: Print out the read data
- Line 32: Release UART resource, release uart3

7.3.2.1 Experimental operation

The experimental operation is the same as above.

The sample code uses the LubanCat 2 board, and the operation is as follows:

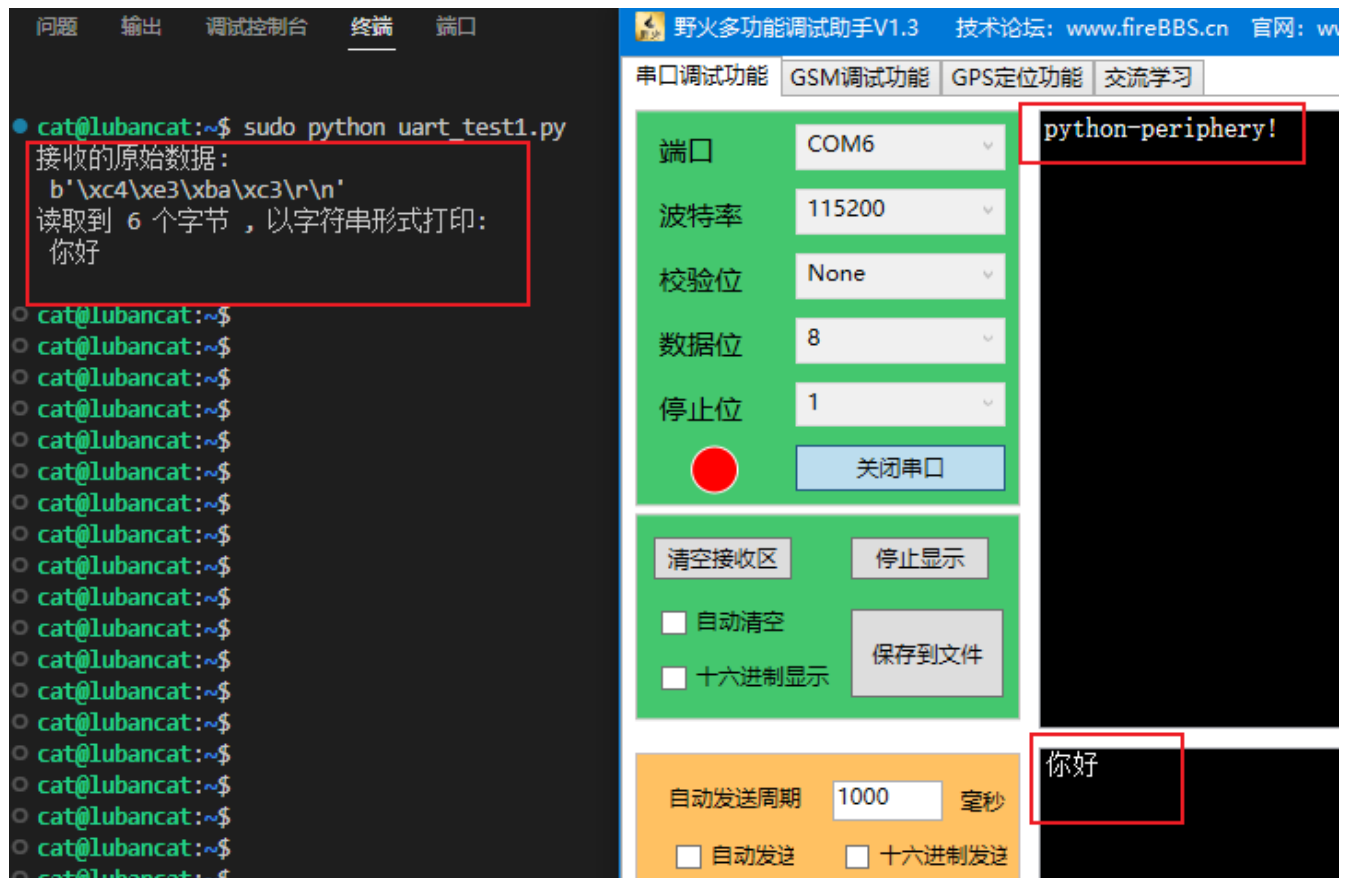
```
# Confirm that the python-periphery package is installed
# Confirm that the UART device tree plugin is enabled

# Execute the following command in the directory where the board uart_test.
↪py is located
sudo python3 uart_test1.py

# You can see the programmed data in the program printed by the serial port.
↪assistant: Hello World!\n
```

After entering the command in the terminal, we edit the data to be sent in the serial port assistant and send it, that is, steps 3 and 4 in the figure.

After waiting for a timeout, you can see the received data printed out by the terminal: Hello



Since then, the experiment has been completed.

For more UART usage of `python-periphery` library, pay attention to: [Periphery UART](#)

Chapter8 I2C communication

The I2C (IIC) protocol is a commonly used communication protocol in electronic devices, through which we can control various electronic devices (Note: generally controlled devices are slaves). For example, the common attitude sensor MPU6050, temperature image sensor MLX90640, 0.96-inch OLED display SSD1306, etc., all communicate with the board through the I2C communication protocol (note: generally initiate control for the host) to communicate.

Regarding the specific content of the I2C protocol, we will not explore too much here, and you can search for the relevant knowledge of the I2C communication protocol to learn.

In this chapter, our focus is to use the two Python libraries introduced earlier [python-periphery](#) and [Adafruit Blinka](#) to use the I2C host on our Lubancat board.

8.1 I2C experiment

In general, we need a specific communication object to complete the experiment in this section, that is, the various types of electronic devices mentioned above.

Considering that the modules or I2C devices in everyone's hands are different, the introduction to the use of the above Python library is divided into two parts.

One part is the introduction of the official demo of the library, and the other part is the introduction of using the corresponding API in the library and using it in the actual module.

The author is using 0.96-inch OLED display module SSD1306 [Embedfire \[OLED screen_I2C_0.96 inch\]](#) Write examples of using the above-mentioned libraries. In actual use, the type of devices that the I2C master on the Lubancat board can communicate with is not limited. Therefore, the author will make detailed comments on the relevant codes in the code. If you do not have the same modules in your hands, you can also perform similar operations to write experimental codes for different modules.

8.2 Experiment preparation

8.2.1 Add I2C resources to the board

The GPIO on the board may not be multiplexed as I2C, which can be loaded by setting the device tree plugin. The pin reference of LubanCat_RK series boards is as follows: “[LubanCat-RK Series-40pin Pin Mapping](#)”

Let's take LubanCat 2 as an example to open the i2c device tree plug-in (LubanCat 2 has two i2c pins):

```
# The configuration files of different boards are different from i2c.
↳ Taking LubanCat 2 as an example, the configuration file is uEnvLubanCat 2.
↳ txt
# I2c3, i2c5 can be turned on
# Log in to the system terminal and open the /boot/uEnv/uEnvLubanCat 2.txt
↳ file

sudo vim /boot/uEnv/uEnvLubanCat 2.txt
#Enter edit mode, cancel the comment in front of I2c3, save and exit the
↳ file, restart the system
```

```
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo

#overlay_end
```

取消前面#注释, 开启i2c3和i2c5, 并保存退出, 重启

uEnvLubanCat2.txt

After restarting, check whether the I2C resource of the board is loaded:

```
# Enter the following command in the terminal to view the I2C resources:
ls -l /dev/i2c-*
```

The I2C resource example on the board is for reference only:

```
cat@lubancat:~$ ls -l /dev/i2c-*
crw-rw---- 1 root i2c 89, 0 Nov 9 09:30 /dev/i2c-0
crw-rw---- 1 root i2c 89, 1 Nov 9 09:30 /dev/i2c-1
crw-rw---- 1 root i2c 89, 3 Nov 9 09:30 /dev/i2c-3
crw-rw---- 1 root i2c 89, 5 Nov 9 09:30 /dev/i2c-5
crw-rw---- 1 root i2c 89, 6 Nov 9 09:30 /dev/i2c-6
cat@lubancat:~$
```

For some simple i2c-related tests, please refer to [“I2C Communication”](#)

8.2.2 Hardware connection

Lubancat LubanCat 2 board, there is no I2C device on the board, and relevant experimental phenomena can be displayed. Therefore, the experimental phenomenon in this section is shown for you by connecting the peripheral device 0.96-inch OLED display SSD1306 at the hardware level, using I2c3.

The connection is shown in the figure, for reference only (I2c3 of LubanCat 2). The 40pin pin connection table of the oled screen and the board is as follows:

OLED screen pins	corresponding GPIO	LubanCat 2 board pins
SDA	GPIO1_A0	I2C3_SDA
SCL	GPIO1_A1	I2C3_SCL
GND	•	GND
VCC	•	3.3V

8.3 Use python-periphery

The I2C function supported by the [python-periphery](#) library is implemented based on the Linux I2C system, so if you want to use the library to use the I2C function, you need the board to provide support. Like the Lubancat board, you can perfectly use the I2C communication function of the [python-periphery](#) library. In this way, we don't need to use GPIO to simulate the I2C host function at the software level.

8.3.1 Install python-periphery

重要: Be sure to skip this if you installed the library in the previous section.

`python-periphery` is installed as follows:

```
# Use the following command to install on the board
sudo pip3 install python-periphery
```

8.3.2 The periphery uses the I2C function

First refer to the sample code for learning the `python-periphery` library for I2C function use:

列表 1: The official code example -i2c (learning reference)

```
1 from periphery import I2C
2
3 # Open i2c1 master
4 i2c = I2C("/dev/i2c-1")
5
6 # Take the EEPROM device as an example, read data from the EEPROM memory.
  ↳ address 0x100, and the I2C device address of the EEPROM device is 0x50
7 msgs = [I2C.Message([0x01, 0x00]), I2C.Message([0x00], read=True)]
8 # Enable I2C transmission
9 i2c.transfer(0x50, msgs)
10 # Print received message
11 print("0x100: 0x{:02x}".format(msgs[1].data[0]))
12 # When the operation is complete, shut down the I2C master to free up.
  ↳ resources
13 i2c.close()
```

Code description:

- Line 4: Apply for I2C resources and occupy the I2C1 host.
- Line 7: Construct the message structure `I2C.Message`. The first member of `I2C.Message` is the list of sent data. The second one specifies the function of this message. The default is I2C write command. In the case of write command, the data in the data list will be sent out. If `read=True` is specified, this

message corresponds to an I2C read command, and the read data will be stored in the data member.

- Line 9, start I2C transmission, send the address of the I2C slave device and the structured message structure.
- Line 17, release I2C resources, release I2C1 master.

Through the code examples of EEPROM device operation given in the [python-periphery](#) library, we can grasp the basic method of using the library for the I2C master. Then you can start programming and use it for specific devices.

Here the author uses the 0.96-inch OLED display SSD1306 module programming display described above:

列表 2: Companion code ‘io/i2c/i2c_test_periphery.py’ file
content

```
1  """ Periphery i2c test using 0.96 inch OLED module """
2  import time
3  from periphery import I2C
4
5  # Turn on the i2c-3 controller
6  i2c = I2C("/dev/i2c-3")
7
8  # Device slave address 0x3c, which is OLED module address
9  I2CSLAVEADDR = 0x3C
10
11 # Read function test using peripheral i2c library
12 def i2c_read_reg(devregaddr):
13     """
14     Read function test using peripheral i2c library
15     """
16     # Construct data structure
17     # The first Message is the address of the device to be read
18     # The second Message is used to store the read back message, pay
19     ↪ attention to the added read=True
```

(下页继续)

(续上页)

```
19     msgs = [I2C.Message([devregaddr]), I2C.Message([0x00], read=True)]
20     # Send message, send to i2cSlaveAddr
21     i2c.transfer(I2CSLAVEADDR, msgs)
22     print("read the register 0x{:02x} from: 0x{:02x}".format(devregaddr,
↪msgs[1].data[0]))
23
24
25 def oled_write_cmd(cmd):
26     """
27     Use the peripheral i2c library to send functional tests
28     """
29     msgs = [I2C.Message([0x00, cmd])]
30     i2c.transfer(I2CSLAVEADDR, msgs)
31
32
33 def oled_write_data(data):
34     """
35     Use the peripheral i2c library to send functional tests
36     """
37     msgs = [I2C.Message([0x40, data])]
38     i2c.transfer(I2CSLAVEADDR, msgs)
39
40
41 def oled_init():
42     """
43     Use OLED module for code function test 0.96 inch OLED module
↪initialization
44     """
45     time.sleep(1)
46     oled_write_cmd(0xAE)
47     oled_write_cmd(0x20)
48     oled_write_cmd(0x10)
```

(下页继续)

(续上页)

```
49     oled_write_cmd(0xB0)
50     oled_write_cmd(0xC8)
51     oled_write_cmd(0x00)
52     oled_write_cmd(0x10)
53     oled_write_cmd(0x40)
54     oled_write_cmd(0x81)
55     oled_write_cmd(0xFF)
56     oled_write_cmd(0xA1)
57     oled_write_cmd(0xA6)
58     oled_write_cmd(0xA8)
59     oled_write_cmd(0x3F)
60     oled_write_cmd(0xA4)
61     oled_write_cmd(0xD3)
62     oled_write_cmd(0x00)
63     oled_write_cmd(0xD5)
64     oled_write_cmd(0xF0)
65     oled_write_cmd(0xD9)
66     oled_write_cmd(0x22)
67     oled_write_cmd(0xDA)
68     oled_write_cmd(0x12)
69     oled_write_cmd(0xDB)
70     oled_write_cmd(0x20)
71     oled_write_cmd(0x8D)
72     oled_write_cmd(0x14)
73     oled_write_cmd(0xAF)
74
75
76 def oled_fill(filldata):
77     """
78     Clear the OLED screen
79     """
80     for i in range(8):
```

(下页继续)

(续上页)

```
81         oled_write_cmd(0xB0 + i)
82         # page0-page1
83         oled_write_cmd(0x00)
84         # low column start address
85         oled_write_cmd(0x10)
86         # high column start address
87         for j in range(128):
88             oled_write_data(filldata)
89
90
91 # Code testing
92 try:
93     # Initialize the OLED screen, SSD1306 is necessary
94     oled_init()
95     # Clear the OLED screen
96     oled_fill(0xFF)
97     # Read register test
98     i2c_read_reg(0x10)
99 finally:
100     print("Test ended normally")
101     # Release resources
102     i2c.close()
```

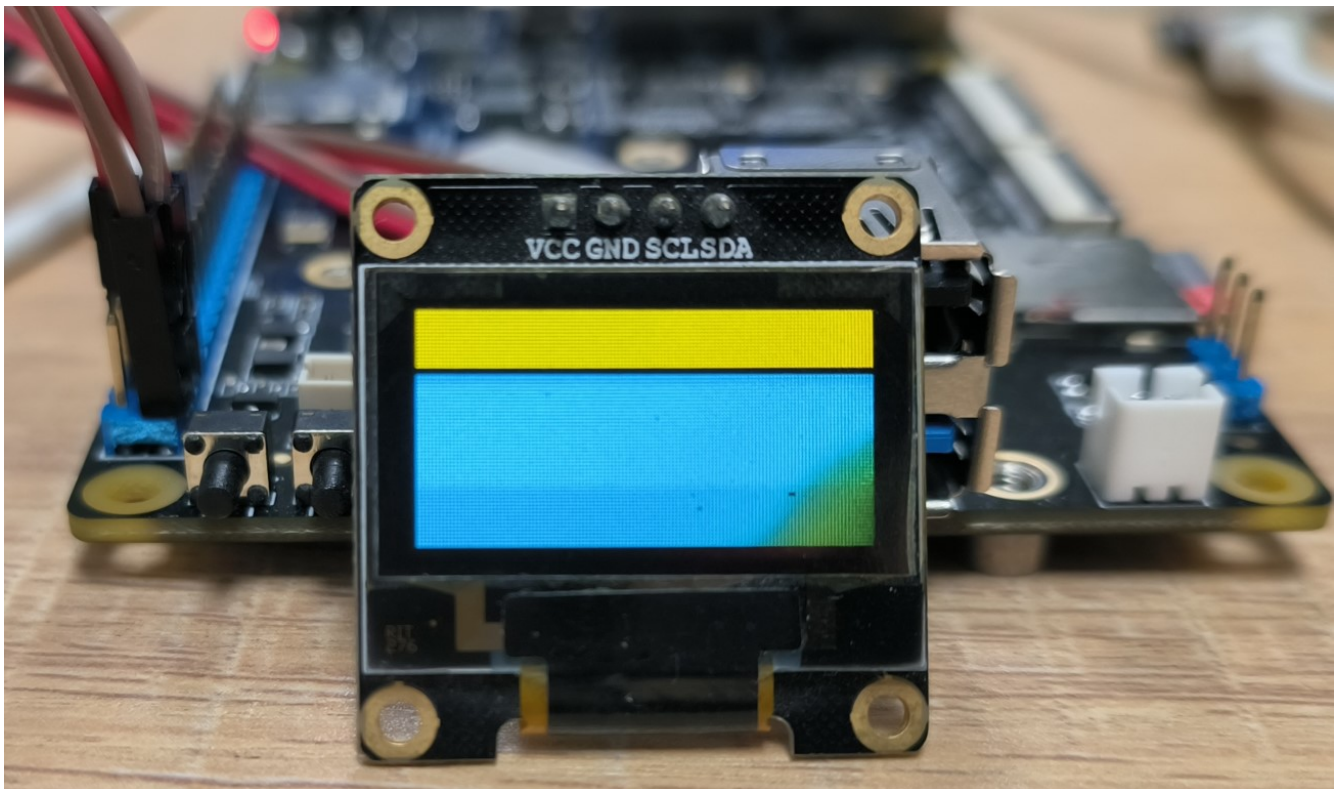
It is not the focus of this tutorial on what commands to send through the I2C host to control the 0.96-inch OLED display SSD1306 module. The purpose of the tutorial is to provide you with a reference object for the use of the API functions of the relevant library through the provided sample code.

8.3.2.1 Experimental operation

The sample code uses the LubanCat 2 board, and the operation is as follows:

```
# Execute the following command in the directory where the board i2c_test.  
→py is located  
sudo python3 i2c_test_periphery.py  
  
# It can be seen that the connected OLED screen is cleared to display color.  
→blocks, and the read register information is printed out in the terminal
```

Display reference (feel free to take an old screen for testing):



8.4 Method 2: Using Adafruit Blinka

Adafruit Blinka is similar to `python-periphery` in that the I2C communication is also implemented based on the I2C subsystem of Linux. It's just that there is a big difference in the usage of the code.

8.4.1 Installing Adafruit Blinka

重要: Be sure to skip this if you installed the library in the previous section.

Adafruit Blinka is installed as follows:

```
# Use the following command to install on the board
sudo apt -y install python3-libgpiod python-periphery # Note: If already
↪ installed, ignore this step
sudo pip3 install Adafruit-Blinka

sudo pip3 install machine
```

8.4.2 Adafruit - Blinka using I2C master

The sample code for input and output using this `Adafruit Blinka` is as follows:

列表 3: The official code example - i2c

```
1 import busio
2 from board import *
3
4 # Apply for I2C resources and occupy the I2C1 host
5 i2c = busio.I2C(SCL, SDA)
6 # Print the scanned slave devices mounted under the I2C1 host
```

(下页继续)

(续上页)

```
7 print(i2c.scan())
8 # The operation is complete, shut down the I2C master to release resources
9 i2c.deinit()
```

The I2C usage example provided by the [Adafruit Blinka](#) library is relatively simple. It only provides scanning devices mounted under the I2C1 host. For specific usage methods of other API functions, you can refer to: [Blinka I2C](#)

Here directly shows the program of using I2C1 host to communicate with 0.96-inch OLED display module SSD1306 module:

列表 4: Supporting code io/i2c/i2c_test_adafruit.py file content

```
1 """ blinka i2c test using 0.96 inch OLED module """
2 import board
3 import busio
4
5 # Initialize OLED command byte array
6 OledInitBuf = bytes(
7     [
8         0xAE,
9         0x20,
10        0x10,
11        0xB0,
12        0xC8,
13        0x00,
14        0x10,
15        0x40,
16        0x81,
17        0xFF,
18        0xA1,
```

(下页继续)

(续上页)

```
19         0xA6,  
20         0xA8,  
21         0x3F,  
22         0xA4,  
23         0xD3,  
24         0x00,  
25         0xD5,  
26         0xF0,  
27         0xD9,  
28         0x22,  
29         0xDA,  
30         0x12,  
31         0xDB,  
32         0x20,  
33         0x8D,  
34         0x14,  
35         0xAF,  
36     ]  
37 )  
38  
39 # Data sending and receiving buffer  
40 OutBuffer = bytearray(1)  
41 InBuffer = bytearray(1)  
42  
43 try:  
44     # Apply for i2c resources  
45     i2c = busio.I2C(board.I2C3_SCL, board.I2C3_SDA)  
46     # Scan I2C device address, test  
47     print("The address of the I2C device mounted on the I2C bus is")  
48     for i in i2c.scan():  
49         print("0x%02x " % i)  
50
```

(下页继续)

(续上页)

```
51     # IIC sends command test (writeto) to initialize OLED
52     # Initialize the OLED, the initial command is stored in the byte array
    ↪oledInitBuf
53     i2c.writeto(0x3C, OledInitBuf)
54
55     # IIC read command test (writeto_then_readfrom), read 0x10 register of
    ↪OLED
56     # After sending data, no stop signal will be generated after sending
    ↪data, and the start signal will be regenerated for reading
57
58     # The sent data content is stored in the byte array outBuffer, and the
    ↪content is OLED register address: 0x10
59     OutBuffer[0] = 0x10
60     # Send register address 0x10, and read the content under register
    ↪address 0x10 into inBuffer
61     i2c.writeto_then_readfrom(0x3C, OutBuffer, InBuffer)
62     # Print data information
63     print("(writeto_then_readfrom)The read content is:0x%02x" % InBuffer[0])
64
65     # IIC read test (readfrom_into), read from the internal address of the
    ↪device, you do not need to specify the address to read
66     i2c.readfrom_into(0x3C, InBuffer)
67     # Print data information
68     print("(readfrom_into) read the content is:0x%02x" % InBuffer[0])
69
70     # OLED clear screen
71     for i in range(8):
72         i2c.writeto(0x3C, bytearray([0x00, 0xB0 + i])) # page0-page1
73         i2c.writeto(0x3C, bytearray([0x00, 0x00])) # low column start
    ↪address
74         i2c.writeto(0x3C, bytearray([0x00, 0x10])) # high column start
    ↪address
```

(下页继续)

(续上页)

```
75     for j in range(128):
76         i2c.writeto(0x3C, bytearray([0x40, 0xFF]))
77
78 finally:
79     # Release i2c bus resources
80     i2c.deinit()
```

Code description:

- Line 45, apply for I2C resources and occupy the I2C1 host
- Line 53, send the data in the byte array OledInitBuf to the slave device mounted under the I2C1 host, the slave device address is 0x3C
- Lines 59~61, send the data in the byte array OutBuffer to the slave device mounted under the I2C1 host. And read a data from the device register address 0x10 to the byte array InBuffer, the slave device address is 0x3C
- Line 66, read from the current address inside the device, read a data to the byte array InBuffer
- Line 80, release I2C resource, release I2C1 master

The experimental operation is the same as the previous section, just run the program on the board.

The SCL and SDA in the `busio.I2C(SCL, SDA)` of the code snippet are LubanCat board resources defined in the Blinka library.

More detailed pin definitions can be directly viewde in [Source code for Adafruit Blinka](#) :

- [Pin definition of LubanCat series board in Adafruit Blinka](#)
- [Pin definition of LubanCat RK series chip in Adafruit Blinka](#)

8.4.3 Adafruit_CircuitPython_SSD1306

In the front, we used the Blinka library to realize the control of the 0.96-inch OLED display SSD1306 module through the I2C host. The Blinka library is a real-time component object reimplemented based on CircuitPython. In short, the Python program developed using the Blinka library can use the hardware resources on the board in real time. If you are familiar with CircuitPython, you will know that there are a lot of code demos based on CircuitPython library to control various electronic devices.

So in other words, is our Lubancat board adapted to the Blinka library, so that we can use the rich functions of CircuitPython. What about the control demo of various sensors? Take the module used by the author when doing the above experiments as an example, the demo of 0.96-inch OLED display SSD1306 module can be used normally.

Then the following author will take the SSD1306 module as an example to demonstrate how to run the program that CircuitPython provides support for the 0.96-inch OLED display SSD1306 module on our Lubancat board.

First of all, find the corresponding warehouse. Here we will demonstrate 0.96-inch OLED display SSD1306. Then you can search the keyword SSD1306 on [Adafruit Github](#) on Adafruit's Github, See [Adafruit_CircuitPython_SSD1306](#). This is the repository to be used. Of course, if you need, You can also search for other common modules such as dht11, mpu6050, etc., you need to explore by yourself.

In short, more Demo repositories follow [Adafruit Github](#)

Below the corresponding warehouse, we can see the method of adding the corresponding library:

```
# Use the following command to install the corresponding sensor library on
↳ the board, and choose one of the following statements:

# Enter under the root user:
pip3 install adafruit-circuitpython-ssd1306
# If you enter as a normal user:
sudo pip3 install adafruit-circuitpython-ssd1306
```

Or you can upload the downloaded warehouse to the board through other means, and enter the warehouse

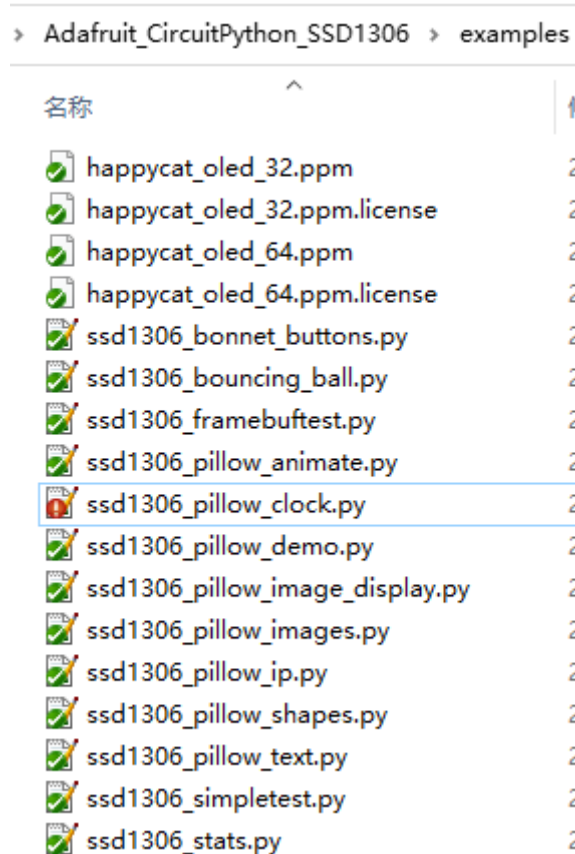
directory to execute the following statement installation:

```
sudo python3 setup.py install
```

Due to board performance and network issues, the installation process will be very long, please wait patiently. If an error is sent during installation, you can try another one of the above methods.

After the installation is complete, you can use the various sample demos provided in the warehouse! It should be noted that the address of the slave device in some demos is not completely suitable for the module in your hand. Therefore, when an error message appears, it may be necessary to modify the address of the slave device in the code Demo.

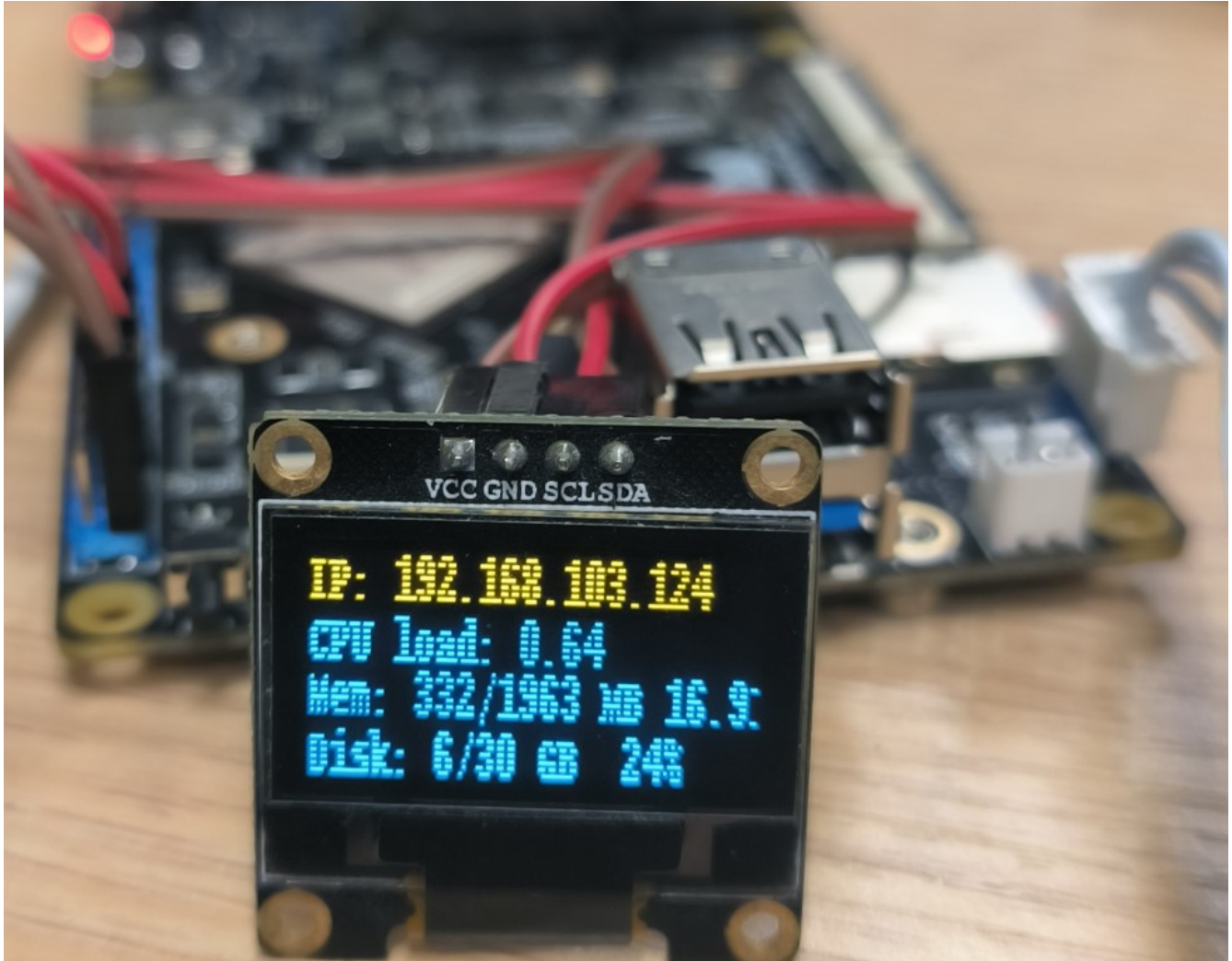
The code demo provided by CircuitPython for the 0.96-inch OLED display SSD1306 module is as follows:



Finally, run any Demo in the terminal to check the effect:

```
sudo python3 ssd1306_stats.py
```

It can display information such as the running status of the Lubancat board in real time:



Chapter9 SPI communication

In the previous chapters, we talked about the use of the I2C communication protocol. In this chapter, we then introduce the use of another communication protocol, the SPI communication protocol. The SPI communication protocol is also a common communication protocol in electronic device communication. However, the communication rate of the SPI communication protocol is higher, and it uses a full-duplex communication method. Therefore, we can often find the interface specified under the SPI communication protocol on electronic devices that require communication speed. For example, some small size LCD screen, AD conversion chip, nRF series radio frequency module, SPI-FLASH and so on.

Regarding the specific content of the SPI protocol, we will not explore too much here, and you can search for the relevant knowledge of the SPI communication protocol to learn.

In this chapter, our focus is to use the [python-periphery](#) library introduced earlier to use the SPI master on our Lubancat board.

9.1 SPI experiment

Since the SPI communication protocol supports full-duplex communication, we can conduct experimental verification through loopback testing. In the experiment, we just connect the MOSI and MISO interfaces of the SPI interface together.

9.2 Experiment preparation

9.2.1 Add SPI resource

The GPIO on the board may not be multiplexed as I2C, which can be loaded by setting the device tree plug-in. The pin reference of the LubanCat_RK series board is as follows: “[LubanCat-RK Series-40pin](#)”

Pin Mapping”

Let’ s take LubanCat 2 as an example to open the SPI device tree plug-in (the pins derived from LubanCat 2 can use spi3):

```
# The configuration files of different boards are different from i2c,
↳taking LubanCat 2 as an example, the configuration file is uEnvLubanCat 2.
↳txt
# Log in to the system terminal and open the /boot/uEnv/uEnvLubanCat 2.txt
↳file

sudo vim /boot/uEnv/uEnvLubanCat 2.txt
#Enter edit mode, cancel the comment in front of I2c3, save and exit the
↳file, restart the system
```

```
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo
#overlay_end
```

取消注释开启spi3

uEnvLubanCat2.txt

After restarting, check whether the SPI resource of the board is loaded:

```
# Enter the following command in the terminal to view the SPI resource:
ls -l /dev/spi*
```

The I2C resource example on the board is for reference only:

```
cat@lubancat:~$ ls /dev/spi*
/dev/spidev3.0
cat@lubancat:~$
```

For the SPI test on linux, you can refer to “[SPI Communication](#)”

9.2.2 Hardware connection

Note: In the hardware connection section, you need to make reference adjustments to the content of this chapter according to your own development environment and other conditions.

Perform a loopback test, the connection is as follows (LubanCat 2):

LuBanCat2系列引脚图					
GPIO	编号	功能	物理引脚		功能
		3.3V	1	2	5V
GPIO1_A0	32	I2C3_SDA	3	4	5V
GPIO1_A1	33	I2C3_SCL	5	6	GND
	8	GPIO0_B0	7	8	UART3_TX
		GND	9	10	UART3_RX
	101	GPIO3_A5	11	12	PWM8
	102	GPIO3_A6	13	14	GND
	103	GPIO3_A7	15	16	GPIO2_D7
MOSI和MISO直接连接		3.3V	17	18	GPIO3_A0
GPIO4_C3	147	MOSI	19	20	GND
GPIO4_C5	149	MISO	21	22	GPIO0_C2
GPIO4_C2	146	SCLK	23	24	CS0
		GND	25	26	CS1
GPIO3_B4	108	I2C5_SDA	27	28	I2C5_SCL
	154	GPIO4_D2	29	30	GND
	110	GPIO3_B6	31	32	PWM9
GPIO3_B5	109	PWM10	33	34	GND
GPIO3_C4	116	PWM14	35	36	GPIO1_B0
	117	GPIO3_C5	37	38	GPIO1_B1
		GND	39	40	GPIO1_B2

9.3 Use python-periphery

The SPI function supported by the [python-periphery](#) library is implemented based on the Linux SPI system, so if you want to use the library to use the SPI function, you need the board to provide support. Like the Lubancat board, you can perfectly use the SPI communication function of the [python-periphery](#) library. In this way, we don't need to use GPIO to simulate the SPI master function at the software level.

9.3.1 Install python-periphery

重要: Be sure to skip this if you installed the library in the previous section.

[python-periphery](#) is installed as follows:

```
# Use the following command to install on the board
sudo pip3 install python-periphery
```

9.3.2 The periphery uses the SPI function

The sample code for SPI function using [python-periphery](#) library is as follows:

列表 1: Supporting code 'io/spi/spi_test_periphery.py' file content

```
1 """ periphery spi test """
2 from periphery import SPI
3
4 # List of data to be sent
5 data_out = [0xAA, 0xBB, 0xCC, 0xDD]
6
```

(下页继续)

(续上页)

```
7 try:
8     # Apply for SPI resources, turn on the spidev3.0 controller, configure
    ↪ the SPI master to work in mode 0, and work at 1MHz
9     spi = SPI("/dev/spidev3.0", 0, 1000000)
10
11     # Send data and receive data to the data_in list at the same time
12     data_in = spi.transfer(data_out)
13
14     # Print the sent data content
15     print("Data sent: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]".
    ↪ format(*data_out))
16     print("Received data: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]".
    ↪ format(*data_in))
17 finally:
18     # Close the requested SPI resource
19     spi.close()
```

Code description:

- Line 5, apply for SPI resources, occupy the SPI3 host, and configure the corresponding working mode
- Line 9, constructing a list of messages to be sent
- Line 12, start SPI transmission, because SPI is a full-duplex communication mode, and it can also receive data while sending data
- Line 19, release SPI resource, release SPI3 master

9.3.2.1 Experimental operation

The sample code uses the LubanCat 2 board, and the operation is as follows:

```
# Execute the following command in the directory where the board spi_test.  
py is located  
sudo python3 spi_test.py  
  
# It can be seen that the received data printed by the terminal is  
consistent with the sent data
```

```
cat@lubancat:~$ sudo python3 spi_test01.py  
发送的数据: [0xaa, 0xbb, 0xcc, 0xdd]  
接收到的数据: [0xaa, 0xbb, 0xcc, 0xdd]  
cat@lubancat:~$
```

Since the MOSI and MISO interfaces of the SPI interface have been connected during the hardware connection. So when the SPI controller sends out data, it will also receive the data back, which is the loopback test.

9.4 Method 2: Using Adafruit Blinka

Adafruit Blinka is similar to [python-periphery](#), and the SPI communication is also implemented based on the SPI subsystem of Linux. It's just that there is a big difference in the usage of the code.

9.4.1 Installing Adafruit Blinka

重要: Be sure to skip this if you installed the library in the previous section.

Adafruit Blinka is installed as follows:

```
# Use the following command to install on the board  
sudo apt -y install python3-libgpiod python-periphery # 注: 如已安装, 忽略此步  
sudo pip3 install Adafruit-Blinka
```

9.4.2 Adafruit - Blinka using SPI master

The sample code for input and output using this [Adafruit Blinka](#) is as follows:

列表 2: Supporting code 'io/spi/spi_test_adafruit.py' file content

```
1  """ blinka spi test """
2  import busio
3  from board import *
4
5  # Data sending and receiving buffer
6  OutBuffer = [0xAA, 0xBB, 0xCC, 0xDD]
7  InBuffer = bytearray(4)
8
9  try:
10     # Apply for spi resources
11     spi = busio.SPI(SCLK, MOSI, MISO)
12
13     # Lock the SPI first when configuring
14     spi.try_lock()
15     # Configure the SPI host operating rate as 1MHz, clock polarity as 0,
16     ↪ clock phase as 0, and a single data bit as 8 bits
17     spi.configure(1000000, 0, 0, 8)
18     # The configuration operation is completed and unlocked
19     spi.unlock()
20
21     # SPI communication, while sending and reading, the sent data is stored
22     ↪ in OutBuffer, and the read data is stored in InBuffer
23     spi.write_readinto(OutBuffer, InBuffer)
24
25     print(
26         "(write_readinto)Received data: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x
27         ↪{:02x}]" .format (
```

(下页继续)

(续上页)

```
25         *InBuffer
26     )
27 )
28
29     # SPI communication, write-only usage demonstration
30     spi.write(OutBuffer)
31     print(
32         "(OutBuffer)Sent data: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]".
↪format(*OutBuffer)
33     )
34
35     InBuffer = [0, 0, 0, 0]
36     # SPI communication, read-only usage demonstration
37     spi.readinto(InBuffer)
38     print(
39         "(readinto)Received data: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]".
↪format(*InBuffer)
40     )
41 finally:
42     # Release spi resources
43     spi.deinit()
```

Code description:

- Line 11: Apply for SPI resources, occupying SPI3 host
- Lines 14~18: Configure the working mode of the SPI3 host, and the resources need to be locked before configuration
- Line 21: Use the SPI3 host to send the data in the byte array OutBuffer, and read the data to the InBuffer at the same time
- Line 30: Use SPI3 to send data only, not to read
- Line 37: Use SPI3 to read data only, not send

After running it shows:

```
cat@lubancat:~$ sudo python spi_test_adafruit.py
(write_readinto)接收到的数据: [0xaa, 0xbb, 0xcc, 0xdd]
(OutBuffer)发送的数据: [0xaa, 0xbb, 0xcc, 0xdd]
(readinto)接收到的数据: [0x00, 0x00, 0x00, 0x00]
cat@lubancat:~$
```

In addition, there is a supporting routine `io/spi/ssd1306_spi_status.py`, which will use `spi` to control the screen display. For details, refer to the previous subsection of `i2c` using the `Adafruit_CircuitPython_SSD1306` library to control the screen display.

[Adafruit Blinka](#) The library does not provide examples of using SPI objects. You can refer to related API functions: [CircuitPython SPI](#).

Chapter10 Screen Display - Pygame

Our Lubancat boards support a variety of display devices, and the boards generally have HDMI interfaces and MIPI DSI interfaces.

10.1 Introduction to the Pygame library

Pygame is a tool for developing games in the Python library. Perhaps it is a bit restrictive to use Pygame. Pygame can be used not only for game development, but also for display processing, multimedia device processing, etc. The library provides many multimedia and display device operation functions. The display module provides various functions to control the Pygame display interface (display). This chapter will use this module for screen display.

10.2 Experiment preparation

10.2.1 Add screen resources

```
# Enter the following command in the terminal to view the display device.
↪resources:
cat@lubancat:~$ ls /dev/dri/card*
/dev/dri/card0  /dev/dri/card1

# After connecting the screen, there will be:
cat@lubancat:~/lcd$ ls /dev/fb*
/dev/fb0
```

The example of display device resources on the board is for reference only (take LubanCat 2, ubuntu image as an example):

```
cat@lubancat:/boot/dtb$ ls -al
total 1829
drwxrwxr-x 3 root root 3072 Feb 14 2019 .
drwxr-xr-x 6 root root 1024 Nov 26 14:10 ..
drwxrwxr-x 2 root root 5120 Feb 14 2019 overlay
-rw-r--r-- 1 root root 152325 Nov 21 14:45 rk3566-lubancat-zero-mipi.dtb
-rw-r--r-- 1 root root 149830 Nov 21 14:45 rk3566-lubancat-zero.dtb
-rw-r--r-- 1 root root 153471 Nov 21 14:45 rk3566-lubancat1-mipi.dtb
-rw-r--r-- 1 root root 153087 Nov 21 14:45 rk3566-lubancat1-n-mipi.dtb
-rw-r--r-- 1 root root 150600 Nov 21 14:45 rk3566-lubancat1-n.dtb
-rw-r--r-- 1 root root 150984 Nov 21 14:45 rk3566-lubancat1.dtb
-rw-r--r-- 1 root root 160739 Nov 21 14:45 rk3568-lubancat2-io.dtb
-rw-r--r-- 1 root root 161399 Nov 21 14:45 rk3568-lubancat2-mipi+hdmi.dtb
-rw-r--r-- 1 root root 161407 Nov 21 14:45 rk3568-lubancat2-mipi.dtb
-rw-r--r-- 1 root root 159721 Nov 21 14:45 rk3568-lubancat2-n.dtb
-rw-r--r-- 1 root root 158916 Nov 21 14:45 rk3568-lubancat2.dtb
-rw-rw-r-- 1 root root 145719 Nov 21 15:01 rk356x-lubancat-rk_series.dtb
cat@lubancat:/boot/dtb$
```



The default is to use the rk3568-LubanCat 2.dtb device tree, and select the corresponding device tree by creating a soft connection:

```
#You must first switch to the boot directory
cd /boot
#Switch device tree for mipi screen
ln -sf dtb/rk3568-LubanCat 2-mipi.dtb rk-kernel.dtb
#Reboot to use mipi screen
sudo reboot
```

For more information on screen display, please refer to “Screen” .

10.2.2 Hardware connection

For the hardware connection section, you need to make reference adjustments to the content of this chapter according to your own development environment and other conditions. The experiment here is to connect the mipi screen (5.5 inches) to the LubanCat 2 board, and the board system has a desktop.

10.2.3 Pygame library installation

We use the apt tool to install.

```
# Enter the following command in the terminal to install the Pygame library:  
sudo apt -y install python3-pygame  
# If it was installed before, there is no need to reinstall
```

10.2.4 The Pygame library uses

After installing the corresponding library, we can use the installed Pygame library to write some test code.

Although the display driver is DRM, the driver also implements the simulated FB device. Before testing, we can simply use Linux Framebuffer. The Pygame library uses the Linux Framebuffer mechanism to realize the function of adjusting the display device. The Linux Framebuffer maps each point on the screen into a linear memory space. The program can simply change the value of this memory to change the color of a certain point on the screen. You can use the following commands to continuously write random data to /dev/fb0, making the screen blurry.

```
# Enter the following command in the terminal to test:  
cat /dev/urandom > /dev/fb0
```

For more usage, please refer to “[Screen Display \(framebuffer\)](#)”。

10.3 Test code one

The test code is as follows:

列表 1: Supporting code ‘io/lcd/lcd_test1.py’ file content

```
1  """ Screen display test using pygame """
2  import os
3  import sys
4  import time
5  import pygame
6
7  # Setting up the system environment, no mouse.
8  os.environ["SDL_NOMOUSE"] = "1"
9
10 # Initialize display device
11 pygame.display.init()
12
13 # Set the range of the display window to the screen size and return a
    ↪screen object
14 screen = pygame.display.set_mode((448, 418))
15
16 # Set window name
17 pygame.display.set_caption("test")
18
19 # Load images using the pygame image module
20 ball = pygame.image.load("test.png")
21
22 # Bitwise copy image to window
23 screen.blit(ball, (0, 0))
24
25 # Refresh window
26 pygame.display.flip()
27
28 time.sleep(5)
29 sys.exit()
```

The above code will display the image below on a window on the mipi display:



10.3.1 Experimental procedure

Upload the test code and test pictures in the supporting code Lcd directory to the same directory of the development board:

```
# Enter the following command in the terminal, because access to hardware_
↳ pays attention to sudo permission:
sudo python3 lcd_test1.py
```

10.4 Test code two

列表 2: Supporting code ‘io/lcd/lcd_test2.py’ file content

```
1  """ Screen testing with pygame """
2  import os
3  import sys
4  import time
5  import pygame
6
7  class PyScope:
8      """ Define a PyScope class for screen testing """
9
10     screen = None
11
12     def __init__(self):
13         """The initialization method of the PyScope class, using framebuffer_
14         ↪to construct the image buffer that pygame will use"""
15
16         # Set the system environment to no mouse mode
17         #os.environ["SDL_NOMOUSE"] = "1"
18         pygame.display.init()
19
20         # Create, set the window size used by pygame
21         self.screen = pygame.display.set_mode((600, 600))
22
23         #Set window title
24         pygame.display.set_caption('lcd_test2.py')
25
26         # Fill background color to gray
27         self.screen.fill((156, 156, 156))
```

(下页继续)

(续上页)

```
28     # Initialize the font library
29     pygame.font.init()
30
31     # Use the default font to display English
32     font=pygame.font.Font (None, 32)
33     text=font.render("Hello!", True, (255, 255, 255))
34     self.screen.blit(text, (100, 100))
35
36     # Display text using default font
37     text=font.render("Display Test!", True, (255, 0, 0))
38     self.screen.blit(text, (100, 150))
39
40     # Update screen content
41     pygame.display.flip()
42
43     def __del__(self):
44         "This method will be called when exiting the pygame library, you
45         ↪can add resource release operations here"
46         pygame.display.quit()
47
48     def test(self):
49         while True:
50             # Loop to get events, listen to events
51             for event in pygame.event.get():
52                 # Determine whether the user mouse clicked the close button
53                 if event.type == pygame.QUIT:
54                     #Uninstall all modules
55                     pygame.quit()
56                     #Terminate program
57                     sys.exit()
58                 #Update screen content
59                 #pygame.display.flip()
```

(下页继续)

(续上页)

```
59
60 # Create a test instance and start testing
61 scope = PyScope()
62 # Call the test method of the scope class
63 scope.test()
```

The code function simply looks at the comments above. In the `__init__` method, the window display will be initialized, and finally the code will display two lines of English in the pop-up window.

10.4.1 Experimental procedure

Upload the test code in the supporting code Lcd directory to any directory of the development board:

```
# Enter the following command in the terminal:
sudo python3 lcd_test2.py
```

After the command is executed, a pop-up window will appear on the system desktop, and the window can be closed by clicking the mouse or closing the program.

10.5 Reference

This chapter does not introduce the use of the basic functions of the display module in detail. For more functions of the Pygame display module, please refer to [Pygame display](#).

For the use of Python Pygame library, please refer to [Pygame tutorial](#).

Chapter11 Audio playback - Pygame

Among LubanCat-RK series boards, LubanCat 2 and LubanCat 1 have audio rk809-codec. Through the audio codec chip. In this chapter, we will use the music module of the Python-Pygame library to write test codes for audio playback experiments on the lubancat board.

11.1 Introduction to the Pygame library

Pygame is a tool for developing games in the Python library. Perhaps it is a bit restrictive to use Pygame. Pygame can be used not only for game development, but also for display processing, multimedia device processing, etc. The library provides many multimedia and display device operation functions.

11.2 Experiment preparation

11.2.1 Sound resources on the board

Take LubanCat 2 as an example:

```
# Enter the following command in the terminal to view the display device.
↪resources:
ls /dev/snd/
```

Check the audio device resource example on the board, just for reference:

```
cat@lubancat:~/sound$ ls /dev/snd/
by-path  controlC0  pcmC0D0c  pcmC0D0p  seq  timer
```

Among them, controlC0 is used for sound card control, such as channel selection, microphone control, etc. pcmC0D0c is a pcm device for recording, pcmC0D0p is a pcm device for playback, seq is a sequencer, and timer is a timer.

For more information about the Lubancat audio system in linux, please refer to [“Audio”](#) .

11.2.2 Hardware connection

LubanCat 1 and LubanCat 2 boards have earphone + microphone interface, just connect the earphone or other playback devices to the Lubancat interface.

11.2.3 Pygame library installation

We use the apt tool to install.

```
# Enter the following command in the terminal to install the Pygame library:  
sudo apt -y install python3-pygame
```

11.2.4 The Pygame library uses

After installing the corresponding library, we can use the installed Pygame library to write some test code.

11.3 Test code

测试代码如下：

列表 1: Supporting code ‘../io/sound/sound_test.py’ file content

```
1  """ pygame audio playback test """
2  import sys
3  import time
4  import threading
5
6  if len(sys.argv) == 2:
7      m_filename = sys.argv[1]
8  else:
9      print(
10         """Instructions:
11         python3 sound_test.py <music_name.mp3>
12         example: python3 sound_test.py test.mp3"""
13     )
14     sys.exit()
15
16 import pygame
17
18 exitright = 'e'
19
20 def get_quitc():
21     'Get exit character'
22     # Global variable
23     global exitright
24     # Get user input
25     exitright = input("After entering the letter q, press enter to exit_
↵ playback\n")
26
27 try:
28     # Initialize the audio device
29     pygame.mixer.init()
```

(下页继续)

(续上页)

```
30     # Check audio device initialization
31     if pygame.mixer.get_init() is None:
32         raise RuntimeError('The audio resource was not properly initialized,
↪ please check if the audio device is available',)
33     # Load audio files, mp3 files are not played here, because pygam's_
↪ playback support for MP3 format is limited
34     pygame.mixer.music.load(m_filename)
35     # Set the volume to 50%
36     pygame.mixer.music.set_volume(0.2)
37     # Play audio file 1 time
38     pygame.mixer.music.play(0)
39
40     # Start the child thread, waiting for the user to enter the exit_
↪ character
41     quitthread = threading.Thread(target=get_quitc, daemon=True)
42     quitthread.start()
43     # Get the audio playback status, if it is playing, wait for the_
↪ playback to finish
44     while pygame.mixer.music.get_busy():
45         # Idle delay, release cpu
46         if exitright in ('q', 'Q'):
47             raise
48         time.sleep(1)
49 except pygame.error as e:
50     print("An exception occurred while executing the program: ", e, "\
↪ nWaiting for the audio resource to be released")
51     # Try to stop playback
52     pygame.mixer.music.stop()
53     # Wait for an audio resource to be free
54     while pygame.mixer.music.get_busy():
55         time.sleep(1)
56 finally:
```

(下页继续)

(续上页)

```
57     # Uninstall the initialized audio device
58     print("Quit playing")
59     pygame.mixer.quit()
60     sys.exit()
```

The code function has been given in detail in the comments.

11.3.1 Experimental procedure

Before testing the code, please place the test audio files of various formats in the same directory as the code in the io/sound directory of the supporting code folder, and then run the program.

The test code plays a user-specified audio file.

```
# Enter the following command in the terminal:
sudo python3 sound_test.py test.mp3
# or
sudo python3 sound_test.py test.wav
# or
sudo python3 sound_test.py test.ogg
```

There are a lot of resources loaded when the pygame library is imported, so it will take a while for the experimental phenomenon to appear. After the program runs normally, you can hear the audio played on the playback device~

During the music playing process, if you want to exit, please operate according to the prompt information printed by the terminal, enter q in the terminal and press Enter, and the program will end. If the program terminates abnormally, the audio device resources will be occupied, and the development board needs to be restarted before it can be used again.

Running shows:

```
cat@lubancat:~/test/sound$ python3 sound_test.py test.wav
pygame 1.9.6
Hello from the pygame community. https://www.pygame.org/contribute.html
输入字母q后，按下回车以退出播放
q
退出播放
cat@lubancat:~/test/sound$ ls
sound_test.py  test.mp3  test.ogg  test.wav
cat@lubancat:~/test/sound$
```

11.3.2 Reference

For more information about the use of the Python Pygame library, please refer to [Pygame Tutorial](#) .

About Pygame music, please refer to [Pygame music](#).

Chapter12 Camera use-Opencv

The Lubancat RK series boards support the use of common sensors such as cameras. The boards all have 24pin mipi csi interfaces. The camera used in the test is ov5648. This chapter will use the opencv-python library to use the camera and perform simple processing.

提示: If there is no special mention, the tutorials in this book are based on Python 3.8.10 version (the mirror system is Ubuntu20.04) for experiments and explanations. To learn the content of this chapter, you need a camera hardware device, please prepare by yourself.

12.1 Introduction to opencv-python library

OpenCV is a classic dedicated library in computer vision. It supports multi-language, cross-platform, and powerful. It was founded by Gary Bradsky in Intel in 1999 and released its first version in 2000. OpenCV now supports numerous algorithms related to computer vision and machine learning and is expanding day by day. OpenCV supports various programming languages, such as C++, Python, Java, etc., and is available on different platforms, including Windows, Linux, OS X (the system has been renamed macOS), Android, and iOS. High-speed GPU operation interfaces based on CUDA and OpenCL are also under active development.

OpenCV-Python is a Python-based library, or OpenCV's Python application interface, designed to solve computer vision problems, combining the best features of the OpenCV C++ API and the Python language.

12.2 Add camera resource to Lubancat board

In this section of the experiment, the LubanCat 2 board is used as an example for demonstration. Before powering on, connect the camera to the mipi csi interface. After the system starts:

```
# Enter the following command in the terminal to view the camera device_
↪resources:
ls /dev/video*
```

Check the camera device resource example on the board, for reference only:

```
cat@lubancat:~$ ls /dev/video*
/dev/video-dec0  /dev/video1  /dev/video4  /dev/video7
/dev/video-enc0  /dev/video2  /dev/video5  /dev/video8
/dev/video0      /dev/video3  /dev/video6
```

More camera usage references under linux system “Camera” . This chapter will introduce tools such as v4l-utils, and you can view the relevant information of the camera device.

12.3 Related tools and library installation

We will use some tools in the experiment, enter the following command to install (if installed, it is not necessary):

1、Install some dependent libraries or tools:

```
# Enter the following command in the terminal:
sudo apt update
sudo apt -y install python3-pil python3-numpy python3-pip git wget

# Libraries that may be required, some may be installed by default
sudo apt install libavcodec-dev libavformat-dev libswscale-dev
sudo apt install libgstreamer-plugins-base1.0-dev libgstreamer1.0-dev
```

Just wait for the installation to complete.

2、Install the opencv-python library:

Install directly using the command (recommended):

```
# The python3-opencv package can be installed using the apt command:
sudo apt-get install python3-opencv

# Or use pip to specify the opencv version to install:
pip3 install opencv-python==4.7.0.68
```

To get the whl file installation, you can go here to [download](#) the whl file corresponding to the python version and architecture

```
# For example, the following whl file can be installed using the command:
pip3 install opencv_python-4.5.4.60-cp37-cp37m-manylinux_2_17_aarch64.
manylinux2014_aarch64.whl
```

Or download the source code to compile and install, you can refer to this [link](#) .

提示: If you use the Debian10 system, you need to use opencv_python to control the camera, and you need to install a higher version of opencv_python and pip3. Use the command (update pip3 and install whl with root authority): `sudo pip3 install --upgrade pip`, and then choose the above method to install opencv_python.

3、Verify that the installation was successful:

Open an interactive Python terminal (or IPython):

```
cat@lubancat:~$ python3
Python 3.8.10 (default, Nov 14 2022, 12:59:47)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cv2 as cv
```

(下页继续)

(续上页)

```
>>> print(cv.__version__)  
4.2.0  
>>>
```

12.4 Camera to take pictures

列表 1: Companion code ‘io/camera/camera_photo.py’ file
content

```
1 import os  
2 import cv2  
3  
4 # frame width and height  
5 width = 640  
6 height = 480  
7  
8 num = 1  
9  
10 # Create a VideoCapture object and open the system's default camera (you  
11    ↳ can also open the video or the specified device)  
12 cap = cv2.VideoCapture(0)  
13  
14 # Can't open camera  
15 if not cap.isOpened():  
16     raise RuntimeError('Could not open camera.')  
17  
18 # Set frame width and height  
19 cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)  
20 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
```

(下页继续)

(续上页)

```
21 while(cap.isOpened()):
22
23     # Returns two parameters, ret indicates whether it is opened normally,
    ↪ frame is an image array
24     ret, frame = cap.read()
25
26     # The window displays, named Lubancat_Camera_test
27     cv2.imshow("Lubancat_Camera_test", frame)
28
29     # Delay 1ms, and return the value val according to the keyboard input,
    ↪ which is the keyboard connected to the board
30     val = cv2.waitKey(1) & 0xFF
31
32     # The keyboard input 's' is captured, and the picture is saved to the
    ↪ current directory
33     if val == ord('s'):
34         print("=====")
35         # The first parameter is the name of the picture to be saved as,
    ↪ and the second parameter is the image to be saved, in jpeg format
36         cv2.imwrite("photo" + str(num) + ".jpg", frame)
37         print("width = ", width)
38         print("height = ", height)
39         print("success to save photo: "'photo' + str(num) + ".jpg")
40         print("=====")
41         num += 1
42
43     # If the key 'q' is detected, exit, it is the keyboard connected to the
    ↪ board
44     if val == ord('q'):
45         break
46
47 # Release camera
```

(下页继续)

(续上页)

```
48 cap.release()
49 # Close the window
50 cv2.destroyAllWindows()
```

Use the mirroring system with desktop, connect the keyboard and screen, run the program on the board system, and use the camera to read the above notes in detail. Run the program with the command:

```
sudo python3 camera_photo.py
```

At this time, a pop-up window will appear on the screen. When the keyboard presses “s”, the picture will be saved to the current directory, and the installation of “q” will exit the program. Run example:

```
cat@lubancat:~/test/camera/test$ sudo python3 camera_photo.py
[ WARN:0] global ../modules/videoio/src/cap_gstreamer.cpp (935) open OpenCV | GStreamer warning: Cannot query video
(python3:43255): Gtk-WARNING **: 10:33:20.314: Locale not supported by C library.
    Using the fallback 'C' locale.
=====
frame width = 640
frame height = 480
success to save photo: photo1.jpg
=====
frame width = 640
frame height = 480
success to save photo: photo2.jpg
=====
cat@lubancat:~/test/camera/test$ ls
camera_photo.py camera_video.py out photo1.jpg photo2.jpg
```

12.5 Camera capture, record video

Each frame in the video represents an image, and we record the video by capturing a frame of image and compressing and saving it.

列表 2: Companion code 'io/camera/camera_video01.py' file
content

```
1 import os
2 import cv2
3
4 # frame width, height, frame rate
5 width = 640
6 height = 480
7 fps = 25.0
8
9 # Create a VideoCapture object and open the system default camera
10 video = cv2.VideoCapture(0)
11
12 # Set frame width and height
13 video.set(cv2.CAP_PROP_FRAME_WIDTH, width)
14 video.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
15
16 # Define the writing format of the video file, the uncompressed YUV color_
  ↳coding type (the video file saved in this format is very large, you need_
  ↳to pay attention), some formats may not be supported, the encoder fails,_
  ↳and related libraries need to be installed
17 # fourcc = cv2.VideoWriter_fourcc(*'MJPG')
18 fourcc = cv2.VideoWriter_fourcc('I','4','2','0')
19
20 # It is used to save multiple images into video files. The first parameter_
  ↳is the name of the video file to be saved, and the second function is the_
  ↳codec code
21 # The third parameter is the frame rate of the saved video, and the fourth_
  ↳parameter is the size of the saved video file, which must be the same_
  ↳size as the image
22 # out = cv2.VideoWriter('output.mp4',fourcc, fps, (width,height))
```

(下页继续)

(续上页)

```
23 out = cv2.VideoWriter('output.avi',fourcc, fps, (width,height))
24
25 while(video.isOpened()):
26     ret, frame = video.read()
27     if ret is True:
28         # window displays, named Lubancat_Camera_video
29         cv2.imshow('Lubancat_Camera_video',frame)
30
31         # Store the captured image, the saved video has no sound
32         out.write(frame)
33
34         # Delay 1ms, if the keyboard presses "q", exit, it is the keyboard_
↪connected to the board
35         if cv2.waitKey(1) & 0xFF == ord('q'):
36             break
37     else:
38         print("Unable to read camera!")
39         break
40
41 # Release resources
42 video.release()
43 out.release()
44 cv2.destroyAllWindows()
```

Run the program, it will capture and record, and the video will be saved in the current directory. See the note above for the detailed process. Exit when the keyboard presses “q” , or directly interrupt the running of the program.

To play video, you can also use cv2.VideoCapture, refer to the following:

列表 3: Companion code ‘io/camera/camera_video02.py’

```
1 import cv2
2
3 # Create a VideoCapture object and select the video file to play
4 cap = cv2.VideoCapture('output.avi')
5
6 while(cap.isOpened()):
7     ret, frame = cap.read()
8
9     # Window displays, namedLubancat_Camera_video
10    cv2.imshow('Lubancat_Camera_video', frame)
11
12    # Delay 1ms, if the keyboard presses "q", exit, it is the keyboard_
    ↪connected to the board
13    if cv2.waitKey(1) & 0xFF == ord('q'):
14        break
15
16 # Release resources
17 cap.release()
18 cv2.destroyAllWindows()
```

12.6 Reference

https://docs.opencv.org/4.6.0/dc/d4d/tutorial_py_table_of_contents_gui.html

Chapter13 Jupyter Notebook

13.1 Introduction to Jupyter Notebook

Jupyter Notebook is a web-based application for interactive computing. Due to the interactive nature of this tool, if we use this tool for software development, we can view the data changes after the code is executed in real time after each command is written. So in Python software development, Jupyter Notebook can be said to be a sharp tool.



The Jupyter Notebook application can run in a Linux environment, and it is also deployed on a server. Then the function of our Lubancat board is similar to that of a traditional PC, and it also works under the Linux environment. Naturally, we can also install Jupyter Notebook software on our Lubancat board and use it to develop our own Python programs.

Next, let's install the Jupyter Notebook software.

13.2 Jupyter Notebook installation

13.2.1 Software Installation

Using the Lubancat board, you can easily install Jupyter Notebook with only two instructions.

重要: Subsequent operations are performed under the user user command.

```
# Enter the following command in the terminal:  
sudo apt -y install ipython3  
sudo apt -y install jupyter
```

ipython3 is a python interactive shell that we will use when using Jupyter Notebook to develop python applications.

Jupyter is the Jupyter Notebook application we are going to install.

Execute the above two commands in sequence and wait for the installation to complete.

13.2.2 Software configuration

Next we need to generate the configuration file for Jupyter Notebook. When we start Jupyter Notebook, the application starts the program according to the specific configuration.

Enter the following command in the terminal:

```
# Generate configuration files for Jupyter Notebook  
jupyter notebook --generate-config  
  
# After input, there will be the following prompt, and the configuration_  
↪file is different according to the logged-in user:  
Writing default config to: /home/cat/.jupyter/jupyter_notebook_config.py
```

In the prompt, the default configuration file has been generated in the currently logged in user directory --cat, then we open the corresponding configuration file to modify it according to this directory.

```
# Before modifying the configuration file, create the working directory of
↪Jupyter Notebook
mkdir /home/cat/jupyter

# Open the configuration file with vim editor
vim /home/cat/.jupyter/jupyter_notebook_config.py
```

After opening the configuration file (press a to enter edit mode):

```
c.NotebookApp.allow_remote_access = True # Allow remote connections
c.NotebookApp.ip = '*' # Listen to all ip access
c.NotebookApp.notebook_dir = '/home/debian/jupyter' # The working directory
↪of jupyter. If the directory does not exist, it must be created
c.NotebookApp.token = '' # Access without token key is not safe but
↪convenient
```

In the default configuration, the options are commented by # by default, we only need to cancel the comments corresponding to the options that need to be modified. And modify the configuration according to the content of the above configuration, or directly add the above four lines and save it.

As an extra note, the directory specified in the `c.NotebookApp.notebook_dir` option is the working directory when Jupyter Notebook starts. This directory must exist, if it does not exist, please create it first!

13.2.3 Security Login Configuration

In the example of modifying the configuration file we provided for you, we have disabled the token login option by default, just for your convenience. This is an unsafe approach, because any user can log in to use the Jupyter Notebook service, and file security cannot be guaranteed.

If you need a safe production environment, you can follow the steps below, if not, skip this section:

```
# Enter the following command in the terminal:
ipython3

# In the started python editor type:
from notebook.auth import passwd

passwd()
# Enter password: appears, enter any password, and copy and save the_
→ generated key.
Out[2]: 'sha1:274e5a04060e:e65098703b7e03836b01c261b6ca6d959049a3e9'
# ctrl+z exit ipython3
```

```
# Open the configuration file with vim editor
vim /home/cat/.jupyter/jupyter_notebook_config.py

# Then comment the token configuration option
# c.NotebookApp.token = ''

# Add password to log in, fill in the password you just generated
c.NotebookApp.password =
→ 'sha1:274e5a04060e:e65098703b7e03836b01c261b6ca6d959049a3e9'
```

Since the software has been installed, we can start Jupyter Notebook and use it.

13.3 Precautions before using Jupyter Notebook

重要: When using Jupyter Notebook to operate peripherals, be sure to refer to this section.

When we use Python to operate some peripheral hardware, root user privileges are required. Similarly, if you use Jupyter Notebook to operate some peripheral hardware, you also need root privileges. The solution

then is to log in as root and start Jupyter Notebook as root.

It should be noted that the environment configuration will be checked when Jupyter Notebook starts, and different users have their own configuration files!

Therefore, before logging in to Jupyter Notebook as the root user, you need to go through the previous configuration process again as the root user.

That is: before starting Jupyter Notebook as the root user, you need to log in to the root user first, And use the `jupyter notebook --generate-config` command as the root user to generate a configuration file and modify it to configure the Jupyter Notebook environment.

13.4 Jupyter Notebook uses

13.4.1 start software

Enter the following command to start the software.

```
# Enter the following command in the terminal to start Jupyter Notebook:
# Allow root users to use, do not open the browser when starting the_
↪software
jupyter notebook --no-browser --allow-root
```

After the software is started, the following prompt will be printed:

```
cat@lubancat:~$ jupyter notebook --no-browser --allow-root
[W 14:00:12.943 NotebookApp] WARNING: The notebook server is listening on all IP addresses and not using encryption. This is not recommended
[I 14:00:12.978 NotebookApp] Serving notebooks from local directory: /home/cat/jupyter
[I 14:00:12.979 NotebookApp] The Jupyter Notebook is running at:
[I 14:00:12.979 NotebookApp] http://lubancat:8888/
[I 14:00:12.980 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
```

According to the prompt, we enter the corresponding URL in the browser to use the function of Jupyter Notebook.

Take the LubanCat 2 board as an example: there are two network ports on the board, just plug in the network cable, and the computer and the board are in the same network segment:

```
cat@lubancat:~$ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 2a:fa:d5:eb:2c:d3 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 38

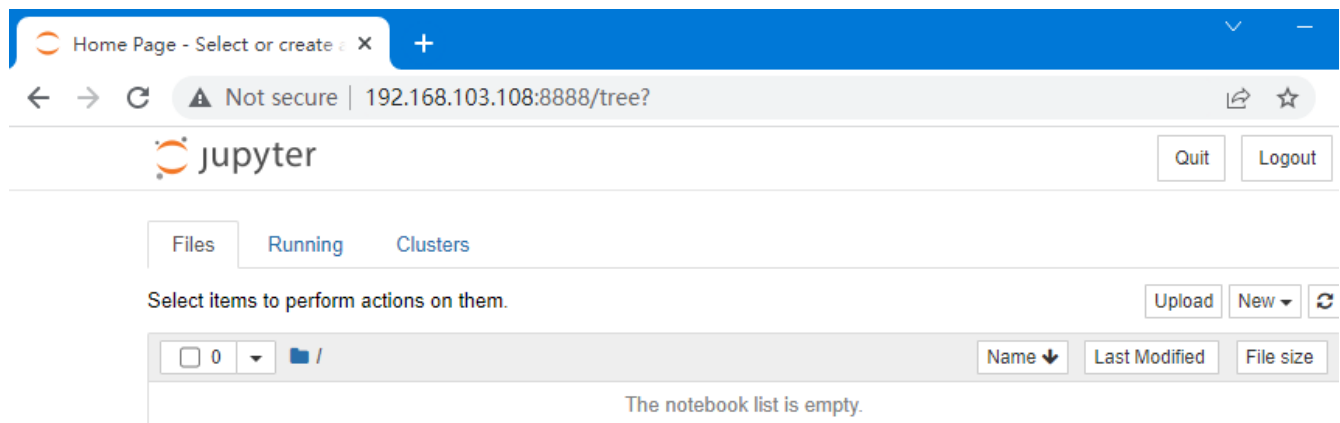
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.103.108 netmask 255.255.255.0 broadcast 192.168.103.255
    inet6 fe80::11b4:3fba:fb2:bbbc prefixlen 64 scopeid 0x20<link>
    ether 2e:fa:d5:eb:2c:d3 txqueuelen 1000 (Ethernet)
    RX packets 786922 bytes 158359403 (158.3 MB)
    RX errors 0 dropped 20401 overruns 0 frame 0
    TX packets 9301 bytes 4816175 (4.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 40

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2529 bytes 1546842 (1.5 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2529 bytes 1546842 (1.5 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Enter in the PC browser (the IP address depends on the actual board):

<http://192.168.103.108:8888/> can visit Jupyter Notebook

Enter the created password, and then go to the login startup interface as shown in the figure:

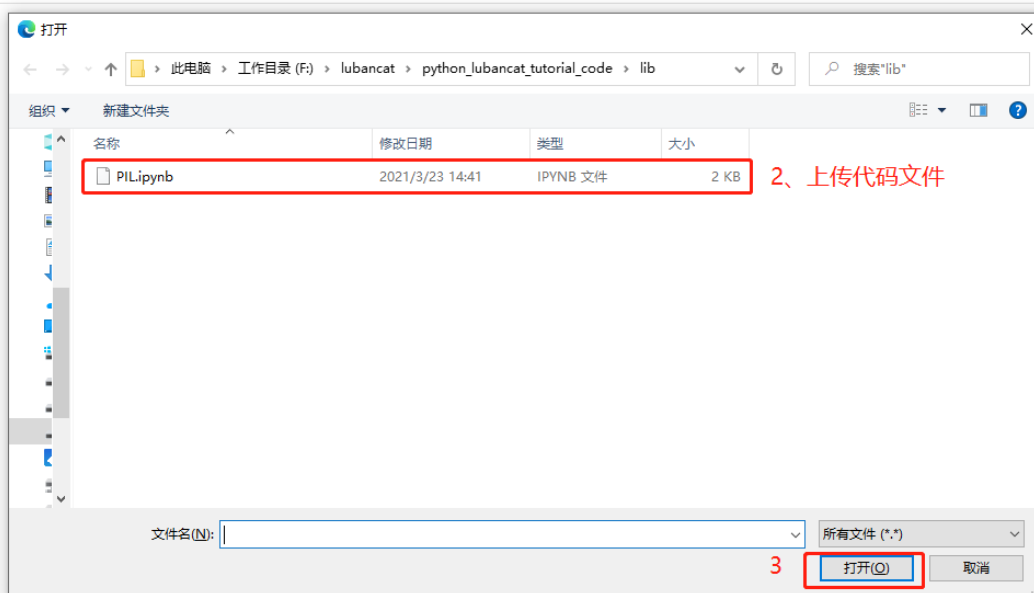


After starting, we can try to use Jupyter Notebook to write simple code for testing:



13.4.2 Upload code

Sometimes we can also use code written by others. We only need to download the corresponding file and upload it to jupyter to use it.



上传完成后点击文件名可打开

点击上传

Chapter14 PIL library

14.1 Introduction to the PIL library

PIL (Python Image Library) is a powerful image processing library for python. Using this library, we can do a lot of processing work on images. We can install the Python-PIL library on the Lubancat board and write some test codes through jupyter to use the library.

14.2 PIL library installation

We use the apt tool to install.

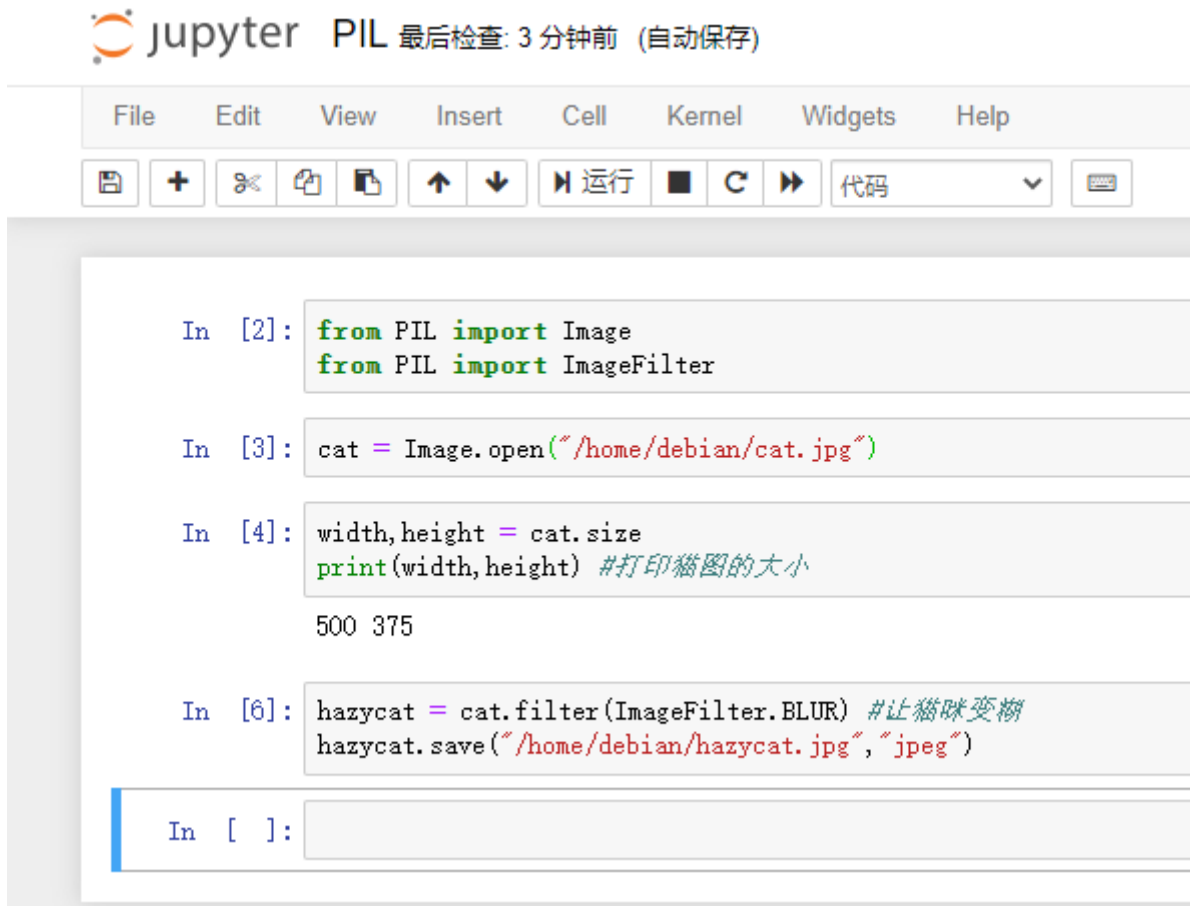
```
# Enter the following command in the terminal to install the PIL library:  
sudo apt -y install python3-pil
```

Just wait for the installation to complete.

14.3 PIL library usage

After installing the corresponding library, we can use the jupyter installed earlier to write the test code.

The test code is as follows:



Jupyter PIL 最后检查: 3 分钟前 (自动保存)

File Edit View Insert Cell Kernel Widgets Help

运行 代码

```
In [2]: from PIL import Image
        from PIL import ImageFilter

In [3]: cat = Image.open("/home/debian/cat.jpg")

In [4]: width,height = cat.size
        print(width,height) #打印猫图的大小
        500 375

In [6]: hazycat = cat.filter(ImageFilter.BLUR) #让猫咪变糊
        hazycat.save("/home/debian/hazycat.jpg","jpeg")

In [ ]:
```

The effect is as follows:

Original image:



After code execution:



You can upload the supporting code directory “libdemo\PIL\PIL.ipynb” file to jupyter for use.

More about the usage of the Python image processing library, you need to explore by yourself. It should be noted that after the Python3 version, the image processing library is pillow, which is a compatible version based on PIL.

Chapter15 Numpy library

15.1 Introduction to the Numpy library

NumPy (Numerical Python) is a powerful numerical computing extension processing library for Python that supports a large number of dimensional arrays and matrix operations. In addition, a large number of mathematical function libraries are provided for array operations. We can see Numpy in many fields of Python applications, such as scientific computing, artificial intelligence, image processing and so on. We can install the Python-NumPy library on the Lubancat board, and write some test codes through jupyter to use the library.

15.2 NumPy library installation

We use the apt tool to install, and what we install here is the Python3 version of the Numpy library. If you need the NumPy library in the Python2 environment, please delete 3 in the following commands by yourself.

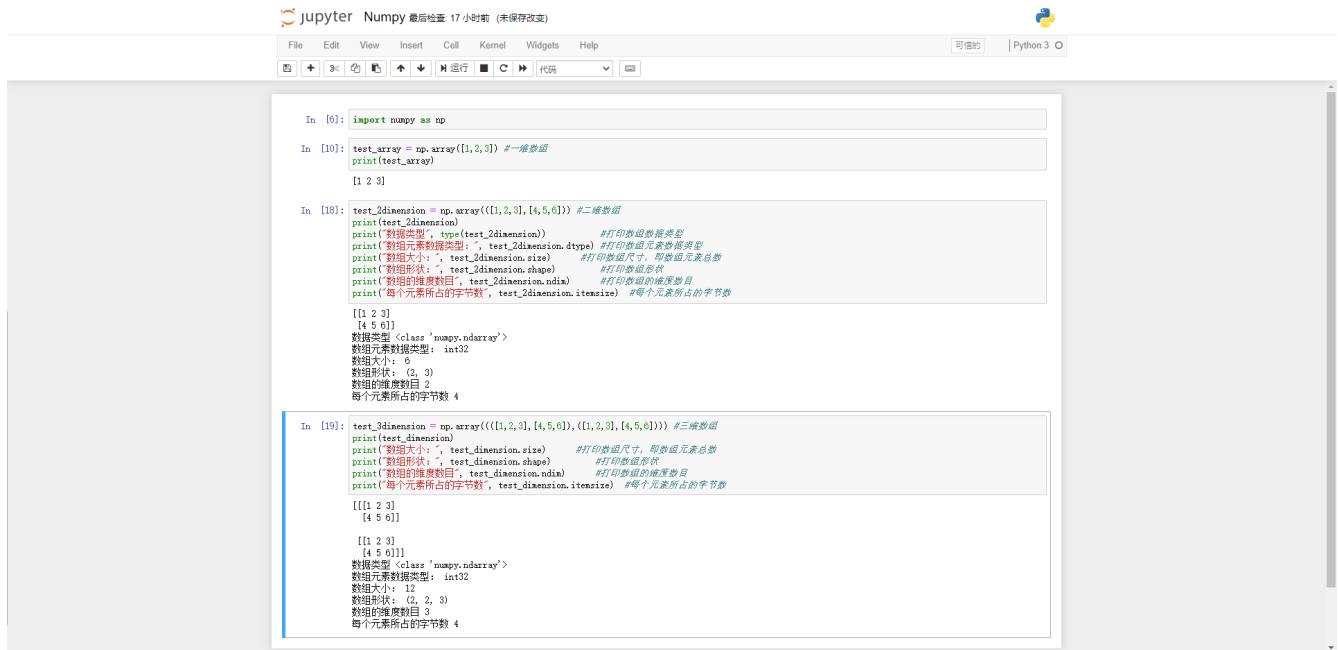
.. code-block:: bash

```
# Enter the following command in the terminal to install the numpy library, and wait for the  
installation to complete: sudo apt -y install python3-numpy
```

15.3 NumPy library used

After installing the corresponding library, we can use the jupyter installed earlier to write the test code (for jupyter installation, refer to the previous **Jupyter Notebook** chapter).

The test code and its effects are as follows:



```
In [6]: import numpy as np

In [10]: test_array = np.array([1,2,3]) #一维数组
print(test_array)
[1 2 3]

In [18]: test_2dimension = np.array([[1,2,3],[4,5,6]]) #二维数组
print(test_2dimension)
[[1 2 3]
 [4 5 6]]
print("数据类型: ", type(test_2dimension)) #打印数组数据类型
print("数组元素数据类型: ", test_2dimension.dtype) #打印数组元素数据类型
print("数组大小: ", test_2dimension.size) #打印数组大小, 即数组元素总数
print("数组形状: ", test_2dimension.shape) #打印数组形状
print("数组的维度数目", test_2dimension.ndim) #打印数组的维度数目
print("每个元素所占的字节数", test_2dimension.itemsize) #每个元素所占的字节数
[[1 2 3]
 [4 5 6]]
数据类型: <class 'numpy.ndarray'>
数组元素数据类型: int32
数组大小: 6
数组形状: (2, 3)
数组的维度数目: 2
每个元素所占的字节数: 4

In [19]: test_3dimension = np.array([[[1,2,3],[4,5,6]], [[1,2,3],[4,5,6]]]) #三维数组
print(test_3dimension)
[[[1 2 3]
 [4 5 6]]
 [[1 2 3]
 [4 5 6]]]
print("数据类型: ", type(test_3dimension)) #打印数组数据类型
print("数组元素数据类型: ", test_3dimension.dtype) #打印数组元素数据类型
print("数组大小: ", test_3dimension.size) #打印数组大小, 即数组元素总数
print("数组形状: ", test_3dimension.shape) #打印数组形状
print("数组的维度数目", test_3dimension.ndim) #打印数组的维度数目
print("每个元素所占的字节数", test_3dimension.itemsize) #每个元素所占的字节数
[[[1 2 3]
 [4 5 6]]
 [[1 2 3]
 [4 5 6]]]
数据类型: <class 'numpy.ndarray'>
数组元素数据类型: int32
数组大小: 12
数组形状: (2, 2, 3)
数组的维度数目: 3
每个元素所占的字节数: 4
```

You can upload the supporting code directory “libdemo\Numpy\Numpy.ipynb” file to jupyter for use.

For more information about the use of the Python Numpy library, please refer to [NumPy Tutorial](#)

Chapter16 Pandas library

16.1 Introduction to the Pandas library

Pandas is a tool based on NumPy, which is often used in the field of data analysis. Pandas can be used to process all kinds of messy data or tabular data. The library provides many tools needed to operate large data sets. We can install the Python-Pandas library on the Lubancat board, and write some test codes through jupyter to use the library.

16.2 Pandas library installation

We use the apt tool to install.

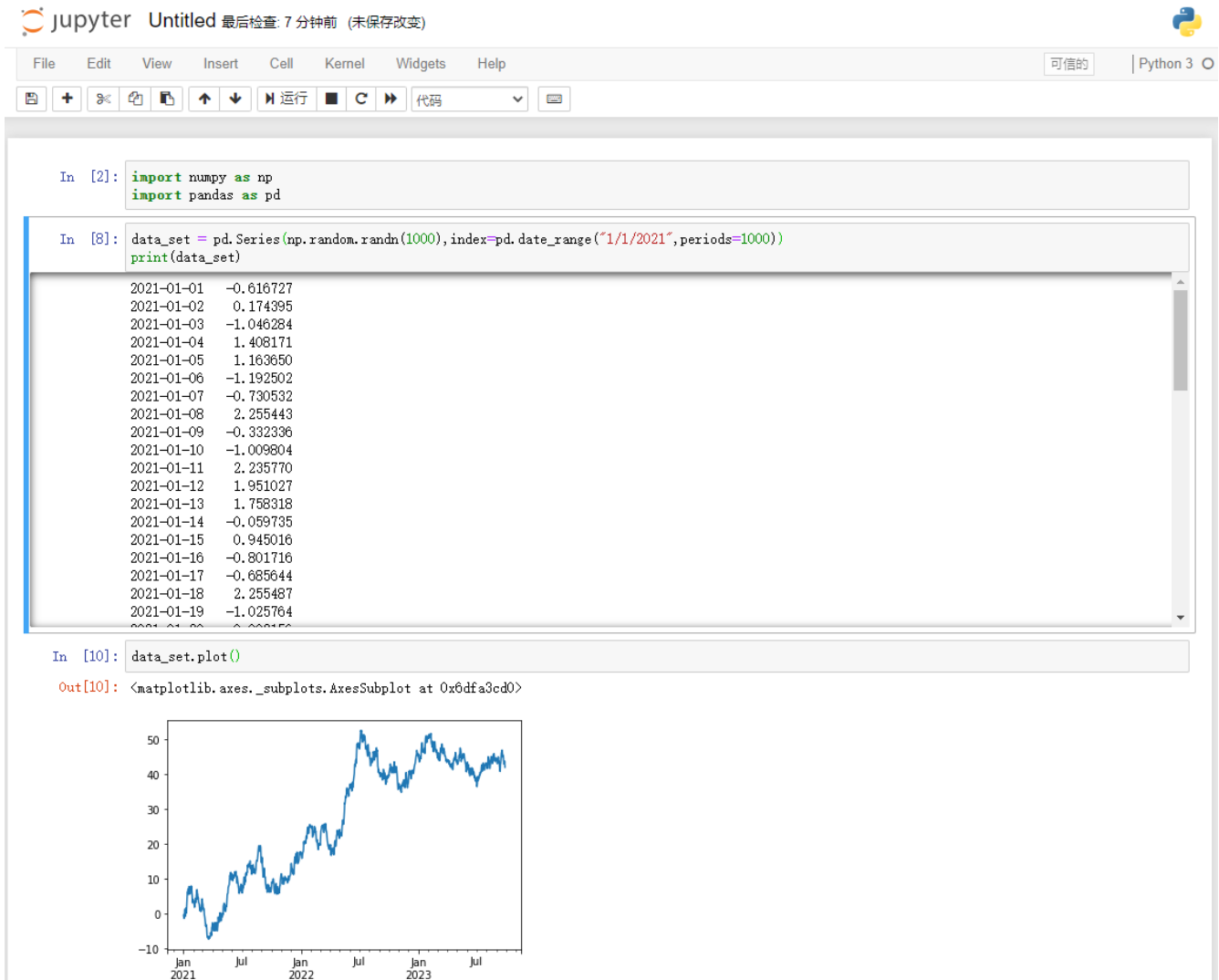
```
# Enter the following command in the terminal to install the Pandas library:  
sudo apt -y install python3-pandas
```

Wait until the installation is complete.

16.3 Pandas library usage

After installing the corresponding library, we can use the jupyter installed earlier to write the test code.

The test code and its effects are as follows:



You can upload the supporting code directory “libdemo\Pandas\Pandas.ipynb” file to jupyter for use.

For more information about the use of the Python Numpy library, please refer to [Pandas Tutorial](#)

Chapter17 Web library - Flask

17.1 Introduction to the Flask library

Flask is one of the most common web frameworks for building web applications in Python, same as [Django](#). The difference is that Flask is very lightweight, making it very easy to build basic web applications.

We can install the Python-flask library on the Lubancat board and use the library by writing some test code. Deploy a simple web page on our board.

17.2 Flask library installation

We use the apt tool to install.

```
# Enter the following command in the terminal to install the flask library:  
sudo apt -y install python3-flask  
  
#Or  
  
pip3 install flask
```

Wait until the installation is complete.

17.3 Flask library usage

After installing the corresponding library, we can use the library to deploy a simple web page.

17.3.1 New project directory

Create a new directory first, which is convenient for project management, and the file directory is relatively simple.

```
# Create a new directory to store our web projects and enter the directory
mkdir webapp_flask && cd webapp_flask

# Create the directory of our webapp
mkdir -p app/static
mkdir app/templates
mkdir tmp
```

17.3.2 Add code file

17.3.2.1 __init__.py

Create a simple initialization script for our webapp_flask package, placed in the app directory we just created:

```
# Create a new __init__.py in the app directory
vim __init__.py
```

The content is as follows:

列表 1: __init__.py

```
1 from flask import Flask
2
3 app = Flask(__name__)
4 from app import view
```

17.3.2.2 views.py

In Flask, views are written as Python functions. Each view function maps to one or more requested URLs.

Let's write the first view function views.py, the file is placed in the created app directory:

```
# Create a new view.py in the app directory
vim view.py
```

The content is as follows:

列表 2: view.py

```
1 from app import app
2
3 @app.route('/')
4 @app.route('/index')
5 def index():
6     return "Hello, Lubancat!"
```

17.3.2.3 run.py

The final step is to create a script that starts our application's web server. This script is run.py, and put it in our webapp_flask directory, which is the top-level directory of the project webapp_flask:

```
# Create a new run.py in the webapp_flask directory
nano run.py
```

The content is as follows:

列表 3: run.py

```
1 #!/usr/bin/python3
2 from app import app
3 # Debug mode, accessible via host address "192.168.103.108" 访问
4 app.run(debug = True, host="192.168.103.108")
```

After run.py is created, some permissions need to be granted:

```
# Add permissions
chmod a+x run.py
```

The final file directory organization is as follows:


```
cat@lubancat:~/webapp_flask$ tree ../webapp_flask/
../webapp_flask/
|-- app
|   |-- __init__.py
|   |-- static
|   |-- templates
|   |-- view.py
|-- run.py
|-- tmp
4 directories, 3 files
```

17.3.3 Run webapp

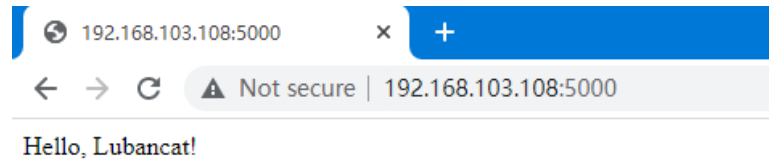
After the above operations, one of our simplest webapps has been organized. Next, we can start the app and view the effect through the browser.

```
# run app
./run.py
```

The test code and its effects are as follows:

```
cat@lubancat:~/webapp_flask$ ./run.py
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Running on http://192.168.103.108:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 347-097-024
```

The app has started normally, and the terminal has printed out the service information. The information indicates that the web has been mapped to the host whose IP address is 192.168.103.108 and port number is 5000 that we set in the previous code. Therefore, we can access our deployed webpage <http://192.168.103.108:5000/> through this URL.



You can upload the supporting code libdemo\Flask\webapp_flask directory to the lubancat board for testing.

17.3.4 Reference

[Flask official document](#)

Chapter18 Web library - Django

18.1 Introduction to Django Libraries

Django is an open source model-view-controller (MVC) style web application framework driven by the Python programming language, and it is also the mainstream framework for web development. We can install the Python-Django library on the Lubancat board and use the library through some simple operations. Deploy a simple web page on our board.

18.2 Django library installation

We use the apt tool to install.

```
# Enter the following command in the terminal to install the Django library:  
sudo apt -y install python3-django
```

Wait until the installation is complete.

18.3 Django library uses

After installing the corresponding library, we can use the library to deploy a simple web page.

18.3.1 New project directory

Create a new directory first, which is convenient for project management, and the file directory is relatively simple.

```
# Create a new directory to store our web projects and enter the directory
mkdir webapp
cd webapp
```

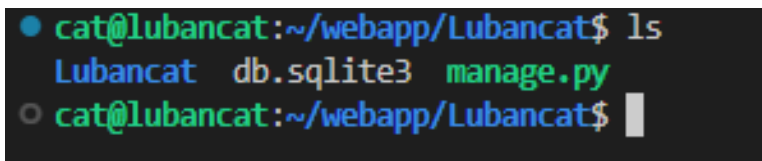
18.3.2 Add code files and configuration

Using Django's built-in manager `django-admin`, we can quickly create a project template.

```
# Enter the following command in the terminal:
django-admin startproject Lubancat

# After the command is executed, the project folder will be automatically
→ generated in the current directory: Lubancat. And then enter the folder
cd ./Lubancat/
```

The manager that comes with Django has already generated a project template for us, and you can see the following content in the directory:



```
cat@lubancat:~/webapp/Lubancat$ ls
Lubancat  db.sqlite3  manage.py
cat@lubancat:~/webapp/Lubancat$
```

Next, we need to modify the configuration file in the project to allow external hosts to access our Lubancat board. Go to the Lubancat directory in the project folder Lubancat and modify the `settings.py` file.

```
# Enter the following command in the terminal:
cd ./Lubancat/
```

(下页继续)

(续上页)

```
# Modify the configuration file
vim settings.py
# Find the ALLOWED_HOSTS configuration, and modify it to the following
→content, save and exit:
ALLOWED_HOSTS = ['*']
```

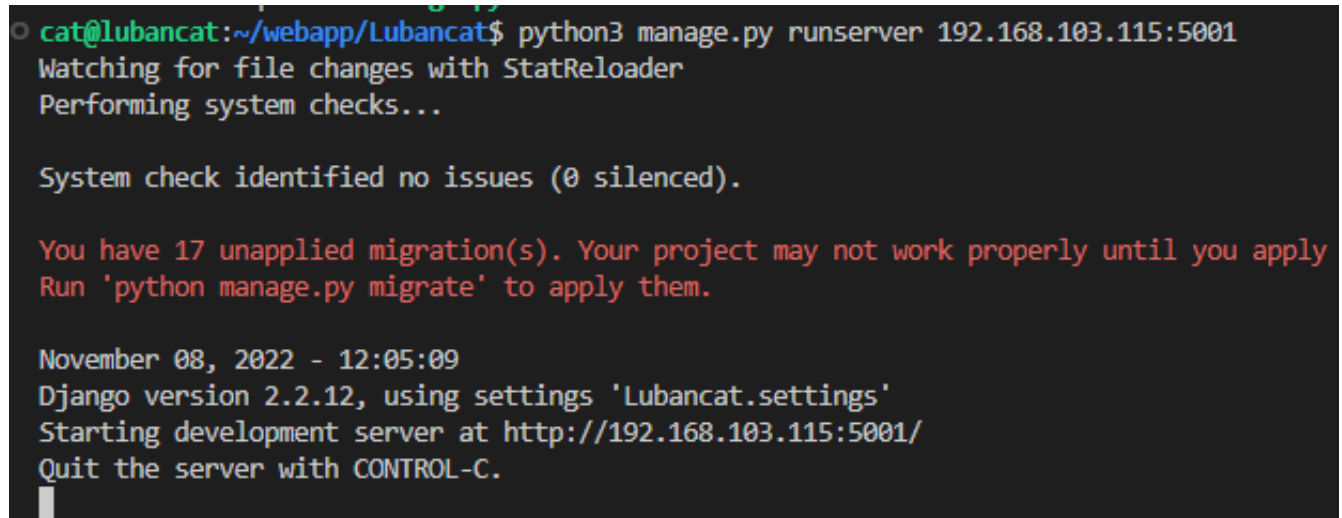
18.3.3 Start webapp

Go back to the project folder Lubancat, run the manage.py file in its directory, and start the server:

```
# Enter the following command in the terminal, and the IP address depends
→on the specific network port ip of the board:
sudo python3 manage.py runserver 192.168.103.115:5001
```

After a while, you can find that the server startup information has been output normally.

The effect is as follows:



```
cat@lubancat:~/webapp/Lubancat$ python3 manage.py runserver 192.168.103.115:5001
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 17 unapplied migration(s). Your project may not work properly until you apply
Run 'python manage.py migrate' to apply them.

November 08, 2022 - 12:05:09
Django version 2.2.12, using settings 'Lubancat.settings'
Starting development server at http://192.168.103.115:5001/
Quit the server with CONTROL-C.
```

The webapp has started normally, and the terminal has printed out the service information. The information indicates that the web has been mapped to the host we set the IP address to 192.168.103.115 and the port number to 5001 in the previous code. Therefore, we can visit <http://192.168.103.115:5001/> <<http://192.168.103.115:5001/>

[//192.168.103.115:5001/](http://192.168.103.115:5001/)>'_to deployed webpage through this URL.



The install worked successfully! Congratulations!

You are seeing this page because `DEBUG=True` is in your settings file and you have not configured any URLs.

18.3.4 Django Version Notes

Note: The Django installed using the apt method in the Lubancat board system is version 1.11.29. If you need to install a specific version of Django, you can use pip to install it, as follows:

```
# Enter the following command in the terminal to install the specified_
↪version of Django:
pip3 install Django==3.1.7
```

After the installation is complete, according to the prompt, put the management tool directory in the environment variable, or put the management tool in the environment variable.

```
The script django-admin is installed in '/home/debian/.local/bin' which is not on
PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning
, use --no-warn-script-location.
```



```
# Enter the following command in the terminal to put the management tool in
↪the environment variable:
sudo cp /home/cat/.local/bin/django-admin /usr/bin/
```

For more information about the use of the Python Django library, please refer to [Django official documentation](#)

Chapter19 Modbus protocol - pymodbus library

In the previous section, we introduced various external connection interfaces on the Lubancat board. You can find that some of these interfaces are widely used in the industrial field. Then, can our Lubancat board expand some powerful functions on the basis of these interfaces?

In this section of the experiment, we will introduce to you a very common industrial bus protocol in the industrial field: Modbus communication protocol, the example is Modbus-RTU.

19.1 Modbus protocol experiment

Our Lubancat board system has rich open source software support. The Modbus communication protocol we will introduce in this section also has corresponding software for us to install and use.

In this section of the experiment, our goal is to use the Python language and -python3-pymodbus to complete the simple communication between the RTU master station and the slave station agreed in the Modbus protocol.

So let's take a brief look at the python3-pymodbus library.

19.2 python3 - pymodbus library

pymodbus is a Modbus protocol Python library based on the BSD open source protocol. Its functions are very powerful, realizing all the functions agreed in the Modbus protocol, and realizing the communication host in a synchronous and asynchronous (asyncio, tornado, twisted) manner. While having good performance, it also provides more possibilities for Python developers to expand application functions when building Modbus protocol applications.

pymodbus supports Ethernet and serial interfaces to carry the physical layer transmission of the Modbus protocol. If you do not use the serial interface (the implementation of the pymodbus serial interface depends on the pyserial library package), then the python3-pymodbus library does not depend on third-party library packages at all, so the library is very lightweight.

We can install the python3-pymodbus library on the Lubancat board, And use the library through some official sample codes to complete the experiments in this section.

19.3 Experiment preparation

19.3.1 Add board uart resources

重要: In this section, the experiment uses a `uart` serial port (slave) and a PC (host) to adapt the connection of the Modbus underlying hardware. Therefore, this section is to demonstrate the addition of `uart` resources. The `pymodbus` library provides a full-stack implementation of the Modbus protocol. Therefore, the use of network port communication at the bottom layer is also supported. You can refer to the content of this section according to the actual situation.

Some GPIOs on the board may not be enabled, you can comment out the loading of some device tree nodes or device tree plug-ins, and restart the system to enable them. The pin reference of LubanCat_RK series boards is as follows: [《LubanCat-RK Series-40pin Pin Comparison Diagram》](#)

In this section of the experiment, the author uses the LubanCat 2 board as an example to demonstrate. According to the 40 pin comparison diagrams, the serial port resources can use UART3. Next, we will open the UART3 device tree plug-in:

```
# The configuration files of different boards are different from pwm,
↳ taking LubanCat 2 as an example, the configuration file is uEnvLubanCat 2.
↳ txt
# Use the following command:
```

(下页继续)

(续上页)

```
sudo vim /boot/uEnv/uEnvLubanCat 2.txt
#Enter edit mode, cancel the comment in front of pwm, save and exit the
↵file, restart the system
```

```
root@lubancat:/boot/uEnv# sudo vim uEnvLubanCat2.txt
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

#dtoverlay=dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-spi3-m1-overlay.dtbo
[dtoverlay=dtb/overlay/lubancat-uart3-m1-overlay.dtbo] 取消#注释, 保存重启, 开启串口

#overlay_end

~
~
~
~
"uEnvLubanCat2.txt" 19L, 610C                               16,1      All
```

If you are using other Lubancat RK series boards, the operation is similar.

```
# Enter the following command in the terminal to view the UART resource:
ls /dev/ttyS*
```

The UART resource example on the board is for reference only:

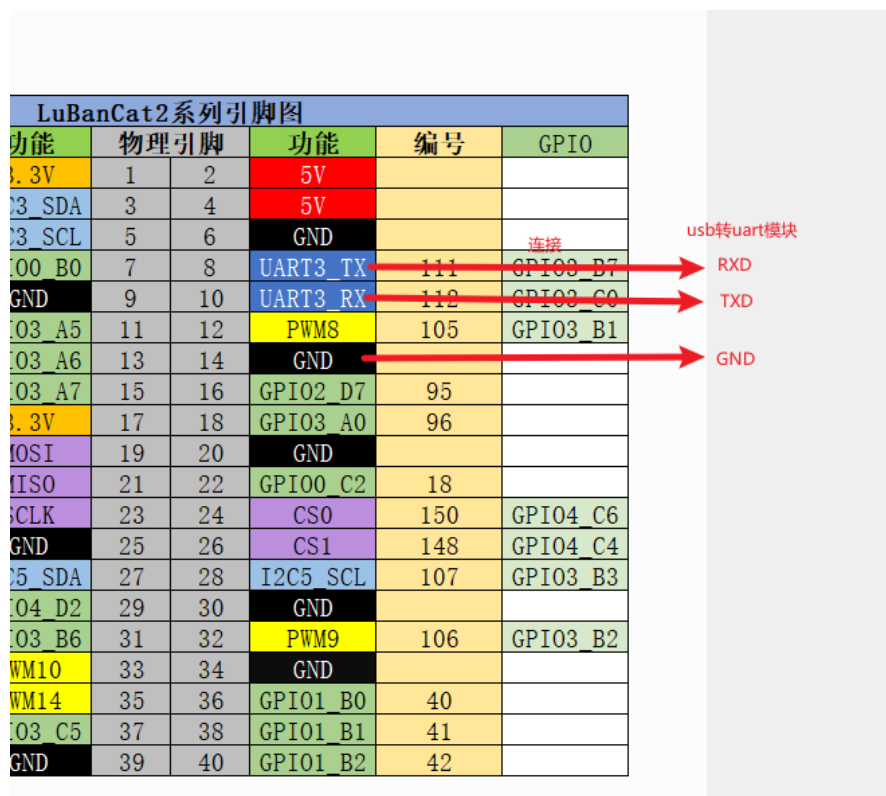
```
cat@lubancat:~$ ls /dev/ttyS*
/dev/ttyS3
cat@lubancat:~$ █
```

Among them, /dev/ttyS3 corresponds to the uart3 resource on the board.

提示: If `Permission denied` or similar words appear, please pay attention to user permissions. Most of the functions of operating hardware peripherals almost require root user privileges. The simple solution is to add `sudo` or run the program as root before executing the statement.

19.3.2 Hardware connection

For the hardware connection section, you need to make reference adjustments to the content of this chapter according to your own development environment and other conditions. This chapter will use a serial port in LubanCat 2: `uart3` to complete the demonstration of data transmission. The connection is as follows (the connection is the same as in **UART Communication** chapter):



Connect to computer via USB to TTL.

重要: The pymodbus library's support for serial devices is based on the pyserial library. The default serial device support of this library is based on uart, so this section also uses uart for demonstration experiments. If you want pymodbus to use the rs485 underlying electrical connection method, you need to modify the underlying source code of the pymodbus library by yourself.

19.3.3 pymodbus library installation

```
# Enter the following command in the terminal:
sudo pip3 install -U pymodbus

#Or directly pull the warehouse source code, use the setuptools tool to
install, etc., refer to the previous chapter on installing python
git clone https://github.com/riptideio/pymodbus.git
```

19.4 Test code

After installing the pymodbus library, you can use the pymodbus library to program. Here we directly modify the examples provided by the official, and then conduct a simple test. This Demo uses pymodbus based on the asyncio asynchronous library to implement Modbus protocol RTU communication and serial port asynchronous host. In the code, we set the external communication port, which is the serial port '/dev/ttyS3' opened in the hardware configuration section, with a baud rate of 115200.

19.4.1 RTU slave slave_server.py

Code show as below:

列表 1: Supporting code ‘slave_server.py’ (main part)

```
1 def get_commandline(server=False, description=None, extras=None):
2     """Read and validate command line arguments"""
3     parser = argparse.ArgumentParser(description=description)
4     parser.add_argument(
5         "--comm",
6         choices=["serial"],
7         help="set communication",
8         default="serial",
9         type=str,
10    )
11    parser.add_argument(
12        "--framer",
13        choices=["rtu"],
14        help="set framer, default depends on --comm",
15        type=str,
16    )
17    parser.add_argument(
18        "--log",
19        choices=["critical", "error", "warning", "info", "debug"],
20        help="set log level, default is info",
21        default="info",
22        type=str,
23    )
24    parser.add_argument(
25        "--port",
26        help="set port",
27        type=str,
28    )
29    if server:
30        parser.add_argument(
```

(下页继续)

(续上页)

```
31         "--store",
32         choices=["sequential"],
33         help="set type of datastore",
34         default="sequential",
35         type=str,
36     )
37     parser.add_argument(
38         "--slaves",
39         help="set number of slaves, default is 0 (any)",
40         default=0,
41         type=int,
42         nargs="+",
43     )
44     if extras:
45         for extra in extras:
46             parser.add_argument(extra[0], **extra[1])
47     args = parser.parse_args()
48
49     # set defaults
50     comm_defaults = {
51         "serial": ["rtu", "/dev/ptyp0"],
52     }
53     framers = {
54         "rtu": ModbusRtuFramer,
55     }
56     pymodbus_apply_logging_config()
57     _logger.setLevel(args.log.upper())
58     args.framer = framers[args.framer or comm_defaults[args.comm][0]]
59     args.port = args.port or comm_defaults[args.comm][1]
60     if args.comm != "serial" and args.port:
61         args.port = int(args.port)
62     return args
```

(下页继续)

(续上页)

```
63
64 def setup_server(args):
65     """Run server setup."""
66     # The datastores only respond to the addresses that are initialized
67     # If you initialize a DataBlock to addresses of 0x00 to 0xFF, a request_
↪to
68     # 0x100 will respond with an invalid address exception.
69     # This is because many devices exhibit this kind of behavior (but not_
↪all)
70     _logger.info("### Create datastore")
71     # Define the co coil register, the storage start address is 0, the_
↪length is 10, and the content is 5 True and 5 False
72     co_block = ModbusSequentialDataBlock(0, [True]*5 + [False]*5)
73     # Define the di discrete input register, the storage start address is 0,
↪the length is 10, and the content is 5 True and 5 False
74     di_block = ModbusSequentialDataBlock(0, [True]*5 + [False]*5)
75     # Define the ir input register, the storage start address is 0, the_
↪length is 10, and the content is a list of increasing values from 0 to 10
76     ir_block = ModbusSequentialDataBlock(0, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
77     # Define the hr holding register, the storage start address is 0, the_
↪length is 10, and the content is a list of decreasing values from 0 to 10
78     hr_block = ModbusSequentialDataBlock(0, [9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
79
80     if args.slaves:
81         # The server then makes use of a server context that allows the_
↪server
82         # to respond with different slave contexts for different unit ids.
83         # By default it will return the same context for every unit id_
↪supplied
84         # (broadcast mode).
85         # However, this can be overloaded by setting the single flag to_
↪False and
```

(下页继续)

(续上页)

```
86     # then supplying a dictionary of unit id to context mapping::
87     #
88     # The slave context can also be initialized in zero_mode which means
89     # that a request to address(0-7) will map to the address (0-7).
90     # The default is False which is based on section 4.4 of the
91     # specification, so address(0-7) will map to (1-8)::
92     context = {
93         0x01: ModbusSlaveContext(
94             di=di_block,
95             co=co_block,
96             hr=hr_block,
97             ir=ir_block,
98             zero_mode=True
99         ),
100         0x02: ModbusSlaveContext(
101             di=di_block,
102             co=co_block,
103             hr=hr_block,
104             ir=ir_block,
105         ),
106         0x03: ModbusSlaveContext(
107             di=di_block,
108             co=co_block,
109             hr=hr_block,
110             ir=ir_block,
111         ),
112     }
113     single = False
114     else:
115         context = ModbusSlaveContext(
116             di=di_block, co=co_block, hr=hr_block, ir=ir_block, unit=1
117         )
```

(下页继续)

(续上页)

```
118         single = True
119
120     # Build data storage
121     args.context = ModbusServerContext(slaves=context, single=single)
122     return args
123
124
125 async def run_async_server(args):
126     """Run server."""
127     txt = f"### start ASYNC server, listening on {args.port} - {args.comm}"
128     _logger.info(txt)
129     # socat -d -d PTY,link=/tmp/ptyp0,raw,echo=0,ispeed=9600
130     #             PTY,link=/tmp/ttyp0,raw,echo=0,ospeed=9600
131     server = await StartAsyncSerialServer(
132         context=args.context, # Data storage
133         #identity=args.identity, # server identify
134         timeout=1, # waiting time for request to complete
135         port=args.port, # serial port
136         # custom_functions=[], # allow custom handling
137         framer=args.framer, # The framer strategy to use
138         # handler=None, # handler for each session
139         stopbits=1, # The number of stop bits to use
140         bytesize=8, # The bytesize of the serial messages
141         # parity="N", # Which kind of parity to use
142         baudrate=115200, # The baud rate to use for the serial device
143         # handle_local_echo=False, # Handle local echo of the USB-to-RS485
144         ↪ adaptor
145         # ignore_missing_slaves=True, # ignore request to a missing slave
146         # broadcast_enable=False, # treat unit_id 0 as broadcast address,
147         # strict=True, # use strict timing, t1.5 for Modbus RTU
148         # defer_start=False, # Only define server do not activate
149     )
```

(下页继续)

(续上页)

```
149     return server
150
151 if __name__ == "__main__":
152     cmd_args = get_commandline(
153         server=True,
154         description="Run asynchronous server.",
155     )
156     run_args = setup_server(cmd_args)
157     asyncio.run(run_async_server(run_args), debug=True)
```

Using pymodbus to simulate the function of a Modbus slave device requires constructing a slave context environment. The environment is specified by the Modbus protocol, and it is necessary to realize various functions of the Modbus slave, such as various registers. In the end, the server is responsible for scheduling the operation of the context, and realizes the storage, reading and maintenance of the data of the slave by the master. On this basis, pymodbus also realizes the interface of the database. Interested you can check the source code and examples of pymodbus.

19.4.2 Experimental procedure

1. From the machine side, upload the supporting test code to the LubanCat 2 board system. Log in to the system terminal and start the slave service:

```
# Enter the following command in the terminal:
sudo python slave_server.py --comm serial --framer rtu --port /dev/ttyS3 --
↪store sequential --log debug --slaves 1
# --comm specifies the serial communication method,,, --framer, specifies_
↪the use of rtu frames, --port specifies the serial port, --log specifies_
↪the debug information
#You can use the command sudo python slave_server.py -h to view the help

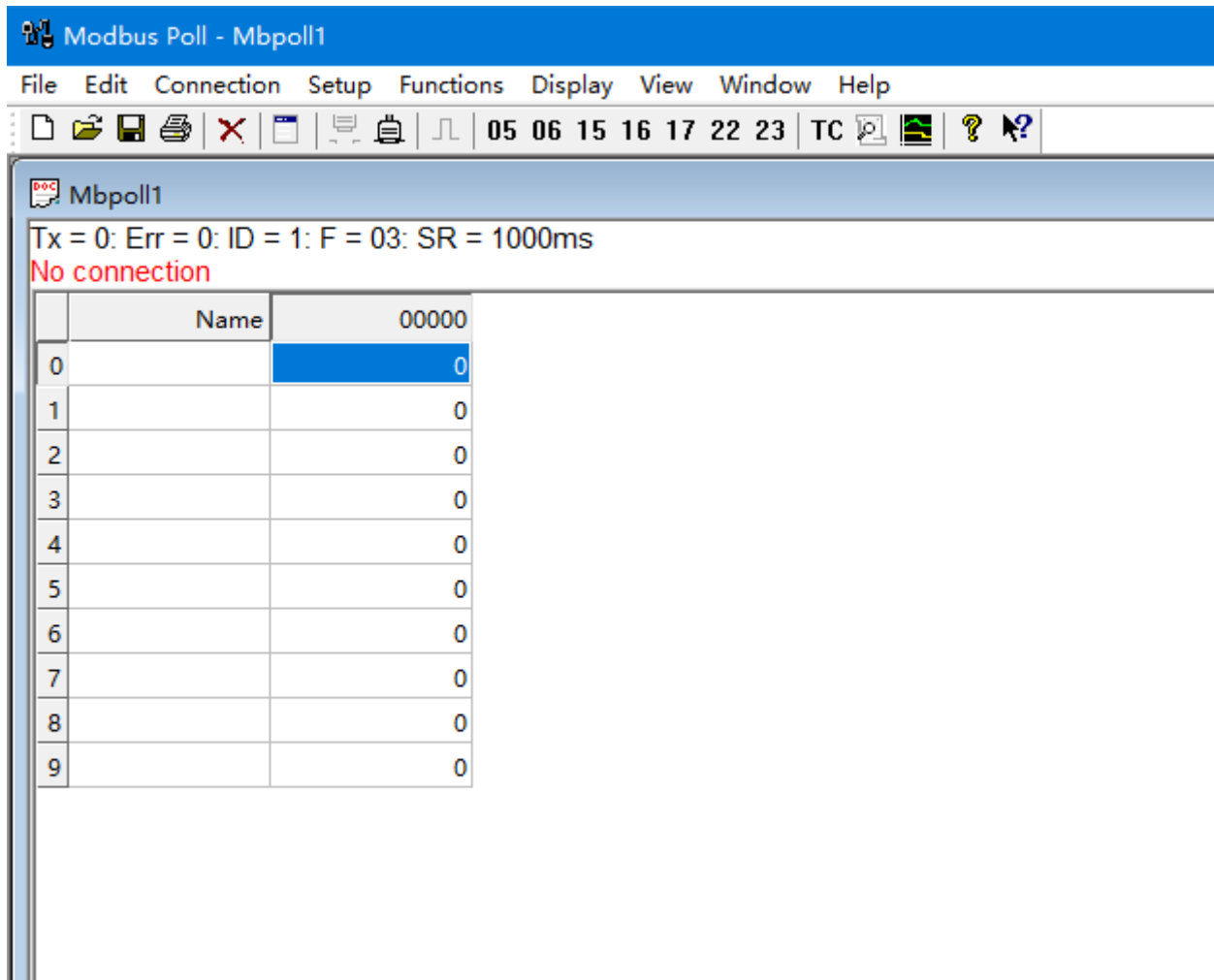
# After using the command, you can see in the terminal:
```

(下页继续)

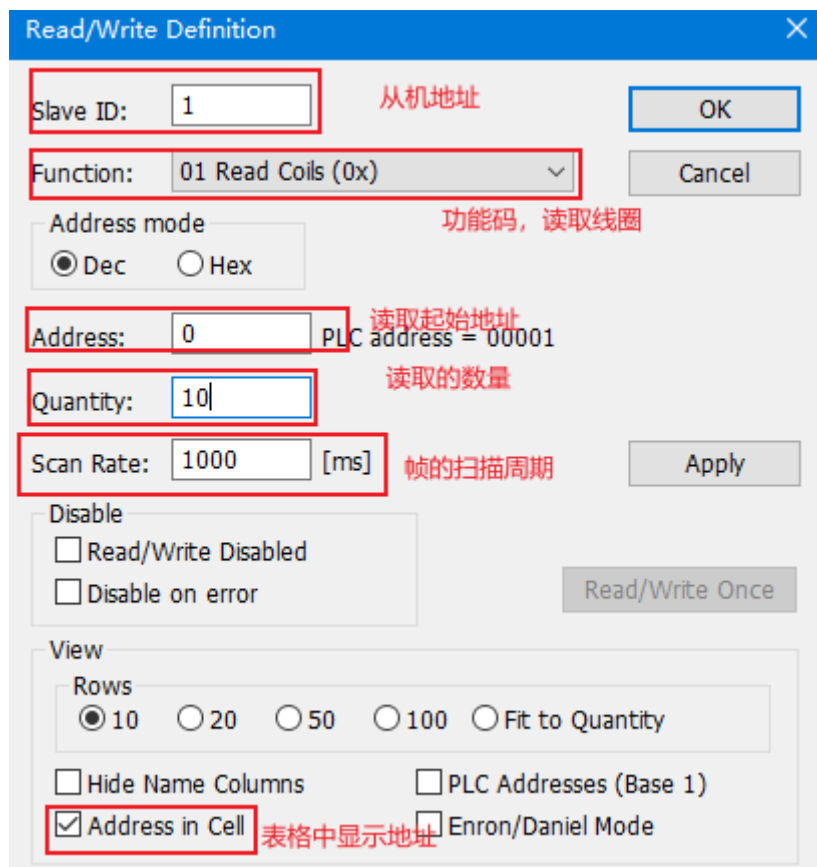
(续上页)

```
cat@lubancat:~$ sudo python server0.py --comm serial --framer rtu --port /  
↪ dev/ttyS3 --store sequential --log debug --slaves 1  
15:10:47 INFO server0:123 ### Create datastore  
15:10:47 DEBUG selector_events:59 Using selector: EpollSelector  
15:10:47 INFO server0:181 ### start ASYNC server, listening on /dev/ttyS3 -  
↪ serial  
15:10:47 DEBUG async_io:130 Serial connection opened on port: /dev/ttyS3  
15:10:47 DEBUG async_io:442 Serial connection established
```

2. On the host side, connect according to the previous hardware, and use the Modbus Poll software on the computer side (trial for 30 days). The download address is as follows: <https://modbustools.com/download.html> After installation, open the software:



Right-click on the blank space, select “Read/write Definition” , and then set the slave address, read and write, etc. Here we set the address to 1 and start reading 10 coils from address 0:

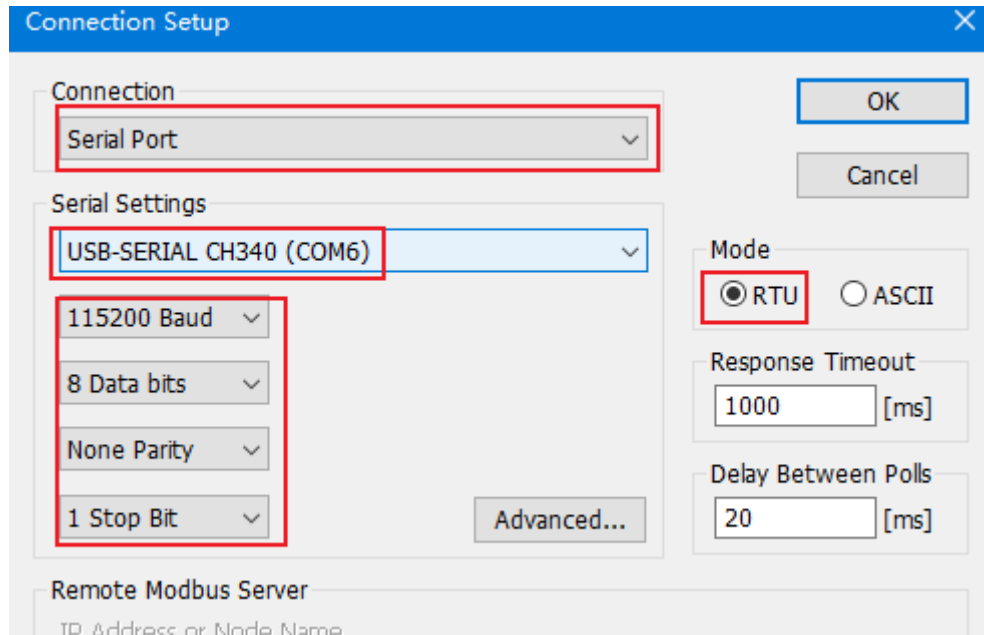


The image shows a 'Read/Write Definition' dialog box with the following settings:

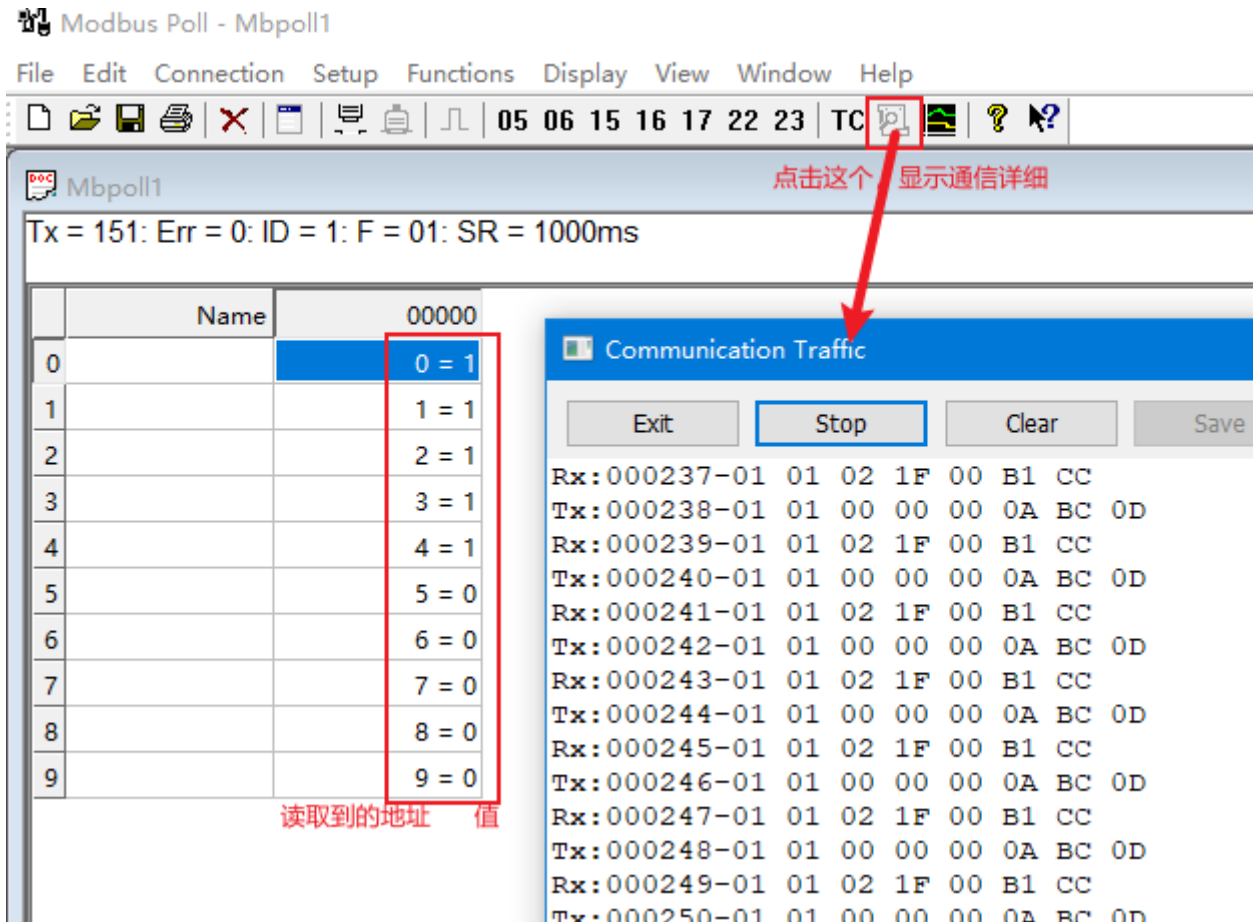
- Slave ID:** 1 (labeled '从机地址' - Slave Address)
- Function:** 01 Read Coils (0x) (labeled '功能码, 读取线圈' - Function Code, Read Coils)
- Address mode:** Dec (selected), Hex (unselected)
- Address:** 0 (labeled '读取起始地址' - Read Start Address, with note 'PLC address = 00001')
- Quantity:** 10 (labeled '读取的数量' - Read Quantity)
- Scan Rate:** 1000 [ms] (labeled '帧的扫描周期' - Frame Scan Period)
- Disable:**
 - ☐ Read/Write Disabled
 - ☐ Disable on error
- View:**
 - Rows:** 10 (selected), 20, 50, 100, Fit to Quantity
 - ☐ Hide Name Columns
 - ☐ PLC Addresses (Base 1)
 - ☒ Address in Cell (labeled '表格中显示地址' - Show address in table)
 - ☐ Enron/Daniel Mode

Buttons: OK, Cancel, Apply, Read/Write Once.

After the setting is complete, click “connection” in the main window, and then select the serial port. Here is COM6, the specific settings are as follows:



After the connection is successful, you can see the read value:

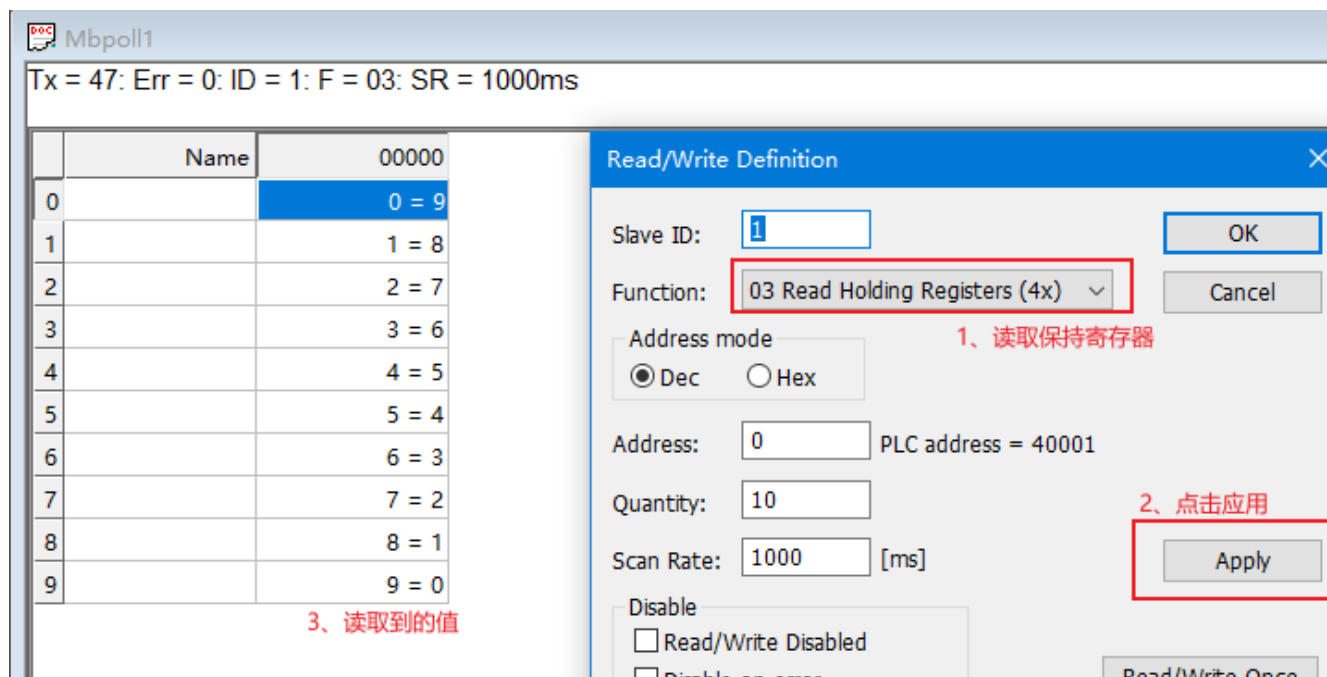


At the same time, the information printed by the board terminal is as follows (specify `-log debug` when executing):

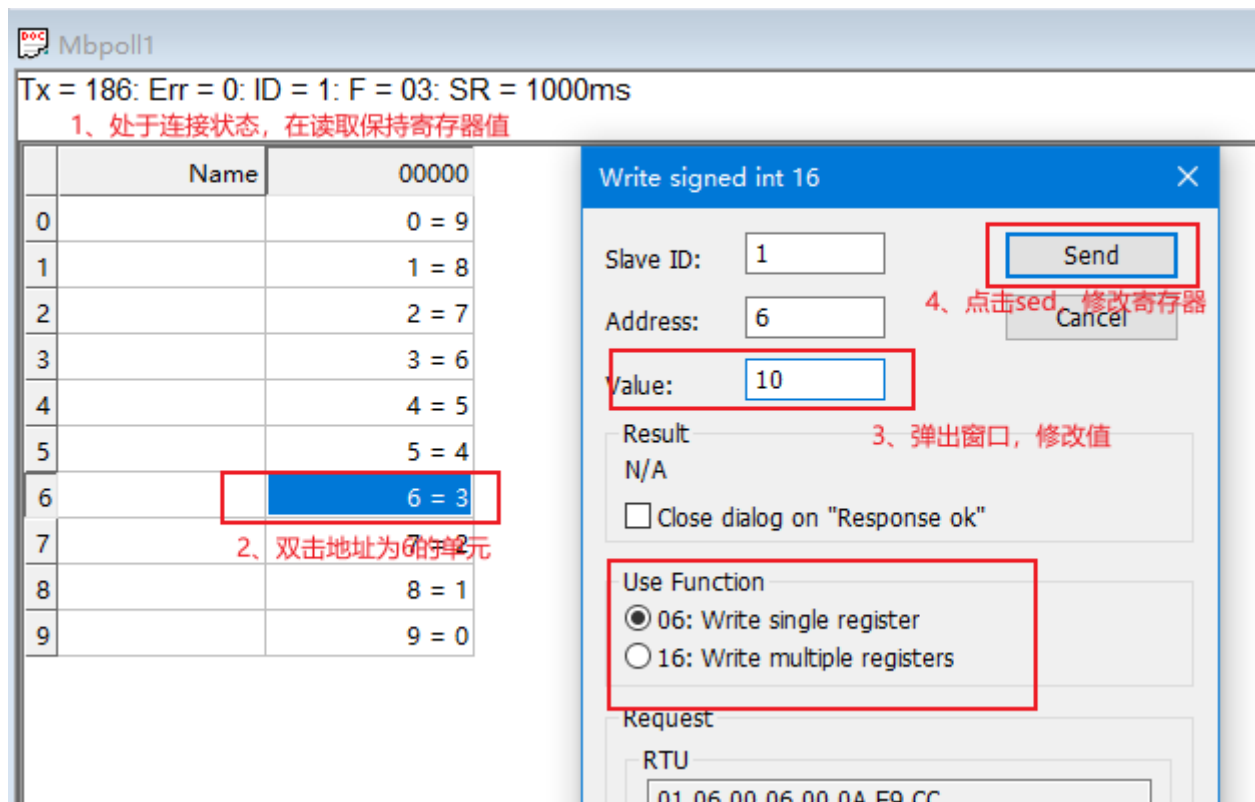
```
cat@lubancat:~$ sudo python server0.py --comm serial --framer rtu --port /dev/ttyS3 --store sequential --log debug --slaves 1
15:10:47 INFO server0:123 ### Create datastore
15:10:47 DEBUG selector_events:59 Using selector: EpollSelector
15:10:47 INFO server0:181 ### start ASYNC server, listening on /dev/ttyS3 - serial
15:10:47 DEBUG async_io:130 Serial connection opened on port: /dev/ttyS3
15:10:47 DEBUG async_io:442 Serial connection established
15:19:10 DEBUG async_io:222 Handling data: 0x1 0x1 0x0 0x0 0x0 0xa 0xbc 0xd
15:19:10 DEBUG rtu_framer:193 Getting Frame - 0x1 0x0 0x0 0x0 0xa
15:19:10 DEBUG factory:216 Factory Request[ReadCoilsRequest: 1]
15:19:10 DEBUG rtu_framer:116 Frame advanced, resetting header!!
15:19:10 DEBUG context:66 validate: fc-[1] address-0: count-10
15:19:10 DEBUG context:80 getValues: fc-[1] address-0: count-10
15:19:10 DEBUG async_io:299 send: [ReadCoilsResponse(10)]- b'0101021f00b1cc'
15:19:10 DEBUG rtu_framer:252 Frame - [b'\x01\x01\x00\x00\x00\n\xbc\r'] not ready
15:19:11 DEBUG async_io:222 Handling data: 0x1 0x1 0x0 0x0 0x0 0xa 0xbc 0xd
15:19:11 DEBUG rtu_framer:193 Getting Frame - 0x1 0x0 0x0 0x0 0xa
15:19:11 DEBUG factory:216 Factory Request[ReadCoilsRequest: 1]
```

After the above is in the connected state, we can also perform operations such as reading and writing other

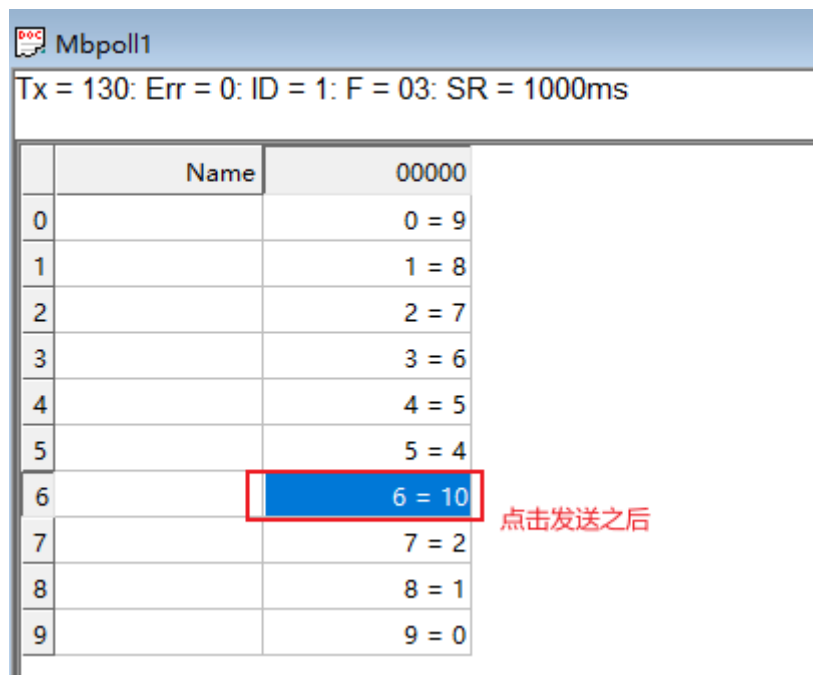
registers. Right-click the blank to enter “Read/write Definition” , and reset the following values:



Modify the value of the holding register:



After clicking Send, read the value of the register:

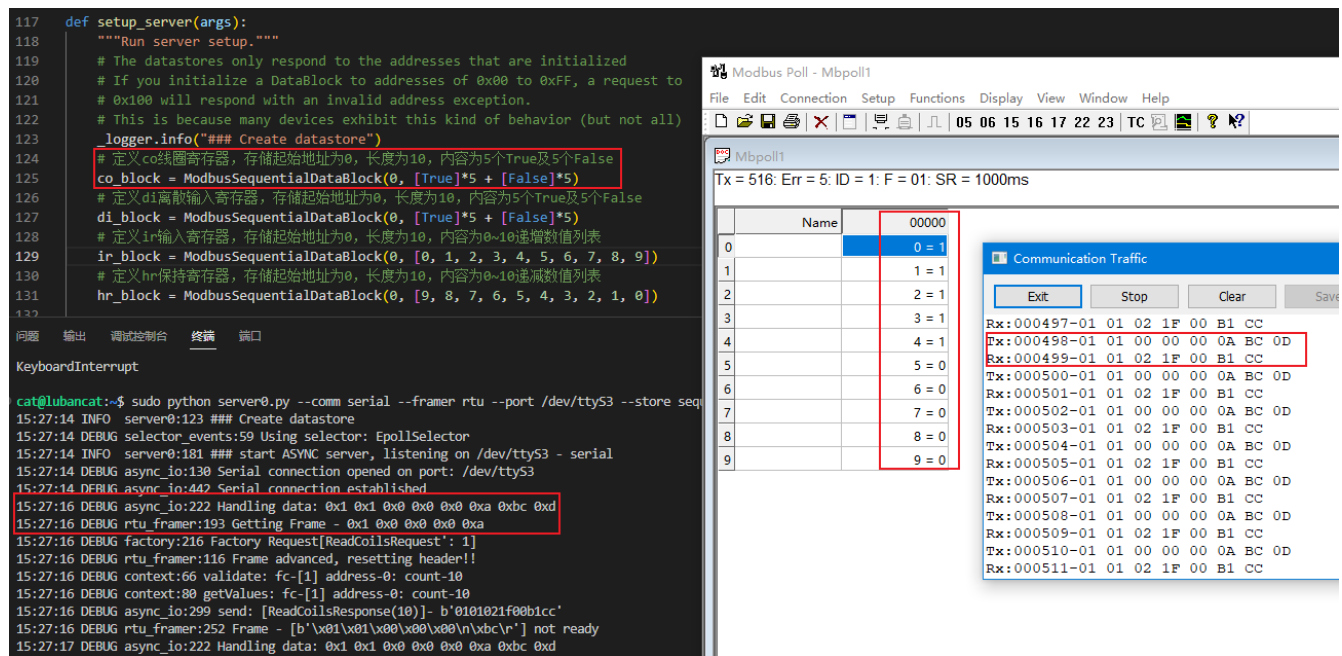


For more detailed operations such as reading, writing and debugging of some registers, you can refer to the help of the Modbus Poll software. In the above test process, it can be seen that the command sent by the host to the slave, the response information received from the slave, and various test results, etc.

19.4.3 Analysis of the phenomenon

We can simply look at some requests and responses that occur during the communication process of the Modbus-RTU master and slave through a picture.

Take Read Multiple Coil Test as an example (click the image to enlarge):



The screenshot shows a terminal window on the left and the Modbus Poll software on the right. The terminal window displays the output of a Python script running on a Raspberry Pi (LubanCat). The script defines a Modbus server with several data blocks. The output shows the server starting and handling a request. The Modbus Poll software on the right shows a table of data blocks. The 'Name' column is highlighted, and the '00000' value is selected. The 'Communication Traffic' window on the right shows the raw Modbus data being exchanged between the master and slave.

In the figure, on the upper computer side of the computer, modbus Poll is used to read and write single or multiple registers and corresponding parameters. When the board slave receives the command, it will make a corresponding response, and the log information of the response can be seen after enabling debug.

The pymodbus library also supports user-defined requests and responses, that is, the content related to user-defined function codes agreed in the Modbus protocol. In addition, the pymodbus library still implements some server functions, such as data persistence, which allows users to store the contents of slave registers through some common databases.

Earlier, we mentioned that the pymodbus library is a full-stack implementation of the Modbus protocol written in the Python language. In the sample code we provide you, only a part of the functionality of the library is shown. To test more, you can follow the Github repository of [pymodbus](#) .

19.4.4 Reference

Official sample code [pymodbus-examples](#) .

Official package and documentation description [pymodbus-doc](#) .

Chapter20 GUI library - PyQt5

This chapter will briefly introduce PyQt5, first use PyQt to create a simple desktop application, and then use Qt Creator on the Lubancat board for simple ui design. The following simple examples are for learning reference, and for more in-depth learning, see the reference materials.

- Platform: Lubancat RK series board
- System: Debian10 (with desktop)
- Python version: Python3.7
- Qt version: system default 5.11.3
- PyQt version: PyQt5

20.1 Introduction to PyQt5 library

PyQt is a combination of Qt application framework and Python, and supports both Python2.x and Python3.x versions. This tutorial uses Python3.7 version and PyQt5 version.

PyQt5 is composed of a series of Python modules, more than 620 classes, 6000 functions and methods. It can run on mainstream operating systems such as Unix, Windows and MacOS. And due to the convenience of Python language development, it is more convenient than C++ when writing code. The fly in the ointment is that the execution efficiency of Python language will be slower than C++.

PyQt5 provides GPL version and commercial version certificates. Free developers can use the free GPL license. If you need to use PyQt for commercial applications, you must purchase a commercial license.

20.2 PyQt5 library installation

20.2.1 Qt library installation

We use the apt tool to install PyQt5 (or use pip to install):

```
# Install via the apt tool
sudo apt-get -y install python3-pyqt5

# Install via pip
pip3 install pyqt5

# qt library installation
sudo apt-get install qt5-default
```

After successful installation:

```
cat@lubancat:~$ python3
Python 3.7.3 (default, Oct 31 2022, 14:04:00)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import PyQt5
>>>
```

Next, let's test the operation of PyQt5 on the Lubancat board.

重要: It is used in the desktop system, please make sure to connect the desktop screen, HDMI or mipi screen before the experiment.

20.3 Basic use of PyQt5 library

We will briefly introduce the writing of pyqt5 desktop applications through a PyQt5 application named pyqt5_demo.py. This application is mainly used to test the simple use of display, touch, signal, slot function, and some components of PyQt5 such as: QWidget, QLCDNumber, QSlider, QVBoxLayout, etc.

20.3.1 Import required modules

```
# Import the sys module, including some external parameters to the program,  
↪exit the program, obtain the system platform, system path and other  
↪operations  
import sys  
# Import some submodules of the QtWidgets module, QWidget class (user  
↪interface base class), QLCDNumber (display LCD-style numbers), QSlider  
↪(slider control)  
# QVBoxLayout (vertical layout of controls, etc.) QApplication (manages  
↪control flow and main settings of GUI applications, etc.)  
from PyQt5.QtWidgets import QWidget,QLCDNumber,QSlider,QVBoxLayout,  
↪QApplication  
# Import PyQt5.QtCore.Qt, some basic functions and classes  
from PyQt5.QtCore import Qt  
  
# If you find it troublesome, you can replace it with ``*`` to import all  
↪the memory, for example:  
from PyQt5.Qt import *
```

Commonly used modules include QtWidgets, QtCore, QtGui, etc. Among them, QtWidgets includes a set of UI element controls, which are used to establish an interface conforming to the system style, and the QtGui module covers a variety of basic graphic function classes (fonts, graphics, icons, colors). QtCore covers the core non-GUI interface functions (time, files, directories, data types, text streams, thread processes and other objects).

20.3.2 Create a class that will act as the main window

```
# Create a class called WinForm, based on QWidget
class WinForm(QWidget):
    # Initialization class
    def __init__(self):
        # Inherit the init of the parent class
        super().__init__()
        # Initialize the window itself
        self.initUI()

    def initUI(self):
        # Create the slider and LCD widgets first
        lcd = QLCDNumber(self)
        slider = QSlider(Qt.Horizontal, self)

        # Set the maximum value and default value of the slider, as
        ↪ well as the length and width, etc., set the default LCD style to display
        ↪ the value
        slider.setMaximum(1000)
        slider.setValue(666)
        slider.setMinimumWidth(200)
        slider.setFixedHeight(60)
        lcd.display(666)

        # Set the layout through QVBoxLayout, vertical distribution
        vBox = QVBoxLayout()
        vBox.addWidget(lcd)
        vBox.addWidget(slider)

        self.setLayout(vBox)
        # valueChanged() is a signal function of Qslider. As long as
        ↪ the value of the slider changes, it will emit a signal, and then connect
        ↪ the receiving part of the signal through connect, which is lcd.
```

(下页继续)

(续上页)

```
slider.valueChanged.connect(lcd.display)

style = "QSlider::groove:horizontal {border:1px solid #999999;
↪height:10px;" \
        "background-color:#666666;margin:2px 0;}" \
        "QSlider::handle:horizontal {background-color:#ff0000;
↪border:1px solid #797979;" \
        "width:50px;margin:-20px;border-radius:25px;}" \

# Set style
slider.setStyleSheet(style)

#self.setGeometry(0,0,800,480)
# Set window title
self.setWindowTitle("Drag the slider to control the digital_
↪display")
```

20.3.3 Create application

```
# name is the name of the current module, when the module directly runs as_
↪main, the following code block will run
if __name__ == '__main__':
    # Each application needs a QApplication instance, and sys.argv is to_
    ↪run the module to pass in the command line parameters, which can be empty.
    app = QApplication(sys.argv)
    # Control instance
    form = WinForm()
    # Set the size, then show the control
    form.resize(800, 480)
    form.show()
```

(下页继续)

(续上页)

```
# The program executes and enters the message event loop, and ends_
↪safely through the exit function
sys.exit(app.exec_())
```

Create a QApplication instance, run the program, create a window form as the main window, and then display it.

20.3.4 Overall program

The overall code of pyqt5_demo.py is as follows:

列表 1: Companion code ‘libdemo/PyQt5/pyqt5_demo.py’

```
1 import sys
2 from PyQt5.QtWidgets import QWidget,QLCDNumber,QSlider,QVBoxLayout,
  ↪QApplication
3 from PyQt5.QtCore import Qt
4
5 class WinForm(QWidget):
6     def __init__(self):
7         super().__init__()
8         self.initUI()
9
10    def initUI(self):
11        # Create the slider and LCD widgets first
12        lcd = QLCDNumber(self)
13        slider = QSlider(Qt.Horizontal, self)
14
15        slider.setMaximum(1000)
16        slider.setValue(666)
17        slider.setMinimumWidth(200)
18        slider.setFixedHeight(60)
```

(下页继续)

(续上页)

```
19         lcd.display(666)
20
21         # Set the layout through QVBoxLayout
22         vbox = QVBoxLayout()
23         vbox.addWidget(lcd)
24         vbox.addWidget(slider)
25
26         self.setLayout(vbox)
27         # valueChanged() is a signal function of Qslider. As long as the
↪value of the slider changes, it will emit a signal, and then connect to
↪the receiving part of the signal, which is lcd.
28         slider.valueChanged.connect(lcd.display)
29
30         style = "QSlider::groove:horizontal {border:1px solid #999999;
↪height:10px;" \
31                 "background-color:#666666;margin:2px 0;}" \
32                 "QSlider::handle:horizontal {background-color:#ff0000;
↪border:1px solid #797979;" \
33                 "width:50px;margin:-20px;border-radius:25px;}" \
34
35         slider.setStyleSheet(style)
36         #self.setGeometry(0,0,800,480)
37         self.setWindowTitle("Drag the slider to control the digital display
↪")
38
39 if __name__ == '__main__':
40     app = QApplication(sys.argv)
41     form = WinForm()
42     form.resize(800, 480)
43     form.show()
44     sys.exit(app.exec_())
```

The terminal directly runs our pyqt5_demo.py program:

```
sudo python3 pyqt5_demo.py
```

Display effect:



You can upload the supporting code directory 'libdemo\PyQt5\pyqt5_demo.py' to the development board for testing.

20.4 Qt Designer uses

In general, we use python programs to create our own interface applications, but as the application becomes larger or the interface becomes more complex, it may be cumbersome to define all widgets programmatically.

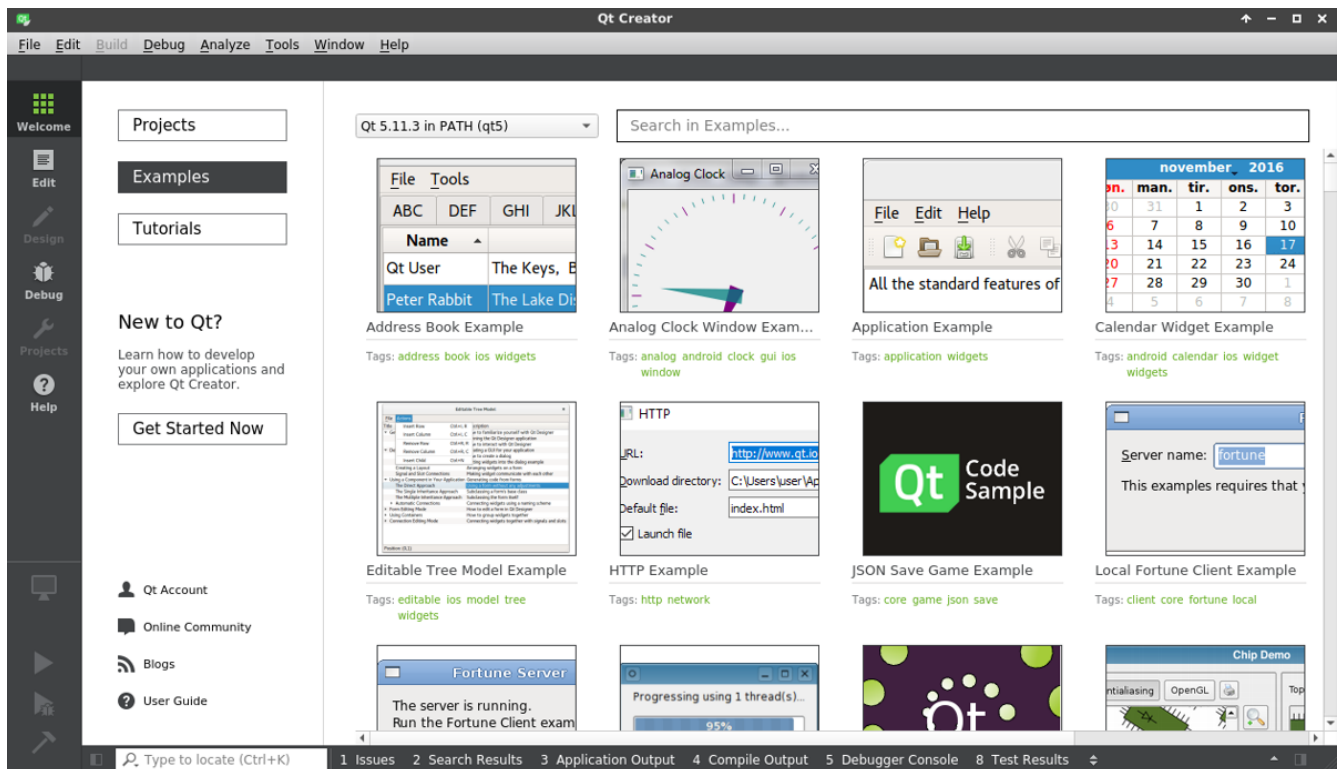
In this case, Qt Designer can be used to specifically create the UI interface in the PyQt program. The following will introduce the use of Qt Designer to create a simple login UI interface.

```
# Install the Qt Creator tool via apt
sudo apt install qtcreator

# Install tools such as qt designer
sudo apt install qttools5-dev-tools
```

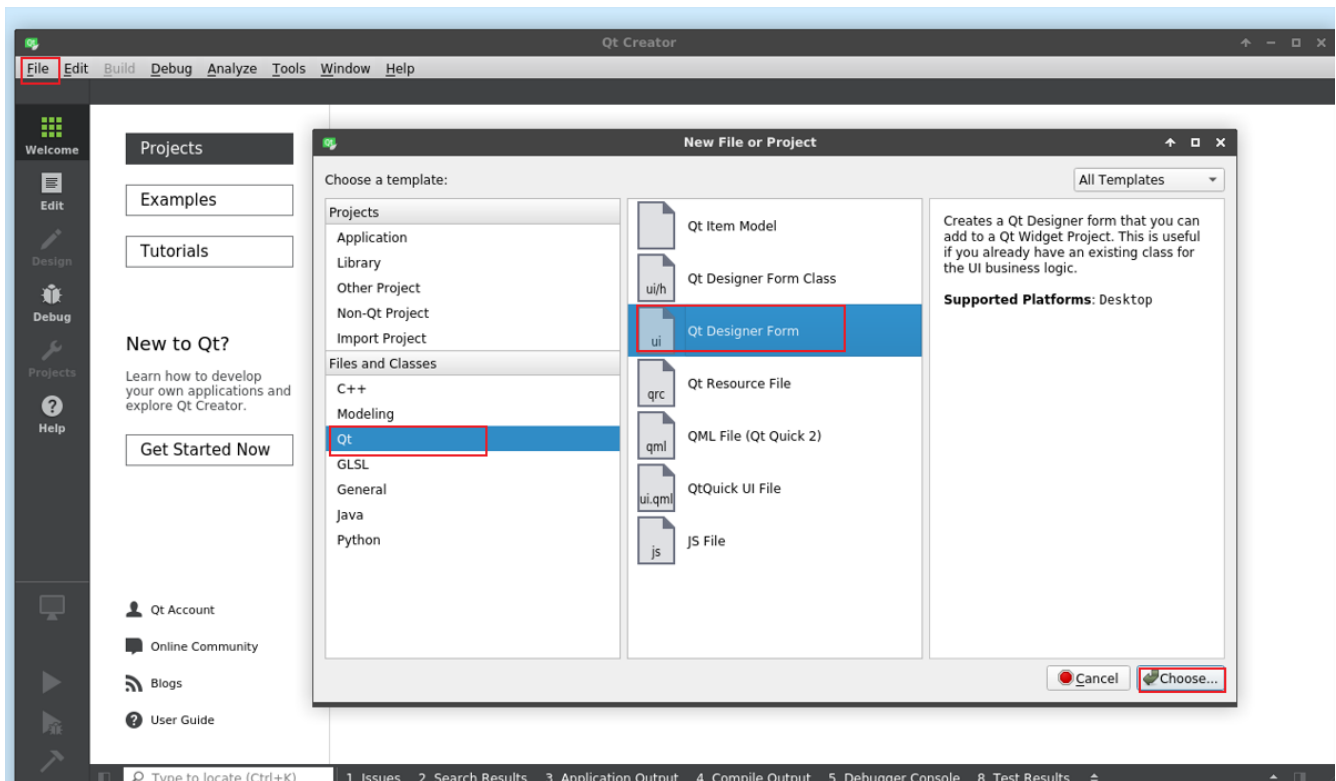
重要: When using Qt Creator in lubancat, when designing some complex interfaces, it will get stuck or get stuck directly, which may be due to insufficient memory. It is recommended to design ui or write pyqt5 programs in a virtual machine or PC, which can be run directly on the board.

After installation, you can search for Qt Creator in the desktop menu, or open Qt Designer directly, and then open the application:



20.4.1 Create an interface UI file

Open Qt Creator and click file, select go to File -> New File or Project, in the Files and Classes column, click QT, select QT Designer Form, Then set the created .ui file name, path, etc., as follows:

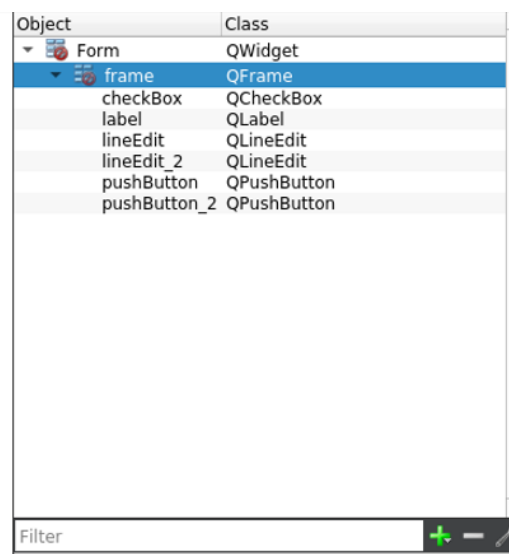
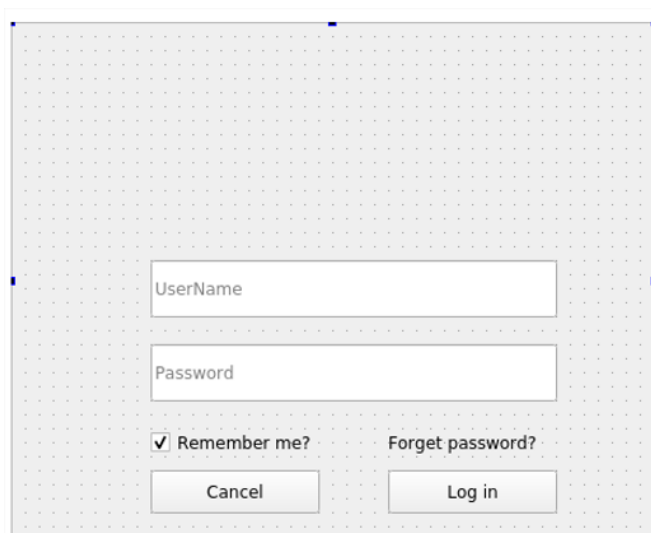
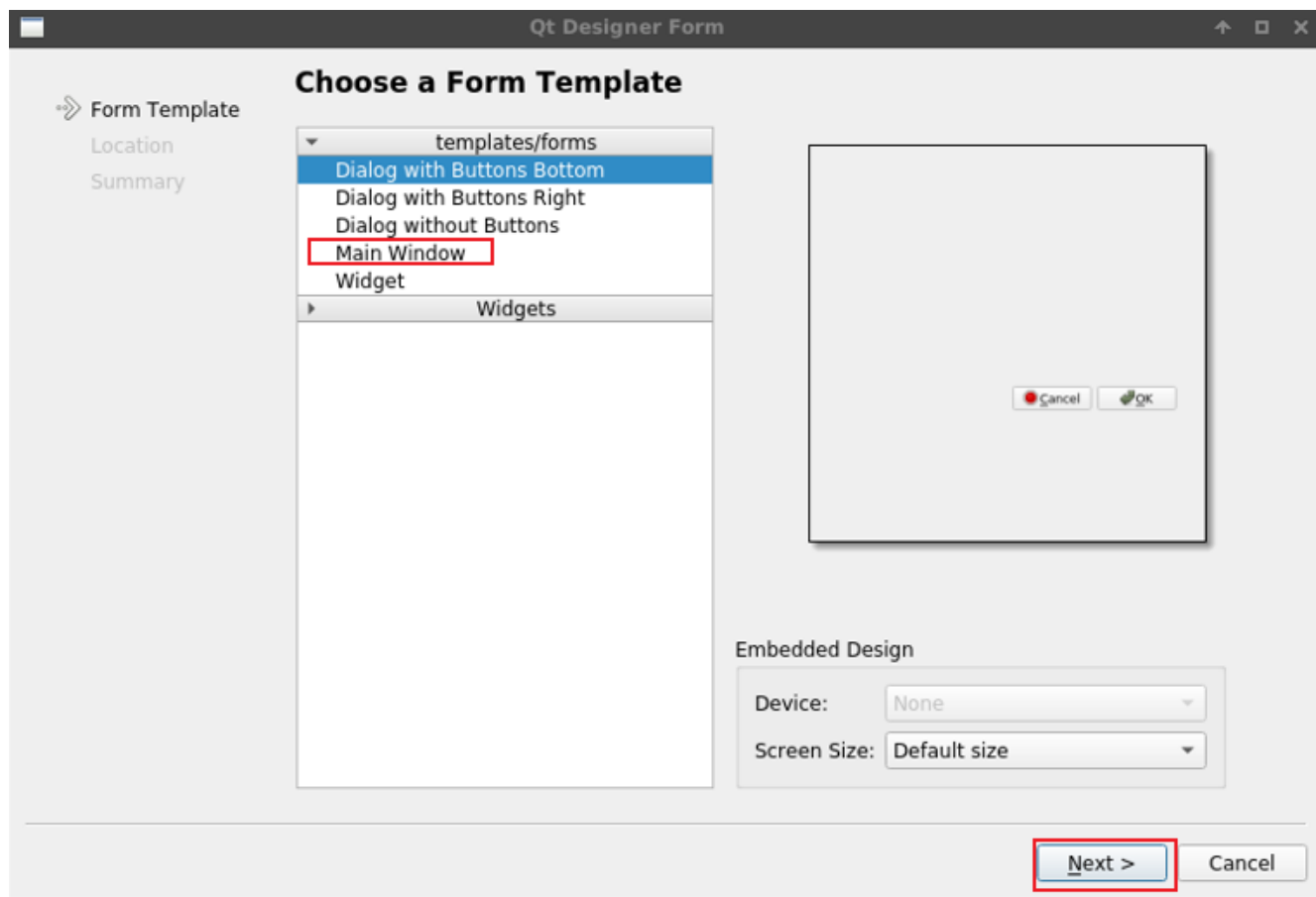


Then select Main Window or other, and finally set the file path and file name:

20.4.2 Interface construction

In the ui interface, add a container Frame to the canvas and adjust the size. Create a simple login interface that requires an account password field, use Input Widgets: Line Edit to log in and cancel using the button “Buttons: PushButton” . If you forget the password box, use the text field Display Widgets: TextLabe, remember the password, use Buttons: CheckBox, drag these controls to the window, set the name, and the layout is as follows:

To set the style, right-click the entire container, select `edit style shell`, and fill in the following



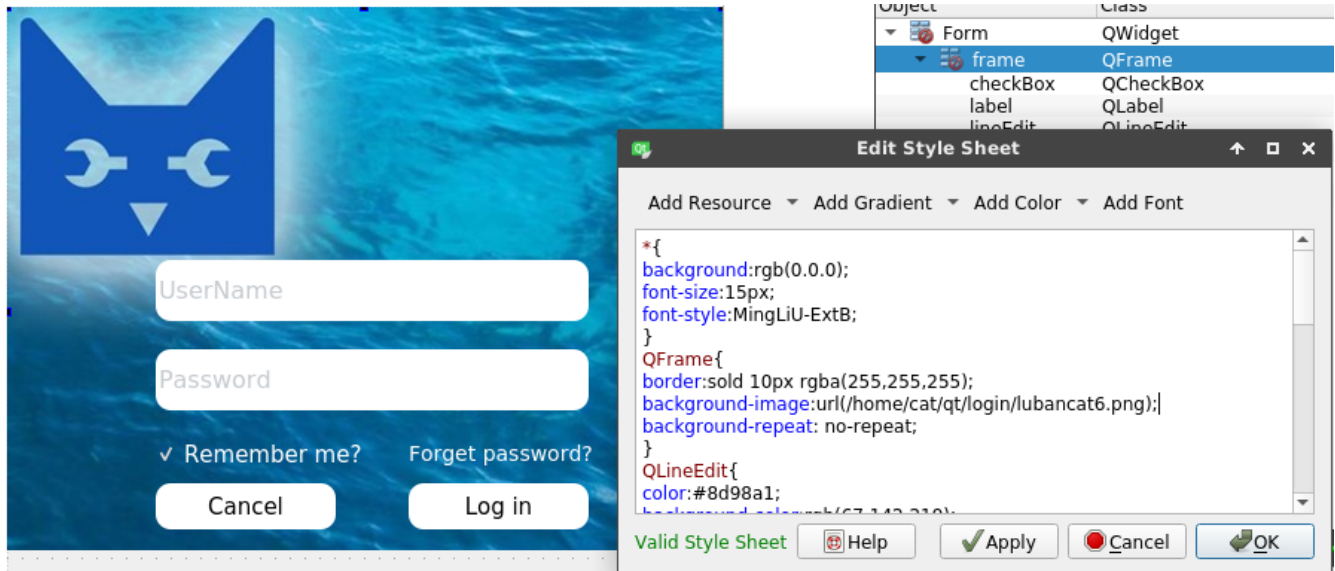
code. You can change the color, style, and choose the path of the background image according to your preferences. This is a simple configuration, and it looks okay:

```
*{
background:rgb(0, 0, 0);
font-size:15px;
font-style:MingLiU-ExtB;
}
QFrame{
border:solid 10px rgba(255,255,255);
background-image:url(/home/cat/qt/login/lubancat6.png);
}
QLineEdit{
color:#8d98a1;
background-color:rgb(0, 0, 0);
font-size:16px;
border-style:outset;
border-radius:10px;
font-style:MingLiU-ExtB;
}
QPushButton{
background:#ced1d8;
border-style:outset;
border-radius:10px;
font-style:MingLiU-ExtB;
}
QPushButton:pressed{
background-color:#405361;
border-style:inset;
font-style:MingLiU-ExtB;
}
QCheckBox{
background:rgba(85,170,255,0);
```

(下页继续)

(续上页)

```
color:white;
font-style:MingLiU-ExtB;
}
QLabel{
background:rgba(85,170,255,0);
color:white;
font-style:MingLiU-ExtB;
font-size:14px;
}
```



提示: For the design of ui and the use of qt designer, you can refer to the Qt-related tutorials.

20.4.3 Call ui file

Using the .ui file designed by qt designer, you can directly use the uic module to import the ui file. Or convert the .ui file into a python module and import it directly.

1. Import the .ui file directly using the uic module:

列表 2: Companion code “libdemo/PyQt5/login/login.py”

```
1 import sys
2 from PyQt5.QtWidgets import QApplication, QMainWindow
3 from PyQt5 import uic
4
5 class LoginWindow(QMainWindow):
6
7     def __init__(self, *args, **kwargs):
8         super().__init__(*args, **kwargs)
9         # Use uic.loadUi to import the .ui file in the current directory
10        uic.loadUi("login.ui", self)
11        self.setWindowTitle("Lubancat login")
12
13 if __name__ == '__main__':
14     app = QApplication(sys.argv)
15     window = LoginWindow()
16     window.show()
17     sys.exit(app.exec_())
```

2. Or convert to python file import

Use the pyuic5 command to convert the .ui file into a python file:

```
# Install the pyqt5-dev-tools tool via apt
sudo apt-get install pyqt5-dev-tools
```

(下页继续)

(续上页)

```
# Use the command to convert the .ui file into a python file
pyuic5 -o login.ui login_ui.py
```

Convert it into a python file, which can be directly imported and used:

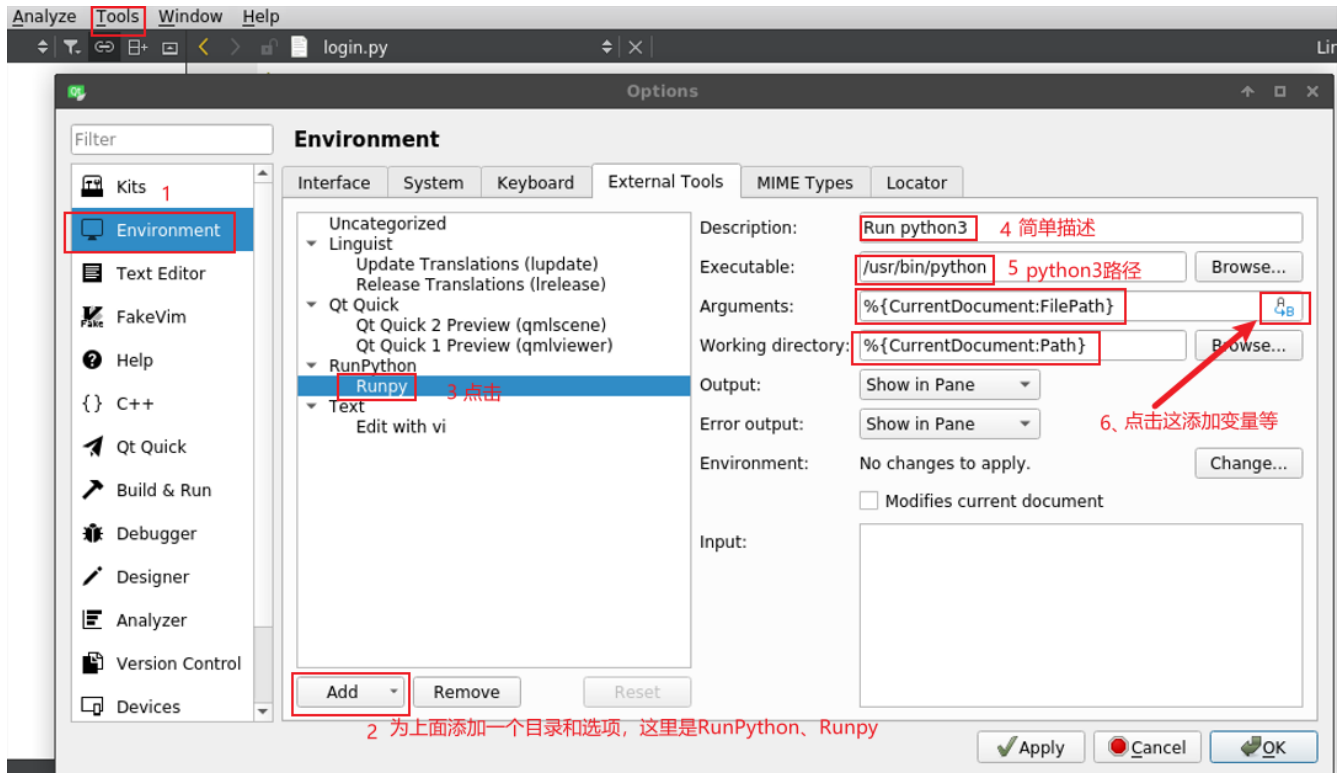
列表 3: Companion code” lib-
demo/PyQt5/login/login_ui_form.py”

```
1 import sys
2 from PyQt5.QtWidgets import QApplication, QMainWindow
3 # Import the converted Ui_Form class
4 from login_ui import Ui_Form
5
6 class LoginWindow(QMainWindow):
7
8     def __init__(self, parent=None):
9         super().__init__(parent)
10        # Create a ui instance
11        self.ui = Ui_Form()
12        # Run setupUi() to display ui
13        self.ui.setupUi(self)
14        self.setWindowTitle("Lubancat login")
15
16 if __name__ == "__main__":
17     app = QApplication(sys.argv)
18     win = LoginWindow()
19     win.show()
20     sys.exit(app.exec())
```

To run the program, you can use the command in the terminal:

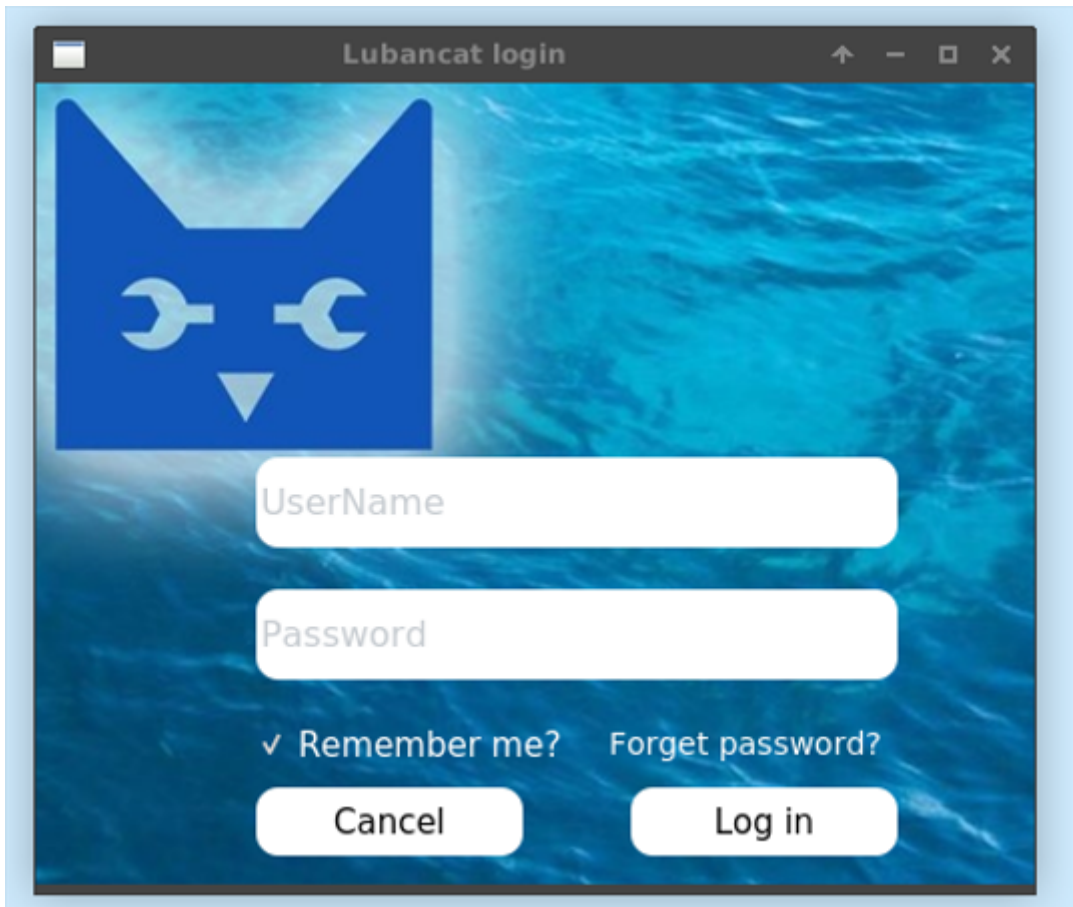
```
# Run the program with the command:
sudo python3 login.py
```

Or add a python runtime environment in Qt Creator. You need to click **Tool->Options...** on the interface and select **Environment** in the pop-up window, then operate according to the picture below, and finally click **apply** to apply.



When running, open the corresponding python file, and then click **Tool->External->RunPython->RunPy**.

Running display effect:



For the source code of the program, refer to the files in the libdemo/PyQt5 directory of the supporting routine. Finally, if you need to package and distribute your application, you can read some documents in the reference link.

20.5 Reference

For more PyQt5 related tutorials, please refer to:

<https://www.riverbankcomputing.com/static/Docs/PyQt5/introduction.html>

<https://www.pythonguis.com/latest/>

Chapter21 Artificial Intelligence

This module takes the deep learning of artificial intelligence as the theme, and is based on Python, allowing readers to develop artificial intelligence on the Lubancat RK series board. **Focus on the deployment of various deep learning frameworks on Lubancat. There are brief descriptions about the training and optimization of the framework model. For more detailed and systematic learning, please refer to various deep learning books and RKNN documents.**

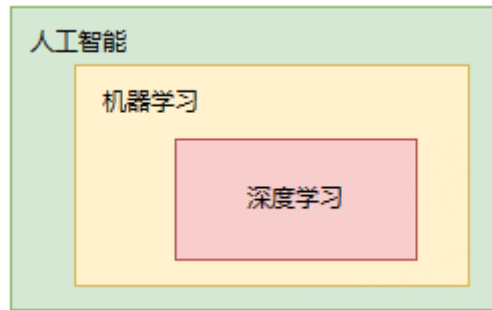
21.1 Introduction

1、**Artificial Intelligence** (AI) : Different scholars and institutions may have different definitions and understandings. The common definition is: artificial intelligence is a new technical science for simulating, extending and expanding human intelligence theories, methods, technologies and application systems. As for the specific implementation, there are many methods and branches of artificial intelligence. The machine learning and deep learning we often hear are currently more effective implementation methods.

2、**Machine Learning** (ML) : It is an important branch of artificial intelligence. It mainly studies how to let computers automatically improve performance through data or experience, by imitating human learning ability, learning from sample data to obtain experience (model), and then making predictions. Common algorithms for machine learning include supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning.

3、**Deep Learning** (DL) : It is the most popular branch of machine learning algorithms, which has made remarkable progress in recent years and replaced most traditional machine learning algorithms. Simply put, it is a machine learning algorithm based on artificial neural network. Deep learning has a wide range of applications in machine vision, speech recognition, natural language processing and other fields, and is especially suitable for solving perception problems.

The relationship between artificial intelligence, machine learning, and deep learning can be briefly referred to as follows:



4、**Deep Learning Framework:** It is a software library that can help you build, train, and deploy deep learning models. It usually provides advanced abstraction, predefined layers and optimizers, and automatic differentiation, making writing deep learning code easier and more efficient. There are many deep learning frameworks to choose from, such as:

- **TensorFlow:** One of the most popular deep learning frameworks open sourced by Google, based on the calculation method of data flow graph, it supports multiple platforms and languages.
- **PyTorch:** One of the latest deep learning frameworks open sourced by Facebook, based on the Torch library, written in Python language, supports dynamic graphs and automatic differentiation.
- **PaddlePaddle:** Baidu's open source industry-level deep learning platform, which integrates deep learning core training and reasoning frameworks, basic model libraries, end-to-end development kits, and rich tool components.
- **MindSpore:** Huawei's open-source and self-developed AI framework supports deep learning, training, and reasoning in all scenarios on the device, edge, and cloud, and has features such as general automatic differentiation and distributed parallel training.

Some frameworks are listed above. In fact, there are many other deep learning frameworks, such as Caffe, keras and so on.

21.2 Artificial intelligence and Lubancat RK series board

Lubancat RK series boards use Rockchip rk356X processor, rk3568 and rk3566 NPU modules. The processing unit specially used for neural network accelerates the neural network algorithm in the field of artificial intelligence, up to 1TOPS, and supports integer 8 and integer 16 convolution operations, and supports the following deep learning frameworks: TensorFlow, TF-lite, Pytorch, Caffe, ONNX etc.

21.3 Reference link

<https://zh.wikipedia.org/wiki/%E6%9C%BA%E5%99%A8%E5%AD%A6%E4%B9%A0>

<https://zh.wikipedia.org/wiki/%E6%B7%B1%E5%BA%A6%E5%AD%A6%E4%B9%A0>

<https://www.paddlepaddle.org.cn/tutorials/projectdetail/3520300>

<https://tensorflow.google.cn/resources/learn-ml?hl=zh-cn>

Chapter22 NPU

NPU is a processing unit dedicated to neural networks. It is designed to accelerate neural network algorithms in artificial intelligence fields such as machine vision and natural language processing. As the range of applications for AI is expanding, it currently provides functions in various fields, including face tracking, gesture and body tracking, image classification, video surveillance, automatic speech recognition (ASR), and advanced driver assistance systems (ADAS).

LubanCat-RK series boards use Rockchip rk3566 and RK3568 processors. Both rk3568 and rk3566 have built-in NPU modules, up to 1TOPS, support integer 8, integer 16 convolution operations, and support deep learning frameworks: TensorFlow, TF-lite, Pytorch, Caffe, ONNX, etc.

The official RKNN SDK and RKNN-Toolkit2 tools are provided, and the SDK provides a programming interface for the chip platform with RKNPU. The Rockchip NPU platform uses the RKNN model, and the RKNN model can be converted using the RKNN-Toolkit2 software development kit, and deployed to the board using RKNN-Toolkit-Lite2.

This chapter will briefly introduce the use of RKNN-Toolkit2 on PC (Ubuntu system) for model conversion, model inference, performance evaluation, etc., and use RKNN Toolkit Lite2 to deploy on the board.

重要: The test environment of the tutorial in this chapter: Lubancat RK series board, the mirror system is Debian10 (Python 3.7.3, OpenCV 4.7.0.68), and the PC environment uses ubuntu20.04 (python3.8.10). When the tutorial was written, RKNN-Toolkit2 was version 1.4.0, the board npu driver was 0.7.2, rknrt was 1.4.0, and the adb tool was 1.0.40.

22.1 RKNN-Toolkit2

The Toolkit-lite2 tool is used on the PC platform and provides a python interface to simplify the deployment and operation of the model. Through this tool, users can conveniently complete the following functions: model conversion, quantization function, model reasoning, performance and memory evaluation, quantization accuracy analysis, and model encryption function.

The current version of RKNN-Toolkit2 is applicable to the system Ubuntu18.04 (x64) and above, more dependencies and usage information can be found here [“RKNN Toolkit2 Quick Start Guide”](#)

RKNN-Toolkit2 files can be downloaded directly from the official [github url](#) Pull or download from [Netdisc](#) (Extraction codehslu), In 1-Embedfire Open Source Book_Tutorial Documentation->Reference Code. (The obtained RKNN-Toolkit2 file contains RKNN Toolkit Lite2)

22.1.1 RKNN-Toolkit2 installation

Environment installation (PC ubuntu20.04):

```
#Install the python tool, ubuntu20.04 is installed with python3.8.10 by default
sudo apt update
sudo apt-get install python3-dev python3-pip python3.8-venv gcc

#Install related libraries and packages
sudo apt-get install libxslt1-dev zlib1g-dev libglib2.0 libsm6 \
libgl1-mesa-glx libprotobuf-dev gcc
```

Install RKNN-Toolkit2:

```
#Create a directory, because the installed package of ubuntu 20.04 used in the test may be different from the package version required to install and run RKNN-Toolkit2.
```

(下页继续)

(续上页)

```
#In order to avoid other problems, use python venv to isolate the_
↪environment.
mkdir project-Toolkit2 && cd project-Toolkit2
sudo python3 -m venv .toolkit2_env
# Activation into the environment
source .toolkit2_env/bin/activate

#Pull the source code, or copy RKNN-Toolkit2 to this directory
git clone https://github.com/rockchip-linux/rknn-toolkit2.git

#It may be very slow when using pip3 to install packages, set the source
pip3 config set global.index-url https://mirror.baidu.com/pypi/simple

#Install dependent libraries, according to rknn-toolkit2\doc\requirements_
↪cp38-1.4.0.txt
pip3 install numpy
pip3 install -r doc/requirements_cp38-1.4.0.txt

#Install rknn_toolkit2
pip3 install packages/rknn_toolkit2-1.4.0_22dcfef4-cp38-cp38-linux_x86_64.
↪whl
```

Check whether the installation is successful:

```
(.toolkit2_env) llh@-:~/project-Toolkit2$ python
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from rknn.api import RKNN
>>>
```

22.1.2 Model Transformation and Model Inference

In the RKNN-Toolkit2 file, there are routines of various functions under the example directory, enter the `../examples/onnx/yolov5` directory. This Demo shows the process of converting the onnx model to the RKNN model on the PC, then exporting, reasoning, deploying to the NPU platform to run, and retrieving the results.

Excuting an order:

```
# Switch to the examples/onnx/yolov5 directory
cd examples/onnx/yolov5
# Model Transformation and Model Inference
python test.py
```

After running the export successfully, the `yolov5s.rknn` file will be generated in the current directory, which will be used in the next Toolkit Lite2 section.

Running the `yolov5` routine above will also perform model inference, and will output the result picture (`out.jpg`) in the current directory.

22.1.3 Performance and Memory Evaluation

In addition to simulating reasoning tests, RKNN-Toolkit2 can also perform simple board-connected debugging. The usb interface of the lubancat board does not have functions such as usb debug, and cannot be directly debugged by `usb adb`, but it can be debugged by using the network `adb` connection.

The `adbd` and `rknn_server` services need to be started on the connected board debugging board. The `adb` server on the PC needs to be connected to `adbd`, so that the client can communicate with the device. `rknn_server` is a background proxy service running on the board, which is used to receive the protocol transmitted by the PC through USB, then execute the interface corresponding to the board runtime, and return the result to the PC.

```
# Copy the file to the board (you can use scp sftp nfs, etc., the file is_
↳in the supporting routine or see the reference link below, and get it_
↳from the runtime\RK356X\Linux\rknn_server directory of RKNPU2)

# Add executable permission
cd ../usr/bin
chmod +x rknn_server restart_rknn.sh start_rknn.sh

# Start rknn_server
restart_rknn.sh

# View the status of the service adbd, it is active
sudo systemctl status adbd.service
sudo /usr/bin/adbd &
```

The board and PC are connected under a local area network, and the supporting program of this tutorial is used for testing:

```
# Install adb on PC
sudo apt install adb
# open adb server
adb start-server
# Connect the board, according to the actual IP of the board, the default_
↳port is 5555
adb connect 192.168.103.115

# Check the connected device and confirm the device_id when initializing_
↳the runtime
adb devices
cd examples/onnx/yolov5

# Run the program, debug with the board
python test.py
```



Forum:<https://www.firebbs.cn/>

(续上页)

```
I NPUTTransfer: Starting NPU Transfer Client, Transfer version 2.1.0_
↪ (b5861e7@2020-11-23T11:50:36)
D RKNNAPI: =====
D RKNNAPI: RKNN VERSION:
D RKNNAPI:   API: 1.4.0 (bb6dac9 build: 2022-08-29 16:17:01) (null)
D RKNNAPI:   DRV: rknn_server: 1.3.0 (121b661 build: 2022-04-29 11:11:47)
D RKNNAPI:   DRV: rknnrt: 1.4.0 (a10f100eb@2022-09-09T09:07:14)
D RKNNAPI: =====
done
```

Performance

The performance result is just for debugging, ####
may worse than actual performance!

```
Total Weight Memory Size: 7312768
Total Internal Memory Size: 7782400
Predict Internal Memory RW Amount: 134547200
Predict Weight Memory RW Amount: 7312768
```

ID	OpType	DataType	Target	InputShape	OutputShape	DDR Cycles	NPU Cycles	Total Cycles
↪	Time (us)	MacUsage (%)	RW (KB)	FullName				
0	InputOperator	UINT8	CPU	\				
↪	(1, 3, 640, 640)		0		0	0		
↪ 19	\		1200.00	InputOperator:images				
1	Conv	UINT8	NPU	(1, 3, 640, 640), (32, 3, 6, 6), (32)				
↪	(1, 32, 320, 320)		1493409		691200	1493409		
↪ 11956	9.64		4409.25	Conv:Conv_0				
2	exSwish	INT8	NPU	(1, 32, 320, 320)				
↪	(1, 32, 320, 320)		2167674		0	2167674		
↪ 5408	\		6400.00	exSwish:Sigmoid_1_2swish				
3	Conv	INT8	NPU	(1, 32, 320, 320), (64, 32, 3, 3), (64)				
↪	(1, 64, 160, 160)		1632022		921600	1632022		
↪ 4153	36.99		4818.50	Conv:Conv_3				

(下页继续)

(续上页)

```

4      exSwish          INT8      NPU      (1, 64, 160, 160)
↪      (1, 64, 160, 160)      1083837      0      1083837
↪ 2849      \      3200.00      exSwish:Sigmoid_4_2swish
5      Conv            INT8      NPU      (1, 64, 160, 160), (32, 64, 1, 1), (32)
↪      (1, 32, 160, 160)      813640      102400      813640
↪ 1455      11.73      2402.25      Conv:Conv_6
6      exSwish          INT8      NPU      (1, 32, 160, 160)
↪      (1, 32, 160, 160)      541919      0      541919
↪ 1456      \      1600.00      exSwish:Sigmoid_7_2swish
.....(omitted)
142    Conv            INT8      NPU      (1, 128, 80, 80), (255, 128, 1, 1), (255)
↪      (1, 255, 80, 80)      824352      408000      824352
↪ 1040      65.38      2433.88      Conv:Conv_198
143    OutputOperator  INT8      CPU      (1, 255, 80, 80), (1, 80, 80, 256)
↪      \      0      0      0
↪ 1011      \      3200.00      OutputOperator:output
144    OutputOperator  INT8      CPU      (1, 255, 40, 40), (1, 40, 40, 256)
↪      \      0      0      0
↪ 216      \      800.00      OutputOperator:327
145    OutputOperator  INT8      CPU      (1, 255, 20, 20), (1, 20, 20, 256)
↪      \      0      0      0
↪ 178      \      220.00      OutputOperator:328
Total Operator Elapsed Time(us): 117367
=====

```

In the above test, we can see that quantization is enabled, and eval_perf is called to evaluate the performance of the model, and obtain the overall time consumption of the test model and the time consumption of each layer. The above is a simple connection debugging.

For more functional test routines of rknn-toolkit2, refer to <https://github.com/rockchip-linux/rknn-toolkit2/tree/master/examples/functions>.

22.2 RKNN Toolkit Lite2

RKNN Toolkit Lite2 provides a Python programming interface for the Rockchip NPU platform to deploy the RKNN model on the board.

22.2.1 Install RKNN Toolkit Lite2 on the board

Toolkit-lite2 is currently applicable to Debian10/11 (aarch64) system, more dependencies and usage information can be found at [“RKNN Toolkit Lite2 User Guide”](#)

To obtain RKNN Toolkit Lite2, you can obtain it directly from the official github, or in the supporting routine, transfer the file to the board, or directly git to the board. The obtained Toolkit Lite2 directory structure is as follows:

```
.
├── docs
│   ├── Rockchip_User_Guide_RKNN_Toolkit_Lite2_V1.4.0_CN.pdf
│   ├── Rockchip_User_Guide_RKNN_Toolkit_Lite2_V1.4.0_EN.pdf
│   └── change_log.txt
├── examples
│   ├── inference_with_lite
│   │   ├── resnet18_for_rk356x.rknn
│   │   ├── resnet18_for_rk3588.rknn
│   │   ├── space_shuttle_224.jpg
│   │   └── test.py
└── packages
    ├── rknn_toolkit_lite2-1.4.0-cp37-cp37m-linux_aarch64.whl
    ├── rknn_toolkit_lite2-1.4.0-cp39-cp39-linux_aarch64.whl
    └── rknn_toolkit_lite2_1.4.0_packages.md5sum

4 directories, 10 files
```

Environment installation (take LubanCat 2, Debian10 as an example):

```
sudo apt update

#Install other python tools
sudo apt-get install python3-dev python3-pip gcc
```

(下页继续)

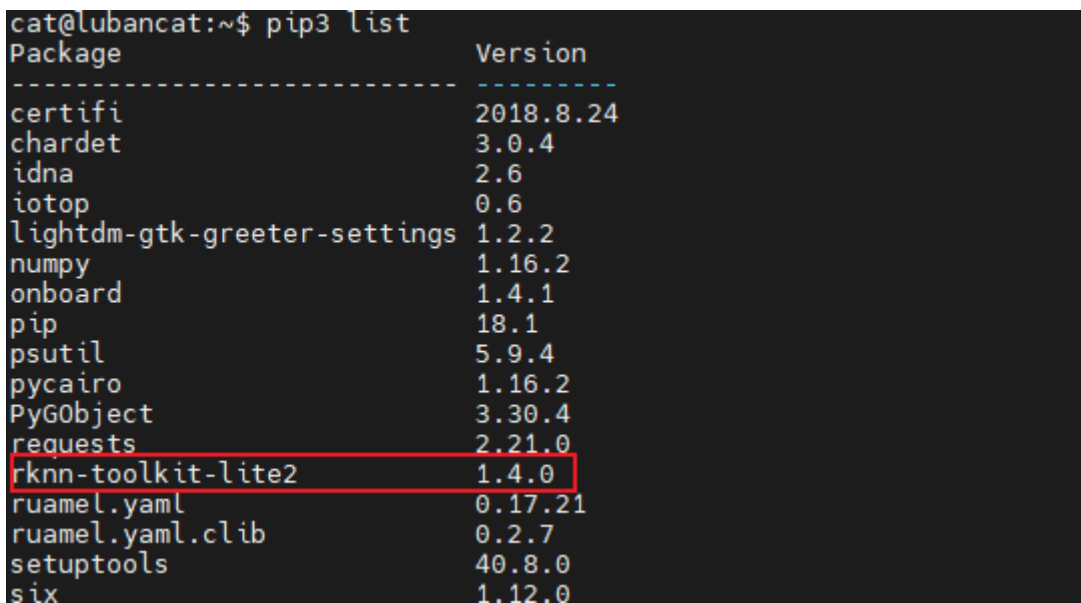
(续上页)

```
#Install related dependencies and packages
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
sudo apt -y install python3-setuptools
pip3 install wheel
```

Toolkit Lite2 tool installation:

```
# Go to the rknn_toolkit_lite2/packages directory, select the whl file of
↳Debian10 ARM64 with python3.7.3 to install:
pip3 install rknn_toolkit_lite2-1.4.0-cp37-cp37m-linux_aarch64.whl
```

Successful installation:



```
cat@lubancat:~$ pip3 list
Package                               Version
-----
certifi                               2018.8.24
chardet                               3.0.4
idna                                   2.6
iotop                                  0.6
lightdm-gtk-greeter-settings          1.2.2
numpy                                  1.16.2
onboard                               1.4.1
pip                                    18.1
psutil                                5.9.4
pycairo                               1.16.2
PyGObject                             3.30.4
requests                              2.21.0
rknn-toolkit-lite2                    1.4.0
ruamel.yaml                           0.17.21
ruamel.yaml.clib                      0.2.7
setuptools                            40.8.0
six                                    1.12.0
```

22.2.2 Board side deployment reasoning

重要： librknnrt.so is a board-side runtime library, which is required for operation. There is librknnrt.so library in the default image /usr/lib directory, but it needs to be updated. You can pull it from the official github, or check the supporting routines, and copy librknnrt.so to the system /usr/lib/ directory.

Run the Demo of RKNN Toolkit Lite2, enter the obtained Toolkit Lite2 directory, and execute the command in the ../examples/inference_with_lite directory:

```
cat@lubancat:~/rknn-toolkit2/rknn_toolkit_lite2/examples/inference_with_lite$ ls
resnet18_for_rk356x.rknn resnet18_for_rk3588.rknn space_shuttle_224.jpg test.py
cat@lubancat:~/rknn-toolkit2/rknn_toolkit_lite2/examples/inference_with_lite$ python test.py
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [10:25:19.339] RKNN Runtime Information: librknnrt version: 1.4.0 (a10f100eb@2022-09-09T09
I RKNN: [10:25:19.339] RKNN Driver Information: version: 0.7.2
I RKNN: [10:25:19.340] RKNN Model Information: version: 1, toolkit version: 1.4.0-c15f5e0b(compile
mework name: PyTorch, framework layout: NCHW
done
--> Running model
resnet18
-----TOP 5-----
[812]: 0.9996696710586548
[404]: 0.0002492684288881719
[657]: 1.632158637221437e-05
[833]: 1.0159346857108176e-05
[466 895]: 9.02384545042878e-06
done
cat@lubancat:~/rknn-toolkit2/rknn_toolkit_lite2/examples/inference_with_lite$
```

Modify the examples/onnx/yolov5 routine in the Toolkit Lite2 tool, simply modify test.py, and use RKNN Toolkit Lite2 to deploy. Import the yolov5s.rknn converted from the previous RKNN-Toolkit2 (refer to the supporting routine for the modified source file):

```
cat@lubancat:~/yolov5$ python test.py
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [10:29:36.521] RKNN Runtime Information: librknnrt version: 1.4.0
(a10f100eb@2022-09-09T09:07:14)
```

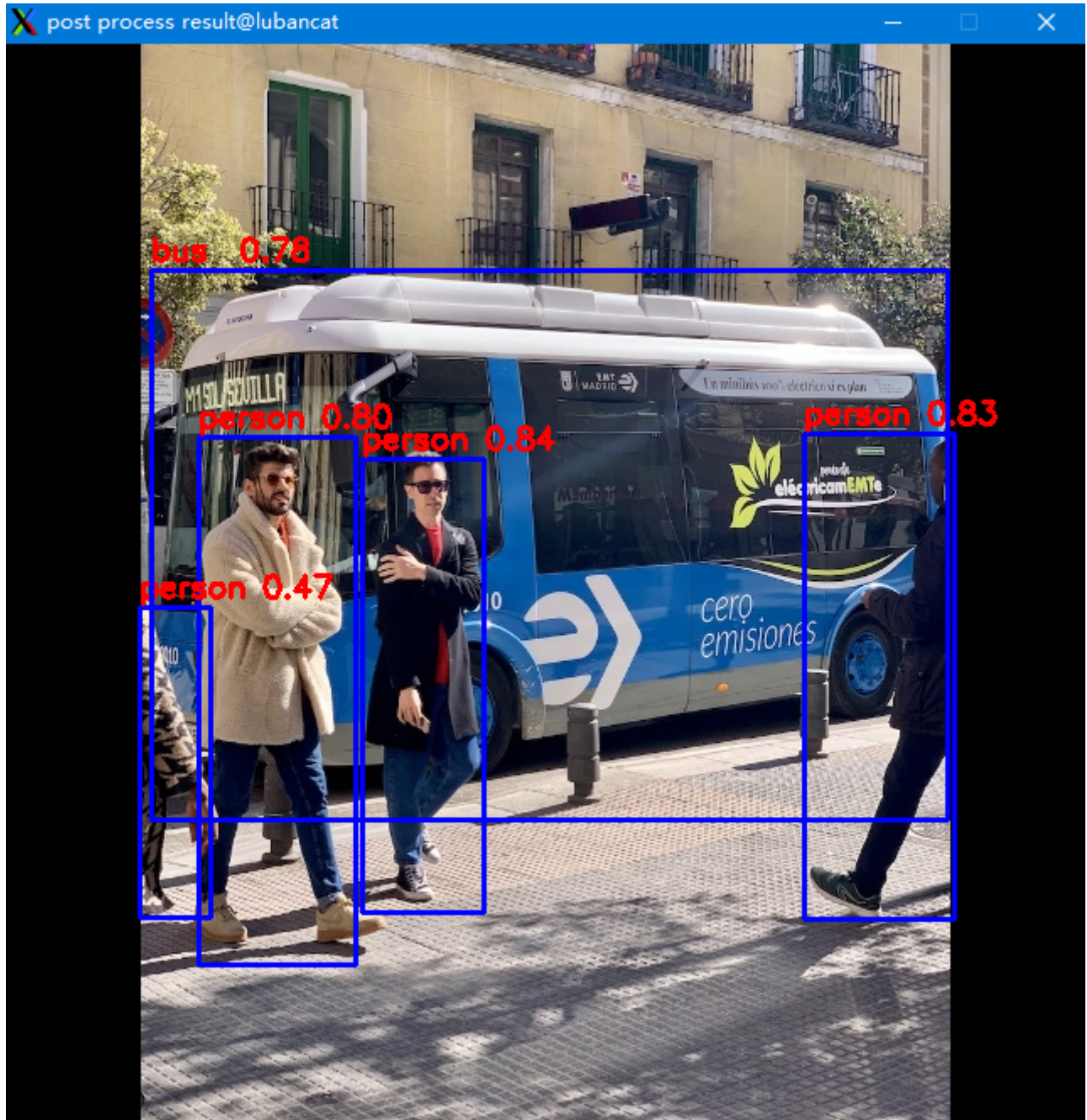
(下页继续)

(续上页)

```
I RKNN: [10:29:36.521] RKNN Driver Information: version: 0.7.2
I RKNN: [10:29:36.524] RKNN Model Information: version: 1, toolkit version:
↳1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳target: RKNPU lite, target platform: rk3568, framework name: ONNX,
↳framework layout: NCHW
done
--> Running model
done
class: person, score: 0.8352155685424805
box coordinate left,top,right,down: [211.9896697998047, 246.17460775375366,
↳283.70787048339844, 515.0830216407776]
class: person, score: 0.8330121040344238
box coordinate left,top,right,down: [473.26745200157166, 231.93780636787415,
↳ 562.1268351078033, 519.7597033977509]
class: person, score: 0.7970154881477356
box coordinate left,top,right,down: [114.68497347831726, 233.11390662193298,
↳ 207.21904873847961, 546.7821505069733]
class: person, score: 0.4651743769645691
box coordinate left,top,right,down: [79.09242534637451, 334.77847719192505,
↳121.60038471221924, 518.6368670463562]
class: bus , score: 0.7837686538696289
box coordinate left,top,right,down: [86.41703361272812, 134.41848754882812,
↳558.1083570122719, 460.4184875488281]

(post process result:1896): dbind-WARNING **: 10:31:30.000: Error
↳retrieving accessibility bus address: org.freedesktop.DBus.Error.
↳ServiceUnknown: The name org.a11y.Bus was not provided by any .service
↳files
cat@lubancat:~/yolov5$ ls
bus.jpg  dataset.txt  onnx_yolov5_0.npy  onnx_yolov5_1.npy  onnx_yolov5_2.
↳npy  out.jpg  test.py  yolov5s.onnx  yolov5s.rknn
```

If it runs normally, it can be displayed (or view the output image out.jpg):



提示: The default supporting routine `../yolov5/test.py` does not use `cv2.imshow()` to display pictures.

22.3 Related frequency settings (rk356X)

The CPU of the Lubancat board is in the `interactive` state by default, and it will dynamically adjust the CPU frequency according to the CPU usage and target load. In order to obtain higher operating speed or performance evaluation, we need to manually fix the frequency, refer to this:

1、CPU frequency settings:

```
# View the current frequency of the CPU
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq

# View the current frequency adjustment policy of the CPU
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_governor

# Check available frequencies
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_frequencies

# Set the user with root authority to set the cpu frequency to the
↪frequency the user wants through the "scaling_setspeed" field of sysfs.
echo userspace > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor

# Set the frequency that needs to be fixed
echo 1800000 > /sys/devices/system/cpu/cpufreq/policy0/scaling_setspeed
```

2、DDR frequency setting:

```
# View the current frequency of DDR
cat /sys/class/devfreq/dmc/cur_freq

# View DDR current FM policy
cat /sys/class/devfreq/dmc/governor

# View the frequency that DDR can set
```

(下页继续)

(续上页)

```
cat /sys/class/devfreq/dmc/available_frequencies

# Set the user with root authority to set the cpu frequency to the
↪frequency the user wants through the "scaling_setspeed" field of sysfs
echo userspace > /sys/class/devfreq/dmc/governor

# The setting requires a fixed frequency, here is 1056000000
echo 1056000000 > /sys/devices/system/cpu/cpufreq/policy0/scaling_setspeed
```

3、NPU frequency setting:

```
# Check NPU to see available frequencies
cat /sys/class/devfreq/fde40000.npu/available_frequencies

echo userspace > /sys/class/devfreq/fde40000.npu/governor
# Set frequency
echo 900000000 > /sys/kernel/debug/clk/clk_scmi_npu/clk_rate

# View the current npu frequency
cat /sys/class/devfreq/fde40000.npu/cur_freq
```

22.4 Reference

RKNPU2 related documents and source addresses:<https://github.com/rockchip-linux/rknpu2>

RKNN-toolkit2 related documents and source addresses:<https://github.com/rockchip-linux/rknn-toolkit2>

Chapter23 Handwritten digit recognition – PaddlePaddle

23.1 PaddlePaddle Introduction



Based on Baidu's years of deep learning technology research and business applications, PaddlePaddle integrates deep learning core training and reasoning frameworks, basic model libraries, end-to-end development kits, and rich tool components. It is China's first self-developed, feature-rich, Open source and open industry-level deep learning platform.

PaddlePaddle has gathered 4.77 million developers, created 560,000 models based on PaddlePaddle, and served 180,000 enterprises and institutions. PaddlePaddle helps developers quickly realize AI ideas, innovate AI applications, and as a basic platform supports more and more industries to realize industrial intelligent

upgrading. As of December 2022, the latest report of the Institute of Communication Communications shows that PaddlePaddle has become the deep learning framework and enabling platform with the largest application scale in China's deep learning market. It has gathered 5.35 million developers and served 200,000 enterprises and institutions. It is based on PaddlePaddle 670,000 models were obtained.

We will complete a simple handwritten digit recognition task based on PaddlePaddle and deploy it to the Lubancat board. Through this chapter, you can simply understand the PaddlePaddle and the deep learning model.

提示: Test environment: lubancat board uses Debian10, PC is ubuntu20.04. PaddlePaddle is the CPU version, rknn-Toolkit2 version 1.4.0.

23.2 Install PaddlePaddle

```
# Use the pip3 tool on the PC side to install the Flying Paddle CPU version
pip3 install paddlepaddle -i https://mirror.baidu.com/pypi/simple

# Verify the installation, enter the python interpreter, and enter:
python
import paddle
paddle.utils.run_check()

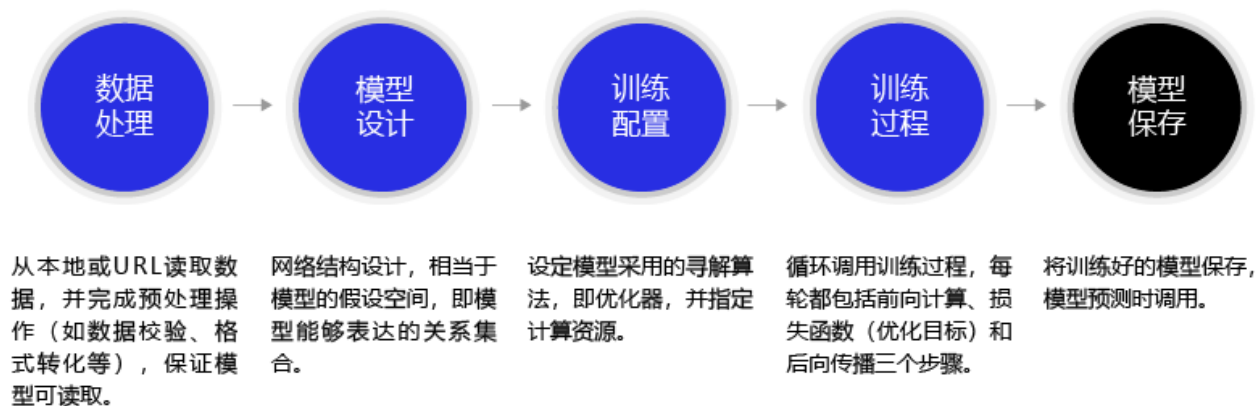
# Verify installation, print version information
print(paddle.__version__)

# Finally, it shows that PaddlePaddle is installed successfully! Let's
↪ start deep learning with PaddlePaddle now.
# Indicates that the installation was successful
```

For detailed installation of paddlepaddle, please refer to [Installation Tutorial](#).

23.3 Handwritten digit recognition task

The task of recognizing handwritten digits is one of the most widely used examples in the field of machine learning, and it is a relatively simple model that is very suitable for beginners. For the overall process of the task, refer to the picture below (picture from Paddle [tutorial](#)):



23.3.1 Prepare training and test sets

Handwritten digit recognition uses grayscale images to represent the digits written by different people. The size of each picture is 28*28. We use the MNIST data set. You can go to the official website <http://yann.lecun.com/exdb/mnist/> to download it, or we load MNIST directly using Paddle.

Here we load the MNIST training set (mode='train') and test set (mode='test'). The training set is used to train the model, and the test set is used to evaluate the effect of the model.

```
# Download the dataset and initialize the DataSet
train_dataset = paddle.vision.datasets.MNIST(mode='train',
↪transform=transform)

test_dataset = paddle.vision.datasets.MNIST(mode='test',
↪transform=transform)
```

23.3.2 Model networking

In the process of digital recognition, LeNet, the built-in model of Flying Paddle, is directly used. Using the high-level API, the network construction and initialization of LeNet can be completed with one line of code, and the network can also be customized. For details, please refer to https://www.paddlepaddle.org.cn/documentation/docs/zh/guides/beginner/model_cn.html

```
# Model network and initialize the network,
lenet = paddle.vision.models.LeNet(num_classes=10)
model = paddle.Model(lenet)
#paddle.summary(lenet, (1, 1, 28, 28))
```

The LeNet model contains 2 Conv2D convolutional layers, 2 ReLU activation layers, 2 MaxPool2D pooling layers, and 3 Linear fully connected layers. Cancel the comment above the line.“paddle.summary(lenet,(1, 1, 28, 28)) “ Print the structure and parameter statistics of the network:

```
-----
Layer (type)      Input Shape      Output Shape     Param #
=====
Conv2D-1          [[1, 1, 28, 28]] [1, 6, 28, 28]   60
ReLU-1            [[1, 6, 28, 28]] [1, 6, 28, 28]   0
MaxPool2D-1       [[1, 6, 28, 28]] [1, 6, 14, 14]   0
Conv2D-2          [[1, 6, 14, 14]] [1, 16, 10, 10]  2,416
ReLU-2            [[1, 16, 10, 10]] [1, 16, 10, 10]  0
MaxPool2D-2       [[1, 16, 10, 10]] [1, 16, 5, 5]    0
Linear-1          [[1, 400]]        [1, 120]          48,120
Linear-2          [[1, 120]]         [1, 84]           10,164
Linear-3          [[1, 84]]          [1, 10]           850
=====
Total params: 61,610
Trainable params: 61,610
Non-trainable params: 0
-----
```

(下页继续)

(续上页)

```
Input size (MB): 0.00
Forward/backward pass size (MB): 0.11
Params size (MB): 0.24
Estimated Total Size (MB): 0.35
-----
```

23.3.3 Model Training and Evaluation

Model training includes multi-round iterations (EPOCH), which traversed through a training data set per round, and each time is obtained from a small batch (mini-batch) sample, and the prediction value is obtained before the model execution, and calculate the loss functional values (LOSS) between Predict_label and true values (True_label). Perform the gradient reverse propagation and update the parameters of the model based on the set optimization algorithm. Observe the decrease in the LOSS value of each round of iteration, and the model training effect can be judged.

```
# Set the optimizer and its learning rate, and pass the parameters of the
↪network into the optimizer, set the loss function and accuracy
↪calculation method
# OPTIMIZER , find the best solution method; LOSS loss function; Metric
↪evaluation index
# Here the configuration uses an ADAM optimizer, and the cross -entropy
↪loss function CrossEntropyLoss is used for classification task assessment.
model.prepare(paddle.optimizer.Adam(parameters=model.parameters()),
              paddle.nn.CrossEntropyLoss(),
              paddle.metric.Accuracy())

# Model training, epoch training round, batch_size batch size, Verbose = 1
↪log in the log in training process.
model.fit(train_dataset, epochs=5, batch_size=64, verbose=1)
```

(下页继续)

(续上页)

```
# Model assessment, send the test dataset into the training model for_
↪evaluation, get the predicted value.
# Calculate the loss functional value (LOSS) between the calculation of the_
↪predicted value and the real value.
# And calculate the evaluation index value (Metric) to evaluate the model_
↪effect.
model.evaluate(test_dataset, batch_size=64, verbose=1)
```

23.3.4 Model saving and exporting onnx model

The well-trained model can be saved into the file. When the training is required for training or inference deployment, load (load) to the memory.

```
1 # Save the model
2 model.save('./output/mnist')
3 # Load the model
4 model.load('output/mnist')
5
6 # Take out a picture from the test concentration
7 img, label = test_dataset[0]
8 # Transform the picture shape from 1*28*28 to 1*1*28*28, add a BATCH_
↪dimension to match the model input format requirements
9 img_batch = np.expand_dims(img.astype('float32'), axis=0)
10
11 # Perform the reasoning and print the results. Here, the Predict_batch_
↪returns a list, which is taken out of the data to get the predictive_
↪results
12 out = model.predict_batch(img_batch)[0]
13 pred_label = out.argmax()
14 print('true label: {}, pred label: {}'.format(label[0], pred_label))
```

After the Paddle model training, it only needs to call the `paddle.onnx.export` interface to convert to the onnx

protocol, and the onnx model will be generated under the specified path. You can also save the Paddle model to the deployment model (static graph model), and then use the Paddle2onnx command line to convert. For the principle of conversion, please refer to <https://www.paddlepaddle.org.cn/documentation/docs/zh/guides/principle_cn.html> _.

```
# The path to be preserved, the current onnx directory
save_path = 'onnx/lenet'
# Specify the input shape and data type for the model
x_spec = paddle.static.InputSpec([1, 1, 28, 28], 'float32', 'x')
# Generate onnx model
paddle.onnx.export(lenet, save_path, input_spec=[x_spec], opset_version=11)
```

23.3.5 Complete program

列 表 1: Supporting code
'paddlepaddle/digital_recognition.py'

```
1 import paddle
2 import numpy as np
3 from paddle.vision.transforms import Normalize
4
5 transform = Normalize(mean=[127.5], std=[127.5], data_format='CHW')
6 # Download the dataset and initialize
7 train_dataset = paddle.vision.datasets.MNIST(mode='train',
8     ↪transform=transform)
9 test_dataset = paddle.vision.datasets.MNIST(mode='test',
10     ↪transform=transform)
11
12 # Model group network and initialize the network
13 lenet = paddle.vision.models.LeNet(num_classes=10)
14 model = paddle.Model(lenet)
```

(下页继续)

(续上页)

```
14 # Model training configuration preparation, preparation of loss functions,
    ↳ optimizers and evaluation indicators
15 model.prepare(paddle.optimizer.Adam(parameters=model.parameters()),
16               paddle.nn.CrossEntropyLoss(),
17               paddle.metric.Accuracy())
18
19 # Model training
20 model.fit(train_dataset, epochs=5, batch_size=64, verbose=1)
21 # Model assessment
22 model.evaluate(test_dataset, batch_size=64, verbose=1)
23
24 # Save the model
25 model.save('./output/mnist')
26
27 # Load the model
28 # model.load('output/mnist')
29
30 # Take out a picture from the test concentration
31 img, label = test_dataset[0]
32 # Transform the picture shape from 1*28*28 to 1*1*28*28, add a BATCH_
    ↳ dimension to match the model input format requirements
33 img_batch = np.expand_dims(img.astype('float32'), axis=0)
34
35 # Perform the reasoning and print the results. Here, the Predict_batch_
    ↳ returns a list, which is taken out of the data to get the predictive_
    ↳ results
36 out = model.predict_batch(img_batch)[0]
37 pred_label = out.argmax()
38 print('true label: {}, pred label: {}'.format(label[0], pred_label))
39
40 # The path to be preserved
41 save_path = 'onnx/lenet'
```

(下页继续)

(续上页)

```
42 # Specify the input shape and data type for the model
43 x_spec = paddle.static.InputSpec([1, 1, 28, 28], 'float32', 'x')
44 # Generate onnx model
45 paddle.onnx.export(lenet, save_path, input_spec=[x_spec], opset_version=11)
```

Simple test shows:

The loss value printed in the log is the current step, and the metric is ↪ the average value of previous steps.

Epoch 1/5

step 938/938 [=====] - loss: 0.0117 - acc: 0.9368 -
↪ 19ms/step

Epoch 2/5

step 938/938 [=====] - loss: 0.0072 - acc: 0.9778 -
↪ 18ms/step

Epoch 3/5

step 938/938 [=====] - loss: 0.0521 - acc: 0.9814 -
↪ 19ms/step

Epoch 4/5

step 938/938 [=====] - loss: 0.0047 - acc: 0.9848 -
↪ 19ms/step

Epoch 5/5

step 938/938 [=====] - loss: 0.0208 - acc: 0.9857 -
↪ 19ms/step

Eval begin...

step 157/157 [=====] - loss: 0.0011 - acc: 0.9843 -
↪ 8ms/step

Eval begin...

step 157/157 [=====] - loss: 2.8939e-04 - acc: 0.
↪ 9825 - 8ms/step

Eval samples: 10000

true label: 7, pred label: 7

(下页继续)

(续上页)

```
I0209 16:10:02.397411 8350 interpretercore.cc:279] New Executor is Running.
2023-02-09 16:10:02 [INFO] Static PaddlePaddle model saved in onnx/
↪paddle_model_static_onnx_temp_dir.
[Paddle2ONNX] Start to parse PaddlePaddle model...
[Paddle2ONNX] Model file path: onnx/paddle_model_static_onnx_temp_dir/model.
↪pdmodel
[Paddle2ONNX] Paramters file path: onnx/paddle_model_static_onnx_temp_dir/
↪model.pdiparams
[Paddle2ONNX] Start to parsing Paddle model...
[Paddle2ONNX] Use opset_version = 11 for ONNX export.
[Paddle2ONNX] PaddlePaddle model is exported as ONNX format now.
2023-02-09 16:10:02 [INFO] ONNX model saved in onnx/lenet.onnx.
```

23.3.6 Simulation reasoning and export RKNN model

Use the RKNN-Toolkit2 tool to convert and simulate the model:

列表 2: supporting code ‘paddlepaddle/rknn_transfer.py’

```
1 INPUT_SIZE = 28
2 IMG_PATH = 'test.jpg'
3
4 # Create RKNN execution objects, print information
5 #rknn = RKNN(verbose=True)
6 rknn = RKNN()
7
8 # Configuration model input, for the pre -processing of NPU on data input
9 # Mean_values: The average value of input
10 # STD_VALUES: The normalized value of the input
11 # target_platform: target platform
12 # More settings refer to the RKNN Toolkit2 user guidance manual
```

(下页继续)

(续上页)

```
13 print('--> Config model')
14 rknn.config(mean_values=[127.5], std_values=[127.5], target_platform='rk3568
   ↪')
15
16 # Load the ONNX model
17 print('--> Loading model')
18 rknn.load_onnx(model='./lenet.onnx')
19 print('done')
20
21 print('--> Building model')
22 ret = rknn.build(do_quantization=False)
23 print('done')
24
25 # Initialize the running environment of the RKNN, run the test
26 print('--> Init runtime environment')
27 ret = rknn.init_runtime()
28 print('done')
29
30 # Enter image processing
31 img = cv2.imread(IMG_PATH)
32 img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
33 img = cv2.resize(img, (28, 28))
34 img = np.expand_dims(img, 0)
35 img = np.expand_dims(img, 0)
36
37 # Simulation reasoning
38 print('--> Running model')
39 outputs = rknn.inference(inputs=[img], data_format='nchw')
40 print("result: ", outputs)
41 print("The forecast number is:", np.argmax(outputs))
42
43 # Export RKNN model file
```

(下页继续)



```
44 print('--> Export rknn model')
45 rknn.export_rknn('./lenet.rknn')
46 print('done')
47
48 # Release RKNN
49 rknn.release()
```

23.4 Board deployment reasoning

23.4.1 Install RKNN Toolkit Lite2 and related libraries

The installation of Toolkit Lite2 refers to the previous “npu usage chapter”

```
# Library -related libraries
sudo apt update
sudo apt -y install python3-pil python3-numpy python3-pip git wget
sudo apt install python3-opencv
```

23.4.2 Reasoning test

Use the RKNN model transformed earlier, and then write a test program (you can also obtain files directly from the supporting example):

列表 3: support code ‘paddlepaddle/rknn_test.py’

```
1  IMG_PATH = '9.jpg'
2  RKNN_MODEL = 'lenet.rknn'
3
4  # Create RKNN
5  rknn_lite = RKNNLite()
6
7  # Import model
8  print('--> Load RKNN model')
9  ret = rknn_lite.load_rknn(RKNN_MODEL)
10 if ret != 0:
11     print('Load RKNN model failed')
12     exit(ret)
13 print('done')
```

(下页继续)

(续上页)

```
14
15 # Init runtime environment
16 print('--> Init runtime environment')
17 ret = rknn_lite.init_runtime()
18 if ret != 0:
19     print('Init runtime environment failed!')
20     exit(ret)
21 print('done')
22
23 # load image
24 img = cv2.imread(IMG_PATH)
25 img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
26 img = cv2.resize(img, (28,28))
27
28 # runing model
29 print('--> Running model')
30 outputs = rknn_lite.inference(inputs=[img])
31 print("result: ", outputs)
32 print("The figures predicted this time are:", np.argmax(outputs))
33
34 rknn_lite.release()
```

Running program:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [11:37:03.185] RKNN Runtime Information: librknrt version: 1.4.0
↳(a10f100eb@2022-09-09T09:07:14)
I RKNN: [11:37:03.185] RKNN Driver Information: version: 0.7.2
I RKNN: [11:37:03.185] RKNN Model Information: version: 1, toolkit version:
↳1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳target: RKNPU lite, target platform: rk3568, framework name: ONNX, (下页继续)
↳framework layout: NCHW
```

(续上页)

```
done
--> Running model
result:  [array([[ -3.5429688 , -4.1445312 , -1.7900391 ,  0.57128906,  2.
↪ 7421875 ,
          -2.015625  , -7.9882812 , -0.52734375,  3.4882812 ,  7.5234375 ]],
          dtype=float32)]
The figures predicted this time are: 9
```

23.5 Summarize

The above is a simple example. It is classified by writing digital data and MNIST by LENET opponents. Call the packaging and test interface to quickly complete the formation and prediction of the model for simple learning reference. If you want to build a model training step by step, you can refer to *paddlepaddle zero basic practice deep learning* <<https://www.paddlepaddle.org.cn/tutorials/projectdetail/4676538>> _.

23.6 Reference link

<https://baike.baidu.com/item/%E9%A3%9E%E6%A1%A8/23472642?fromtitle=PaddlePaddle&fromid=20110894&fr=aladdin>

<https://www.paddlepaddle.org.cn/documentation/docs/zh/install/pip/linux-pip.html>

https://www.paddlepaddle.org.cn/documentation/docs/zh/guides/beginner/quick_start_cn.html

https://www.paddlepaddle.org.cn/documentation/docs/zh/api/index_cn.html

Chapter24 PaddlePaddle FastDeploy

24.1 FastDeploy

For important AI models in industrial landing scenarios, FastDeploy standardizes the model API and provides demo examples that can be downloaded and run. FastDeploy also supports online (service deployment) and offline deployment forms to meet the deployment needs of different developers.

FastDeploy aims to provide AI developers with the optimal solution for model deployment. It has three characteristics: full-scenario, easy-to-use, and extremely efficient.

Full scene: Support GPU, CPU, Jetson, ARM CPU, Rockchip NPU, Amlogic NPU, NXP NPU and other types of hardware. It supports local deployment, service-based deployment, web-side deployment, mobile-side deployment, etc., and supports the three major fields of CV, NLP, and Speech. Supports 16 mainstream algorithm scenarios such as image classification, image segmentation, semantic segmentation, object detection, character recognition (OCR), face detection and recognition, portrait deduction, pose estimation, text classification, information extraction, pedestrian tracking, and speech synthesis.

Ease of use and flexibility: 3 lines of code complete the deployment of the AI model, 1 line of code quickly switches the back-end reasoning engine and deployment hardware, and the unified API enables zero-cost migration of different deployment scenarios.

Extreme efficiency: Compared with the traditional deep learning inference engine, which only focuses on the inference time of the model, FastDeploy focuses on the end-to-end deployment performance of the model task. Through high-performance pre- and post-processing, integration of high-performance inference engines, one-click automatic compression and other technologies, the ultimate performance optimization of AI model inference deployment is realized.

Next, we will simply build the environment and use FastDeploy to deploy the lightweight detection network PicoDet on rk356X.

提示: Test environment: lubancat board uses Debian10, PC side is WSL2 (ubuntu20.04). FastDeploy version 1.0.0, rknn-Toolkit2 version 1.4.0.

24.2 Environment configuration

24.2.1 PC-side model conversion and inference environment construction

It is necessary to install rknn-Toolkit2 and FastDeploy tools for model conversion, etc.

1、Install rknn-Toolkit2

Refer to the previous [“NPU Use Chapter”](#)

2、Install FastDeploy and other tools to install

```
# Install directly with the precompiled library, or compile and install.
↪with FastDeploy
# For detailed reference:
pip3 install fastdeploy-python -f https://www.paddlepaddle.org.cn/whl/
↪fastdeploy.html

# Install paddle2onnx

pip3 install paddle2onnx

# Install paddlepaddle
python -m pip3 install paddlepaddle==0.0.0 -f https://www.paddlepaddle.org.
↪cn/whl/linux/cpu-mkl/develop.html
```

Please refer to [here](#) for paddlepaddle installation details.

24.2.2 Build FastDeploy RKNPU2 inference environment on board

1、Install RKNN driver environment

```
# Use the latest RK driver
git clone https://github.com/rockchip-linux/rknpu2
sudo cp ./rknpu2/runtime/RK356X/Linux/librknn_api/aarch64/* /usr/lib
sudo cp ./rknpu2/runtime/RK356X/Linux/rknn_server/aarch64/usr/bin/* /usr/
↪bin/
```

You can also refer to the previous [《NPU Usage》](#) .

2、Install FastDeploy

Reasoning needs to be deployed on the board side, so compile the Python SDK on the board side, then package it, and install FastDeploy.

```
# Get the source code
git clone https://github.com/PaddlePaddle/FastDeploy.git

# Environment variable
export ENABLE_ORT_BACKEND=ON
export ENABLE_RKNPU2_BACKEND=ON # Use rknpu2 as backend engine
export ENABLE_VISION=ON
export RKNN2_TARGET_SOC=rk356X # Rk356X platform

cd FastDeploy/python

# Compile, Package
sudo python3 setup.py build
sudo python3 setup.py bdist_wheel

# Install
cd dist
```

(下页继续)

(续上页)

```
pip3 install fastdeploy_python-0.0.0-cp37-cp37m-linux_aarch64.whl

# If you don't want to compile and package, use fastdeploy_python-0.0.0-
↪cp37-cp37m-linux_aarch64.whl in the supporting routine to install_
↪directly.
```

提示: rk356X freezes when compiling, probably because of insufficient memory. It is recommended to add a swap partition, at least 4G swap space, the compilation on the board is slow and you need to wait.

24.3 Example deployment inference

24.3.1 Lightweight detection network PicoDet

1、Model conversion

Rknn-Toolkit2 does not support the direct export of the Paddle model as an RKNN model for the time being. You need to use Paddle2ONNX to convert it into an onnx model, and then export the rknn model.

```
# Get the Paddle static graph model and decompress it (you can get it from_
↪the FastDeploy source file documentation, or get it from the supporting_
↪routine of this tutorial).
git clone https://github.com/PaddlePaddle/FastDeploy.git

# Or from the companion routine
git
tar -xvf picodet_s_416_coco_lcnet.zip

# Static graph model conversion onnx model
cd tools/rknpu2/picodet_s_416_coco_lcnet/
```

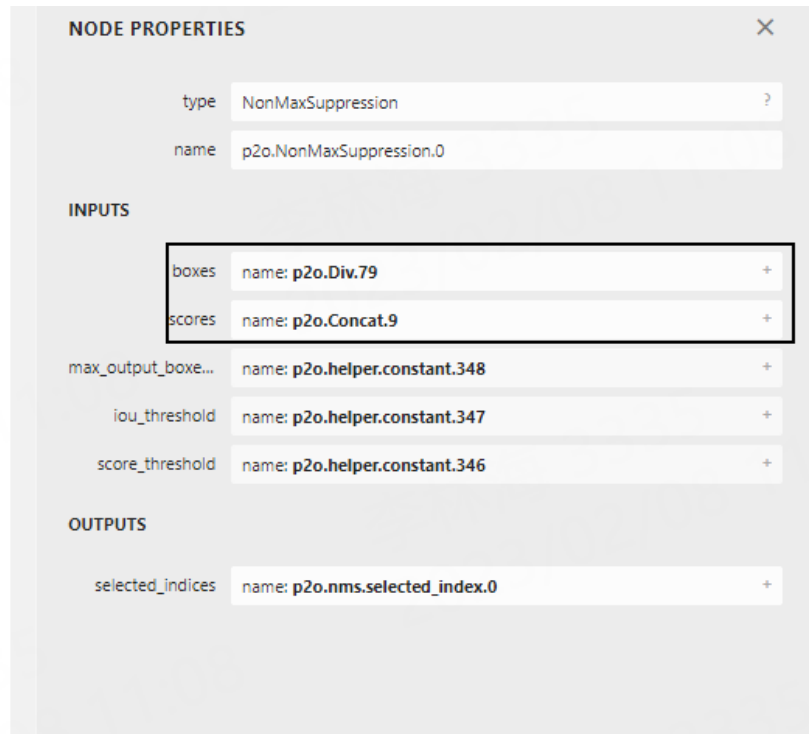
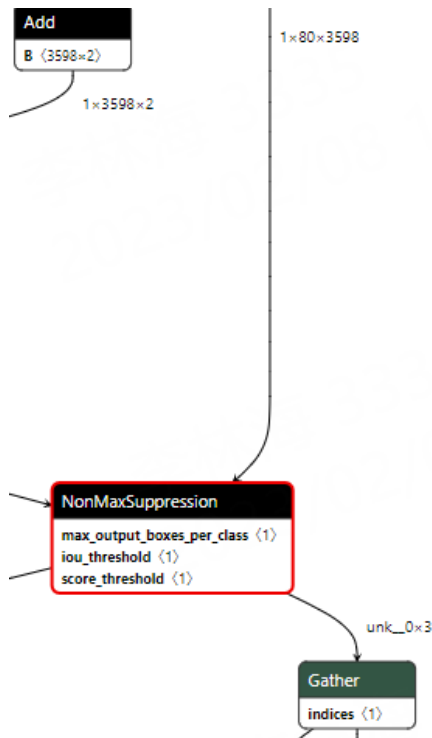
(下页继续)

(续上页)

```
paddle2onnx --model_dir picodet_s_416_coco_lcnet \  
    --model_filename model.pdmodel \  
    --params_filename model.pdiparams \  
    --save_file picodet_s_416_coco_lcnet/picodet_s_416_coco_lcnet.onnx \  
    --enable_dev_version True  
  
# Fixed shape  
python -m paddle2onnx.optimize --input_model picodet_s_416_coco_lcnet/  
    ↪picodet_s_416_coco_lcnet.onnx \  
    --output_model picodet_s_416_coco_lcnet/picodet_  
    ↪s_416_coco_lcnet.onnx \  
    --input_shape_dict '{"image':[1,3,416,416]}'
```

2、Export RKNN model

Depending on the version of Paddle2ONNX (1.0.5 was used for testing), the name of the output node of the conversion model is also different. You need to use [Netron](#) to visualize the model, find the NonMax-Suppression node, confirm the output node name, and then modify the outputs_nodes parameter in the picodet_s_416_coco_lcnet.yaml configuration file:



列表 1: picodet_s_416_coco_lcnet.yaml configuration file

```

1 mean:
2 std:
3 model_path: ./picodet_s_416_coco_lcnet1/picodet_s_416_coco_lcnet.onnx
4 target_platform: RK3568
5 outputs_nodes:
6 - 'p2o.Div.79'
7 - 'p2o.Concat.9'
8 do_quantization: False
9 dataset:
10 output_folder: "./picodet_s_416_coco_lcnet1"

```

```

# Export model
cd tools/rknpu2/
python export.py --config_path config/picodet_s_416_coco_lcnet.yaml --
target_platform rk3568

```

(下页继续)

(续上页)

3、Board side deployment reasoning

```
# Inference deployment on the board side, you can use the rknn model_
↳ exported by yourself earlier, or directly use the supporting routines.

cd examples/vision/detection/paddledetection/rknpu2/python
sudo python3 infer.py --model_file picodet_s_416_coco_lcnet_rk3568_
↳ unquantized.rknn --config_file infer_cfg.yml --image 00000014439.jpg

# Among them --model_file specifies the model file, --config_file specifies_
↳ the configuration file, --image specifies the image that needs to be_
↳ inferred.
```

Running inference shows:

```
[INFO] fastdeploy/vision/common/processors/transform.
↳ cc(159)::FuseNormalizeColorConvert BGR2RGB and Normalize are fused to_
↳ Normalize with swap_rb=1
[INFO] fastdeploy/runtime/backends/rknpu2/rknpu2_backend.
↳ cc(57)::GetSDKAndDeviceVersion rknn_api/rknnrt version: 1.4.0_
↳ (a10f100eb@2022-09-09T09:07:14), driver version: 0.7.2
E RKNN: [15:17:31.563] rknn_set_core_mask: No implementation found for_
↳ current platform!
index=0, name=image, n_dims=4, dims=[1, 416, 416, 3], n_elems=519168,_
↳ size=1038336, fmt=NHWC, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000,_
↳ pass_through=0
index=1, name=scale_factor, n_dims=2, dims=[1, 2, 0, 0], n_elems=2, size=4,_
↳ fmt=UNDEFINED, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000, pass_
↳ through=0
index=0, name=p2o.Div.79, n_dims=4, dims=[1, 3598, 4, 1], n_elems=14392,_
↳ size=28784, fmt=NCHW, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000,_
↳ pass_through=0
```

(下页继续)

(续上页)

```
index=1, name=p2o.Concat.9, n_dims=4, dims=[1, 80, 3598, 1], n_elems=287840,
↪ size=575680, fmt=NCHW, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000, ↪
↪ pass_through=0
[INFO] fastdeploy/runtime/runtime.cc(449)::CreateRKNPU2Backend Runtime ↪
↪ initialized with Backend::RKNPU2 in Device::RKNPU.
[WARNING] fastdeploy/runtime/backends/rknpu2/rknpu2_backend.cc(315)::Infer ↪
↪ The input tensor type != model's inputs type.The input_type need FP16,
↪ but inputs[0].type is FP32
[INFO] fastdeploy/runtime/backends/rknpu2/rknpu2_backend.cc(328)::Infer The ↪
↪ input model is not a quantitative model. Close the normalize operation.
[WARNING] fastdeploy/runtime/backends/rknpu2/rknpu2_backend.cc(315)::Infer ↪
↪ The input tensor type != model's inputs type.The input_type need FP16,
↪ but inputs[1].type is FP32
[INFO] fastdeploy/runtime/backends/rknpu2/rknpu2_backend.cc(328)::Infer The ↪
↪ input model is not a quantitative model. Close the normalize operation.
DetectionResult: [xmin, ymin, xmax, ymax, score, label_id]
268.500000,91.562500, 329.500000, 290.500000, 0.822266, 0
104.312500,84.187500, 129.500000, 171.500000, 0.810547, 0
172.375000,81.875000, 194.375000, 172.000000, 0.762207, 0
68.437500,47.156250, 82.312500, 96.187500, 0.759766, 0
380.000000,117.250000, 398.000000, 183.125000, 0.750977, 0
213.125000,41.343750, 224.125000, 82.125000, 0.666016, 0
246.125000,42.875000, 257.500000, 85.500000, 0.535645, 0
331.750000,119.625000, 389.750000, 283.500000, 0.502930, 0
229.000000,46.625000, 245.125000, 105.937500, 0.500000, 0
37.968750,140.625000, 72.687500, 178.875000, 0.493164, 0
122.000000,47.031250, 129.500000, 63.625000, 0.430420, 0
15.351562,119.625000, 35.531250, 156.750000, 0.406006, 0
328.000000,118.000000, 361.500000, 276.500000, 0.348145, 0
229.000000,45.750000, 239.750000, 85.875000, 0.334229, 0
234.000000,56.343750, 248.875000, 109.875000, 0.318115, 0
0.632812,154.875000, 24.468750, 177.250000, 0.354248, 24
```

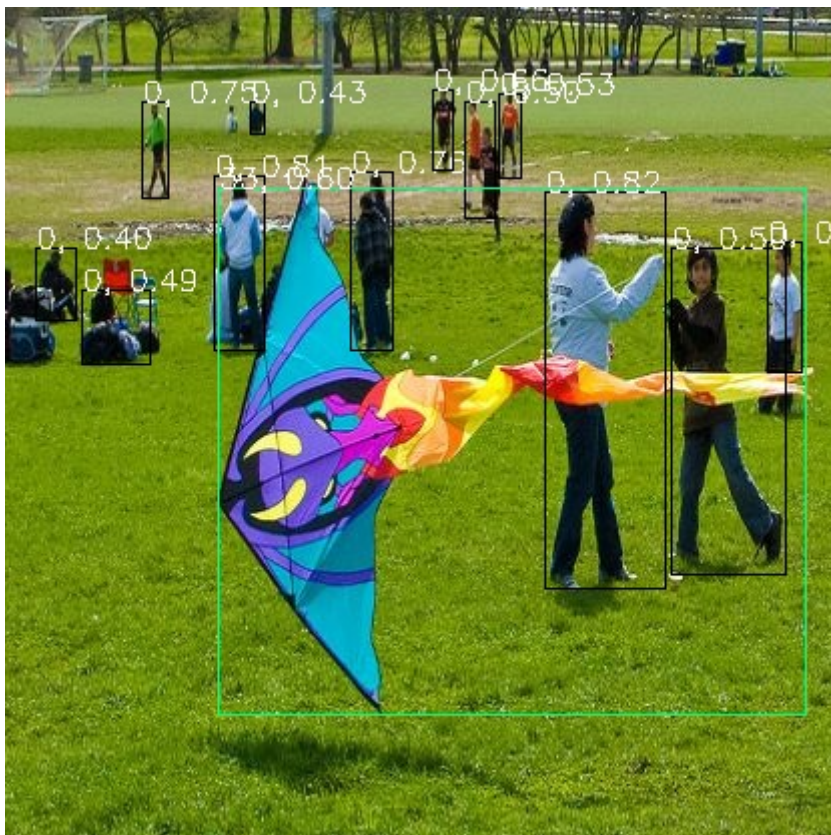
(下页继续)

(续上页)

```
106.250000,89.875000, 399.500000, 354.250000, 0.607422, 33
107.000000,87.250000, 206.250000, 354.000000, 0.363037, 33
```

Visualized result save in ./visualized_result.jpg

Simple test results show:



24.4 Reference link

For more documents such as environment construction, deployment optimization, and deployment examples, please refer to the following:

<https://github.com/PaddlePaddle/FastDeploy/tree/develop/docs>



<https://github.com/PaddlePaddle/FastDeploy/tree/develop/docs/cn/faq/rknpv2>

<https://github.com/rockchip-linux/rknn-toolkit2>

Chapter25 Flower image classification— TensorFlow

TensorFlow is a symbolic mathematical system based on data stream programming (DataFlow Programming). It is widely used in programming implementation of various machine learning algorithms.

This chapter will briefly introduce a flower image classification task of TensorFlow, use TF.KERAS.SEQUENTIAL model to simply build a model, and finally convert it to the RKNN model to deploy the RK series board in Lubancat.

提示: Test environment: The Lubancat board uses Debian10, and the PC side is Ubuntu20.04. TensorFlow version 2.6.2, RKNN-Toolkit2 version 1.4.0.

25.1 Install TensorFlow environment

Use Ubuntu or Windows system to install TensorFlow in PC. The following example is in Ubuntu20.04 system:

```
# Install Python and update PIP, you need to use Python 3.6-3.9 and PIP 19.
↪0 and higher versions
sudo apt update
sudo apt install python3-dev python3-pip python3-venv
sudo python3 -m pip install pip --upgrade

# Generally, a virtual environment is created
sudo python3 -m venv .tensorflow_venv
```

(下页继续)

(续上页)

```
# Order the latest stable version Tensorflow directly, or you can specify  
↳ the next version (GPU and CPU merger)  
pip3 install tensorflow
```

For detailed installation steps and requirements, please refer to [Tensorflow tutorial](#) .

提示: In the Windows system (Windows64 -bit operating system), you need to install Python, VC ++, *Anaconda* <<https://www.anaconda.com/products/distribution>> _, etc. If the NVIDIA GPU is required, the GPU driver is required, CUDA, CUDNN. The version between them corresponds to reference. Here is <https://tensorflow.google.cn/install/source_windows?hl=en#gpu> _. Between the graphics card driver and the CUDA version, please refer to the following <<https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html>>. There are many installation tutorials on the Internet. According to your own equipment, search for a new date document.

25.2 Image classification

The following will introduce a simple example to classify flower images using TF.KERAS.SEQUENTIAL model. For details, please refer to <https://tensorflow.google.cn/tutorials/images/classification#overfitting>

25.2.1 Prepare data set

Download data set, including 3700 images, use `tf.keras.utils.image_dataset_from_directory` to divide the data 80% for test sets, and the remaining 20% is used to verify the test set.

列表 1: tensorflow_classification.py(Reference supporting example)

```
1  # Retrieve data
2  import pathlib
3  dataset_url = "https://storage.googleapis.com/download.tensorflow.org/
   ↳example_images/flower_photos.tgz"
4  data_dir = tf.keras.utils.get_file('flower_photos', origin=dataset_url,
   ↳untar=True)
5  data_dir = pathlib.Path(data_dir)
6
7  # Setting parameters
8  batch_size = 32
9  img_height = 180
10 img_width = 180
11
12 # Use tf.keras.utils to introduce the dividing data set, divided into
   ↳training sets and verification sets
13 train_ds = tf.keras.utils.image_dataset_from_directory(
14 data_dir,
15 validation_split=0.2,
16 subset="training",
17 seed=123,
18 image_size=(img_height, img_width),
19 batch_size=batch_size)
20
21 val_ds = tf.keras.utils.image_dataset_from_directory(
22 data_dir,
23 validation_split=0.2,
24 subset="validation",
25 seed=123,
26 image_size=(img_height, img_width),
```

(下页继续)

(续上页)

```
27 batch_size=batch_size)
28
29 class_names = train_ds.class_names
30 #print(class_names)
31
32 # Process data
33 normalization_layer = layers.Rescaling(1./255)
34 train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
35 val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
36 num_classes = len(class_names)
```

提示: Some API interface descriptions and use of TensorFlow are referenced <https://tensorflow.google.cn/api_docs/python/tf>

25.2.2 Construction and training model

Keras Sequential model construction: It mainly contains three volumes (TF.KERAS.Layers.conv2D), each of which has a largest pooling layer (TF.karas.Layers.maxpooling2D) in each convolutional block. Finally, there is a full connection layer (TF.KERAS.Layers.Density), all use RELU as the activation function.

列表 2: tensorflow_classification.py(Reference supporting example)

```
1 # Construct a network structure
2 model = Sequential([
3     layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
4     layers.Conv2D(16, 3, padding='same', activation='relu'),
5     layers.MaxPooling2D(),
6     layers.Conv2D(32, 3, padding='same', activation='relu'),
```

(下页继续)

(续上页)

```
7     layers.MaxPooling2D(),
8     layers.Conv2D(64, 3, padding='same', activation='relu'),
9     layers.MaxPooling2D(),
10    layers.Flatten(),
11    layers.Dense(128, activation='relu'),
12    layers.Dense(num_classes)
13 ])
14
15 # Configure the optimizer 'adam', loss function, evaluation indicator
16 model.compile(optimizer='adam',
17               loss=tf.keras.losses.SparseCategoricalCrossentropy(from_
18 ↪ logits=True),
19               metrics=['accuracy'])
20
21 # Check the network structure of each layer
22 model.summary()
23
24 # Training model
25 epochs=10
26 model.fit(
27     train_ds,
28     validation_data=val_ds,
29     epochs=epochs,
30 )
```

列表 3: Check the network structure of each layer

Model: "sequential_1"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 180, 180, 3)	0

(下页继续)

(续上页)

conv2d (Conv2D)	(None, 180, 180, 16)	448
max_pooling2d (MaxPooling2D)	(None, 90, 90, 16)	0
conv2d_1 (Conv2D)	(None, 90, 90, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 45, 45, 32)	0
conv2d_2 (Conv2D)	(None, 45, 45, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 22, 22, 64)	0
flatten (Flatten)	(None, 30976)	0
dense (Dense)	(None, 128)	3965056
dense_1 (Dense)	(None, 5)	645
=====		
Total params: 3,989,285		
Trainable params: 3,989,285		
Non-trainable params: 0		

```
92/92 [=====] - 65s 644ms/step - loss: 1.4799 -  
accuracy: 0.3689 - val_loss: 1.1546 - val_accuracy: 0.4877  
Epoch 2/10  
92/92 [=====] - 52s 560ms/step - loss: 1.0406 -  
accuracy: 0.5777 - val_loss: 1.0439 - val_accuracy: 0.5913  
Epoch 3/10  
92/92 [=====] - 51s 555ms/step - loss: 0.8372 -  
accuracy: 0.6764 - val_loss: 1.0274 - val_accuracy: 0.6022
```

(下页继续)

(续上页)

```
Epoch 4/10
92/92 [=====] - 51s 552ms/step - loss: 0.6344 -
↪accuracy: 0.7698 - val_loss: 1.0563 - val_accuracy: 0.5831
Epoch 5/10
92/92 [=====] - 51s 553ms/step - loss: 0.4266 -
↪accuracy: 0.8539 - val_loss: 1.2350 - val_accuracy: 0.6104
Epoch 6/10
92/92 [=====] - 51s 553ms/step - loss: 0.2695 -
↪accuracy: 0.9087 - val_loss: 1.4331 - val_accuracy: 0.6213
Epoch 7/10
92/92 [=====] - 51s 554ms/step - loss: 0.1561 -
↪accuracy: 0.9533 - val_loss: 1.6564 - val_accuracy: 0.5981
Epoch 8/10
92/92 [=====] - 51s 554ms/step - loss: 0.0725 -
↪accuracy: 0.9785 - val_loss: 2.0034 - val_accuracy: 0.6281
Epoch 9/10
92/92 [=====] - 51s 556ms/step - loss: 0.0440 -
↪accuracy: 0.9888 - val_loss: 2.1280 - val_accuracy: 0.6076
Epoch 10/10
92/92 [=====] - 51s 555ms/step - loss: 0.0194 -
↪accuracy: 0.9973 - val_loss: 2.4841 - val_accuracy: 0.5886
```

Judging from the above training results, the training accuracy increased linearly over 99% over time, and the verification accuracy was about 60%. Too much difference between training accuracy and verification accuracy is overfitting phenomenon. To put it simply, the model can get better training data on training data than other data, but it does not achieve effect on the data set outside the training data, which means that the model is difficult to promote on the new dataset.

Generally, the reason for fitting is that the number of data set samples is too small, the data noise interference in the sample is too large, and the learning model is too complicated. Judging from the training here, it may be that the training dataset is too small, the model remembers noise, etc., and ignores the relationship between the real input and output, resulting in excessive fitting.

We extend the scale of the dataset here, carry out random flip, rotation, and zooming images at the Keras pre-processing layer, and use `tf.keras.Layers.randomflip`, `tf.keras.Layers.randomrotation` <https://tensorflow.google.cn/api_docs/python/tf/Layers/randomrotation> _ and `tf.keras.layers.randomzoom` <https://tensorflow.google.cn/api_docs/python/tf/keras/randomzoom> _ interface. In addition, add the Dropout layer, and for neural network units, it is temporarily discarded from the network at a certain probability. Program examples are as follows:

列表 4: tensorflow_classification.py(Reference supporting example)

```
1  # Build a network structure, add data_augmentation, and pre-processing_
   ↪ layer
2  data_augmentation = keras.Sequential(
3      [
4          layers.RandomFlip("horizontal",
5                             input_shape=(img_height,
6                                             img_width,
7                                             3)),
8          layers.RandomRotation(0.1),
9          layers.RandomZoom(0.1),
10     ]
11 )
12
13 model = Sequential([
14     data_augmentation,      # Add data_augmentation
15     layers.Conv2D(16, 3, padding='same', activation='relu'),
16     layers.MaxPooling2D(),
17     layers.Conv2D(32, 3, padding='same', activation='relu'),
18     layers.MaxPooling2D(),
19     layers.Conv2D(64, 3, padding='same', activation='relu'),
20     layers.MaxPooling2D(),
21     layers.Dropout(0.2),    # Add Dropout layer
22     layers.Flatten(),
```

(下页继续)

(续上页)

```
23     layers.Dense(128, activation='relu'),  
24     layers.Dense(num_classes, name="outputs")  
25 ])
```

Simple test results:

```
Epoch 1/15  
2023-02-20 19:39:40.074247: I tensorflow/stream_executor/cuda/cuda_dnn.  
→cc:369] Loaded cuDNN version 8202  
92/92 [=====] - 115s 759ms/step - loss: 1.3321 -  
→accuracy: 0.4179 - val_loss: 1.0552 - val_accuracy: 0.5872  
Epoch 2/15  
92/92 [=====] - 58s 635ms/step - loss: 1.0306 -  
→accuracy: 0.5978 - val_loss: 0.9923 - val_accuracy: 0.6267  
Epoch 3/15  
92/92 [=====] - 58s 635ms/step - loss: 0.9465 -  
→accuracy: 0.6345 - val_loss: 0.9616 - val_accuracy: 0.6240  
Epoch 4/15  
92/92 [=====] - 61s 661ms/step - loss: 0.8700 -  
→accuracy: 0.6604 - val_loss: 0.8555 - val_accuracy: 0.6717  
Epoch 5/15  
92/92 [=====] - 61s 665ms/step - loss: 0.7899 -  
→accuracy: 0.6993 - val_loss: 0.8514 - val_accuracy: 0.6717  
Epoch 6/15  
92/92 [=====] - 57s 620ms/step - loss: 0.7789 -  
→accuracy: 0.6979 - val_loss: 0.7658 - val_accuracy: 0.6907  
Epoch 7/15  
92/92 [=====] - 59s 640ms/step - loss: 0.7197 -  
→accuracy: 0.7159 - val_loss: 0.7940 - val_accuracy: 0.6826  
Epoch 8/15  
92/92 [=====] - 59s 644ms/step - loss: 0.6956 -  
→accuracy: 0.7367 - val_loss: 0.7710 - val_accuracy: 0.6907
```

(下页继续)

(续上页)

```
Epoch 9/15
92/92 [=====] - 60s 657ms/step - loss: 0.6532 -
↪accuracy: 0.7510 - val_loss: 0.7250 - val_accuracy: 0.7112
Epoch 10/15
92/92 [=====] - 62s 672ms/step - loss: 0.6189 -
↪accuracy: 0.7657 - val_loss: 0.7893 - val_accuracy: 0.7044
Epoch 11/15
92/92 [=====] - 60s 652ms/step - loss: 0.6221 -
↪accuracy: 0.7674 - val_loss: 0.6982 - val_accuracy: 0.7262
Epoch 12/15
92/92 [=====] - 60s 651ms/step - loss: 0.5598 -
↪accuracy: 0.7888 - val_loss: 0.6821 - val_accuracy: 0.7357
Epoch 13/15
92/92 [=====] - 60s 649ms/step - loss: 0.5519 -
↪accuracy: 0.7885 - val_loss: 0.7939 - val_accuracy: 0.7084
Epoch 14/15
92/92 [=====] - 61s 667ms/step - loss: 0.5387 -
↪accuracy: 0.7997 - val_loss: 0.8331 - val_accuracy: 0.6880
Epoch 15/15
92/92 [=====] - 60s 653ms/step - loss: 0.5068 -
↪accuracy: 0.8151 - val_loss: 0.6627 - val_accuracy: 0.7466
```

After Keras pre-processing layer and adding the Dropout layer. You can get the training model.

25.2.3 Test model

列表 5: tensorflow_classification.py(Reference supporting example)

```
1 # Get the import picture
2 sunflower_url = "https://storage.googleapis.com/download.tensorflow.org/
↪example_images/592px-Red_sunflower.jpg"
```

(下页继续)

(续上页)

```
3 sunflower_path = tf.keras.utils.get_file('Red_sunflower', origin=sunflower_  
    ↪url)  
4  
5 img = tf.keras.utils.load_img(  
6     sunflower_path, target_size=(img_height, img_width)  
7 )  
8 img_array = tf.keras.utils.img_to_array(img)  
9 img_array = tf.expand_dims(img_array, 0)  
10  
11 # Test  
12 predictions = model.predict(img_array)  
13 score = tf.nn.softmax(predictions[0])  
14  
15 # output the result  
16 print(  
17     "This image most likely belongs to {} with a {:.2f} percent confidence."  
18     .format(class_names[np.argmax(score)], 100 * np.max(score))  
19 )
```

25.2.4 TensorFlow Lite model

The trained Keras Sequential model, here uses `tf.lite.tfliteconverter.from_keras_model` to generate TensorFlow Lite model:

列表 6: tensorflow_classification.py(Reference supporting example)

```
1 # Convert the model.  
2 converter = tf.lite.TFLiteConverter.from_keras_model(model)  
3 tflite_model = converter.convert()  
4
```

(下页继续)

(续上页)

```
5 # Save the model.
6 with open('model.tflite', 'wb') as f:
7     f.write(tflite_model)
```

TensorFlow Lite converter conversion workflow introduction reference [tutorial](#)

25.2.5 Model conversion and simulation test

Use Rknn-Toolkit2 to export the RKNN model:

列表 7: rknn_transfer.py(Reference supporting example)

```
1 img_height = 180
2 img_width = 180
3 IMG_PATH = 'test.jpg'
4 class_names = ['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
5
6 if __name__ == '__main__':
7
8     # Create RKNN object
9     #rknn = RKNN(verbose='Debug')
10    rknn = RKNN()
11
12    # Pre-process config
13    print('--> Config model')
14    rknn.config(mean_values=[0, 0, 0], std_values=[255, 255, 255], target_
15    platform='rk3568')
16    print('done')
17
18    # Load model
19    print('--> Loading model')
20    ret = rknn.load_tflite(model='model.tflite')
```

(下页继续)

(续上页)

```
20     if ret != 0:
21         print('Load model failed!')
22         exit(ret)
23     print('done')
24
25     # Build model
26     print('--> Building model')
27     ret = rknn.build(do_quantization=False)
28     #ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
29     if ret != 0:
30         print('Build model failed!')
31         exit(ret)
32     print('done')
33
34     # Export rknn model
35     print('--> Export rknn model')
36     ret = rknn.export_rknn('./model.rknn')
37     if ret != 0:
38         print('Export rknn model failed!')
39         exit(ret)
40     print('done')
41
42     #Init runtime environment
43     print('--> Init runtime environment')
44     ret = rknn.init_runtime()
45     #     if ret != 0:
46     #         print('Init runtime environment failed!')
47     #         exit(ret)
48     print('done')
49
50     img = cv2.imread(IMG_PATH)
51     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

(下页继续)

(续上页)

```
52 img = cv2.resize(img, (180, 180))
53 img = np.expand_dims(img, 0)
54
55 #print('--> Accuracy analysis')
56 #rknn.accuracy_analysis(inputs=['./test.jpg'])
57 #print('done')
58
59 print('--> Running model')
60 outputs = rknn.inference(inputs=[img])
61 print(outputs)
62 outputs = tf.nn.softmax(outputs)
63 print(outputs)
64
65 print(
66     "This image most likely belongs to {} with a {:.2f} percent confidence."
67     .format(class_names[np.argmax(outputs)], 100 * np.max(outputs))
68 )
69 #print("The image prediction is:", class_names[np.argmax(outputs)])
70 print('done')
71
72 rknn.release()
```

Test results show:

```
W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
--> Config model
done
--> Loading model
done
--> Building model
done
--> Export rknn model
```

(下页继续)



```
done
```

```
--> Init runtime environment  
W init_runtime: Target is None, use simulator!  
done  
--> Running model  
Analysing : 100%|███████████████████████████████████████████| 18/18└  
↳[00:00<00:00, 90.12it/s]  
Preparing : 100%|███████████████████████████████████████████| 18/18└  
↳[00:01<00:00, 13.98it/s]  
[array([[ -4.5390625 ,  -1.2275391 ,  -0.47338867,   4.75          ,    0.34350586]],  
       dtype=float32)]  
tf.Tensor([[[[9.0598274e-05  2.4848275e-03  5.2822586e-03  9.8018610e-01  1.  
↳1956180e-02]]], shape=(1, 1, 5), dtype=float32)  
This image most likely belongs to sunflowers with a 98.02 percent└  
↳confidence.
```

```
done
```

Deploy reasoning on the lubancat board, use RKNN-Toolkit-Lite2, take RK356X as an example:

列表 8: rknnlite_inference.py(Reference supporting example)

(下页继续)

(续上页)

```
8 rknn_lite = RKNNLite()
9
10 # load RKNN model
11 print('--> Load RKNN model')
12 ret = rknn_lite.load_rknn(RKNN_MODEL)
13 if ret != 0:
14     print('Load RKNN model failed')
15     exit(ret)
16 print('done')
17
18 # Init runtime environment
19 print('--> Init runtime environment')
20 ret = rknn_lite.init_runtime()
21 if ret != 0:
22     print('Init runtime environment failed!')
23     exit(ret)
24 print('done')
25
26 # load image
27 img = cv2.imread(IMG_PATH)
28 img = cv2.resize(img, (180, 180))
29 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
30 img = np.expand_dims(img, 0)
31
32 # runing model
33 print('--> Running model')
34 outputs = rknn_lite.inference(inputs=[img])
35 print("result: ", outputs)
36 print(
37     "This image most likely belongs to {}."
38     .format(class_names[np.argmax(outputs)])
39 )
```

(下页继续)

(续上页)

40

41

```
rknn_lite.release()
```

Simple test results on the board:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [17:28:01.594] RKNN Runtime Information: librknnrt version: 1.4.0
↳ (a10f100eb@2022-09-09T09:07:14)
I RKNN: [17:28:01.594] RKNN Driver Information: version: 0.7.2
I RKNN: [17:28:01.595] RKNN Model Information: version: 1, toolkit version:
↳ 1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳ target: RKNPU lite, target platform: rk3568, framework name: TFLite,
↳ framework layout: NHWC
done
--> Running model
result: [array([[ -3.7226562, -1.2607422, -0.5805664,  3.5585938,  0.296875
↳ ]],
        dtype=float32)]
This image most likely belongs to sunflowers.
done
```

25.4 Summarize

Use TensorFlow to complete a simple flower image classification, and convert the model to the RKNN model. It is deployed on the Lubancat board. The model verification accuracy is more than 70%, and it is used for simple learning. This model test comes from the *image classification tutorial* <<https://tensorflow.google.cn/tutorials/classification>> _.

25.5 Reference link

<https://tensorflow.google.cn/learn?hl=zh-cn>

<https://tensorflow.google.cn/tutorials/images/classification>

<https://tensorflow.google.cn/tutorials?hl=zh-cn>

Chapter26 Resnet18 network –PyTorch

ResNet18 is a convolutional neural network, which has 18 layers of depth, including a heavy convolutional layer and full connection layer with weights. It is a variant of the RESNET series network, using the residual connection to solve the degradation problem of the deep network.

This chapter will briefly introduce PyTorch and the installation environment, and then briefly analyze the next resnet neural network and the source code of PyTorch. Finally, we simply build a resnet18 network with PyTorch to classify the CIFAR-10 and deploy it on the LubanCat board.

提示: Test environment: The Lubancat board uses Debian10, PC is WSL2 (Ubuntu20.04). PyTorch is 1.10.1 CPU version, RKNN-Toolkit2 version 1.4.0.

26.1 PyTorch and ResNet18

Pytorch is an open source deep learning framework that was developed by the TORCH7 team of Facebook Artificial Intelligence Research Institute. It is based on TORCH, but the implementation and application are completed by Python. This framework is mainly used for scientific research and application development in the field of artificial intelligence.

26.1.1 Pytorch installation

Pytorch needs to be installed according to your own environment to enter the official website of PyTorch and check the detailed installation tutorial. The following example is simply installed on Ubuntu20.04:

First go to the *pytorch official website* <<https://pytorch.org/>> _, and select your corresponding environment, such as the pytorch of the Linux system, Python language, and CPU version.

PyTorch Build	Stable (1.13.1)		Preview (Nightly)	
Your OS	Linux	Mac		Windows
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.6	CUDA 11.7	ROCm 5.2	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cpu</pre>			

The default on the page is the latest stable version of PyTorch. If you install the previous version, you can click on the page previous version of pytorch Or click [here](https://pytorch.org/get-started/previous-versions/) <<https://pytorch.org/get-started/previous-versions/>> _.

```
# With PyTorch, you need to install the Python3 and PIP basic environment.
↪first, which can be searched by yourself.
# CPU version:
pip3 install torch torchvision torchaudio --extra-index-url https://
↪download.pytorch.org/whl/cpu

# GPU version (CUDA 11.6):
pip3 install torch torchvision torchaudio --extra-index-url https://
↪download.pytorch.org/whl/cu116
```

Install the GPU version (Nvidia GPU graphics card), you need to install it according to your own graphics card or update [graphics driver](#) . Then install the [CUDA toolkit](#) , [cuDNN](#) .Look at your own situation to install. Finally click [here](#) to check the version of the pytorch and CUDA.

Verify whether the installation is successful:

```
# Test installation, the terminal input python,
>>> import torch
```

(下页继续)

(续上页)

```
>>> torch.__version__
'1.10.1+cu102'

# If you install the CUDA version of PyTorch, you can use the command to
↪ detect the installation version of PyTorch and the binding CUDA version.
>>> torch.version.cuda
'10.2'
>>> torch.cuda.is_available()
True
# Pytorch installation can also use environment and other environments.
↪ according to its actual environment and needs.
```

Regarding program editing tools, you can use Sublime Text, PyCharm, Vim, etc. Here the test environment is to use WSL2 (Ubuntu20.04). Edit Tools use the Jupyter Notebook. You can refer to the installation tutorial. You can refer to it ('<<https://doc.embedfire.com/linux/rk356x/Python/zh/toolpackages/jupyter.html>> ' _). It is similar to using on the Linux system.

26.1.2 ResNet18 Structure Introduction

Residual Neural Network was proposed by the Microsoft Research Institute's Kaiming He and others in 2015. The structure of the resnet can accelerate the training of neural networks very quickly, and the accuracy of the model has also improved.

Resnet is a residual network that can understand it as a sub-network. This sub-network can form a deep network after stacking. There are many variants in the resnet series, such as ResNet18, ResNet34, ResNet50, ResNet101 and ResNet152. Its network structure is as follows (Reference *paper* <<https://arxiv.org/abs/1512.03385>> _):

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Here we mainly look at ResNet18. The basic meaning of ResNet18 is that the basic architecture of the network is ResNet. The depth of the network is 18 layers. It is 18 layers with weights, excluding BN layers and pooling layers. The basic residual units used in ResNet18 are composed of two 3X3 convolutional layers. There is a BN layer and a RELU activation function in the middle.

26.1.3 Resnet18 in pytorch implementation

Resnet18 source code in pytorch: <https://github.com/pytorch/vision/blob/main/torchvision/models/resnet.py>

26.2 Resnet18 implementation

26.2.1 Data set preparation and data pre -processing

Next, we will customize a resnet18 network structure and use the CIFAR-10 dataset for a simple test. The CIFAR-10 dataset consists of 10 categories of 60,000 32X32 color images. Each category has 6,000 images, which are divided into 50,000 trained images and 10,000 test images.

列表 1: resnet18.py(part of the code reference supporting example)

```
1 # Import the downloaded data set and use TORCHVISION to load training sets_
  ↳and test sets.
2 # The parameter Download = True indicates that downloading data from the_
  ↳Internet, stored to the ./data directory can also download it by yourself_
  ↳in the specified directory.
3 train_dataset = torchvision.datasets.CIFAR10('./data', download=True,
  ↳train=True, transform=transform_train)
4 test_dataset = torchvision.datasets.CIFAR10('./data', download=True,
  ↳train=False, transform=transform_test)
5
6 train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_
  ↳size, shuffle=True)
7 test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_
  ↳size, shuffle=False)
8
9 # classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
  ↳'ship', 'truck')
```

Prepare the divided dataset

列表 2: resnet18.py(part of the code reference supporting example)

```
1 # Pre -processing
2 transform_train=torchvision.transforms.Compose([
3     torchvision.transforms.Pad(4),
4     torchvision.transforms.RandomHorizontalFlip(), #The probability of_
  ↳the image is half, and the probability of half does not flip.
5     torchvision.transforms.RandomCrop(32), #Image random cut into 32*32
```

(下页继续)

(续上页)

```
6         torchvision.transforms.ToTensor(), #Turn to Tensor to transform the
↪gray range from 0-255 to 0-1, and normalize
7         #torchvision.transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023,
↪0.1994, 0.2010))
8         torchvision.transforms.Normalize([0.5,0.5,0.5], [0.5,0.5,0.5]) #The
↪average value and variance used in normalization
9     ])
10 transform_test=torchvision.transforms.Compose([
11         torchvision.transforms.ToTensor(),
12         #torchvision.transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023,
↪ 0.1994, 0.2010))
13         torchvision.transforms.Normalize([0.5,0.5,0.5], [0.5,0.5,0.5]) #The
↪average value and variance used in normalization
14     ])
```

26.2.2 Build a model

列表 3: resnet18.py(part of the code reference supporting example)

```
1 # Residual block implementation
2 class ResidualBlock(nn.Module):
3     def __init__(self, in_channels, out_channels, stride=1, downsample=None):
4         super(ResidualBlock, self).__init__()
5         self.conv1 = conv3x3(in_channels, out_channels, stride)
6         self.bn1 = nn.BatchNorm2d(out_channels)
7         self.relu = nn.ReLU(inplace=True) # Replace in place to
↪save memory overhead
8         self.conv2 = conv3x3(out_channels, out_channels)
9         self.bn2 = nn.BatchNorm2d(out_channels)
```

(下页继续)

(续上页)

```
10     self.downsample = downsample                                # shortcut
11 def forward(self, x):
12     residual=x
13     out = self.conv1(x)
14     out = self.bn1(out)
15     out = self.relu(out)
16     out = self.conv2(out)
17     out = self.bn2(out)
18     if(self.downsample):
19         residual = self.downsample(x)
20     out += residual
21     out = self.relu(out)
22     return out
23
24 # Custom a neural network, use nn.model, and initialize each layer of_
  ↪neural network through __init__.
25 # Use Forward to connect data
26 class ResNet(torch.nn.Module):
27     def __init__(self, block, layers, num_classes):
28         super(ResNet, self).__init__()
29         self.in_channels = 16
30         self.conv = conv3x3(3, 16)
31         self.bn = torch.nn.BatchNorm2d(16)
32         self.relu = torch.nn.ReLU(inplace=True)
33         self.layer1 = self._make_layers(block, 16, layers[0])
34         self.layer2 = self._make_layers(block, 32, layers[1], 2)
35         self.layer3 = self._make_layers(block, 64, layers[2], 2)
36         self.layer4 = self._make_layers(block, 128, layers[3], 2)
37         self.avg_pool = torch.nn.AdaptiveAvgPool2d((1, 1))
38         self.fc = torch.nn.Linear(128, num_classes)
39
40     # _make_layers function repeats the residual difference, and the_
  ↪ShortCut part
```

(下页继续)

(续上页)

```
41     def _make_layers(self, block, out_channels, blocks, stride=1):
42         downsample = None
43         if (stride != 1) or (self.in_channels != out_channels):      # The
↪convolution nucleus is 1 for lift dimensions for 1
44             downsample = torch.nn.Sequential(                        # When
↪stride == 2, it is when the output channel is raised at each output
↪channel
45                 conv3x3(self.in_channels, out_channels, stride=stride),
46                 torch.nn.BatchNorm2d(out_channels)
47             )
48             layers = []
49             layers.append(block(self.in_channels, out_channels, stride,
↪downsample))
50             self.in_channels = out_channels
51             for i in range(1, blocks):
52                 layers.append(block(out_channels, out_channels))
53             return torch.nn.Sequential(*layers)
54
55     def forward(self, x):
56         out = self.conv(x)
57         out = self.bn(out)
58         out = self.relu(out)
59         out = self.layer1(out)
60         out = self.layer2(out)
61         out = self.layer3(out)
62         out = self.layer4(out)
63         out = self.avg_pool(out)
64         out = out.view(out.size(0), -1)
65         out = self.fc(out)
66         return out
67
68 # Make model, use cpu
```

(下页继续)

(续上页)

```
69 model=ResNet(ResidualBlock, [2,2,2,2], num_classes).to(device=device)
70
71 # Print the Model structure
72 print(f"Model structure: {model}\n\n")
73
74 # Loss and optimizer
75 criterion = nn.CrossEntropyLoss() #Cross entropy loss function
76 optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate) #Optimal_
↪random gradient decrease
```

Export model structure during testing:

```
Model structure: ResNet (
  (conv): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
↪ bias=False)
  (bn): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
  (relu): ReLU(inplace=True)
  (layer1): Sequential(
    (0): ResidualBlock(
      (conv1): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1),
↪padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1),
↪padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
    )
    (1): ResidualBlock(
      (conv1): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1),
↪padding=(1, 1), bias=False)
```

(下页继续)

(续上页)

```
(bn1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
(relu): ReLU(inplace=True)
(conv2): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
(bn2): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
)
)
(layer2): Sequential(
  (0): ResidualBlock(
    (conv1): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), ↪
↪padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), ↪
↪padding=(1, 1), bias=False)
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, ↪
↪track_running_stats=True)
    )
  )
  (1): ResidualBlock(
    (conv1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
```

(下页继续)

(续上页)

```
(relu): ReLU(inplace=True)
(conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
(bn2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
)
)
(layer3): Sequential(
  (0): ResidualBlock(
    (conv1): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), ↵
↵padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), ↵
↵padding=(1, 1), bias=False)
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, ↵
↵track_running_stats=True)
    )
  )
  (1): ResidualBlock(
    (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
```

(下页继续)

(续上页)

```
(bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_
↳running_stats=True)
)
)
(layer4): Sequential(
  (0): ResidualBlock(
    (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), ↳
↳padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_
↳running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), ↳
↳padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_
↳running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), ↳
↳padding=(1, 1), bias=False)
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, ↳
↳track_running_stats=True)
    )
  )
  (1): ResidualBlock(
    (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), ↳
↳padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_
↳running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), ↳
↳padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_
↳running_stats=True)
```

(下页继续)

(续上页)

```
)  
  
)  
(avg_pool): AdaptiveAvgPool2d(output_size=(1, 1))  
(fc): Linear(in_features=128, out_features=10, bias=True)  
)
```

26.2.3 Training and test model

列表 4: resnet18.py(part of the code reference supporting example)

```
1 if __name__ == "__main__":  
2     # Training model  
3     total_step = len(train_loader)  
4     for epoch in range(0, num_epochs):  
5         for i, (images, labels) in enumerate(train_loader):  
6             images = images.to(device=device)  
7             labels = labels.to(device=device)  
8             outputs = model(images)  
9             loss = criterion(outputs, labels)  
10            # Gradient clearing  
11            optimizer.zero_grad()  
12            # Reverse propagation  
13            loss.backward()  
14            # Update parameter  
15            optimizer.step()  
16            if (i+1) % total_step == 0:  
17                print('Epoch [{} / {}], Step [{} / {}], Loss: {:.4f}'.  
18                    .format(epoch+1, num_epochs, i+1, total_step, loss.  
19                    ↪ item()))  
20            print("Finished Training")
```

```
print('\nTest the model')
# Convert to `Eval` Mode
model.eval() # eval mode (batchnorm uses moving mean/variance instead of
↳mini-batch mean/variance)
with torch.no_grad():

    correct = 0
    total = 0

    for images, labels in test_loader:

        images = images.to(device=device)
        labels = labels.to(device=device)
        outputs = model(images)

        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    print('Accuracy of the network on the 10000 test images: {} %'.
↳format(100 * correct / total))
```

The training model, then the test accuracy on the test set reaches 89.8400%:

```
Epoch [88/100], Step [391/391], Loss: 0.0820
Epoch [89/100], Step [391/391], Loss: 0.0185
Epoch [90/100], Step [391/391], Loss: 0.0166
Epoch [91/100], Step [391/391], Loss: 0.0334
Epoch [92/100], Step [391/391], Loss: 0.0641
Epoch [93/100], Step [391/391], Loss: 0.0359
Epoch [94/100], Step [391/391], Loss: 0.0994
Epoch [95/100], Step [391/391], Loss: 0.0069
```

(下页继续)

(续上页)

```
Epoch [96/100], Step [391/391], Loss: 0.0722
Epoch [97/100], Step [391/391], Loss: 0.0182
Epoch [98/100], Step [391/391], Loss: 0.2182
Epoch [99/100], Step [391/391], Loss: 0.0657
Epoch [100/100], Step [391/391], Loss: 0.0501
Finished Training
```

Test the model

The accuracy rate in the test set picture picture :89.8400 %

26.2.4 Save as onnx model

Here we use `torch.onnx.export` to save the model as the onnx model (can also export PT models, etc.):

列表 5: resnet18.py(part of the code reference supporting example)

```
1 # export onnx (rknn-toolkit2 only support to opset_version=12)
2 x = torch.randn((1, 3, 32, 32))
3 torch.onnx.export(model, x, './resnet18_pytorch_100.onnx', opset_version=12,
  ↪ input_names=['input'], output_names=['output'])
```

26.2.5 Export the RKNN model and simulation test

列表 6: resnet18.py(part of the code reference supporting example)

```
1 def show_perfs(perfs):
2     perfs = 'perfs: {}'.format(perfs)
3     print(perfs)
```

(下页继续)

(续上页)

```
4
5 def softmax(x):
6     return np.exp(x) / sum(np.exp(x))
7
8 if __name__ == '__main__':
9
10     MODEL = './resnet18_pytorch.onnx'
11
12     # Create RKNN
13     # If the test encounters a problem, turn on Verbose = TRUE and check_
↪ the debugging information.
14     # rknn = RKNN(verbose=True)
15     rknn = RKNN()
16
17     # Configuration model, pre -processing
18     print('--> Config model')
19     rknn.config(mean_values=[125.307, 122.961, 113.8575], std_values=[51.
↪ 5865, 50.847, 51.255], target_platform='rk3568')
20     print('done')
21
22     # Loading model
23     print('--> Loading model')
24     #ret = rknn.load_pytorch(model=model, input_size_list=input_size_list)
25     ret = rknn.load_onnx(model=MODEL)
26     if ret != 0:
27         print('Load model failed!')
28         exit(ret)
29     print('done')
30
31     # Build a model
32     print('--> Building model')
33     ret = rknn.build(do_quantization=False)
```

(下页继续)

(续上页)

```
34     if ret != 0:
35         print('Build model failed!')
36         exit(ret)
37     print('done')
38
39     # Export the RKNN model
40     print('--> Export rknn model')
41     ret = rknn.export_rknn('./resnet_18_100.rknn')
42     if ret != 0:
43         print('Export rknn model failed!')
44         exit(ret)
45     print('done')
46
47     # Enter image processing
48     img = cv2.imread('./0_125.jpg')
49     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
50     img = cv2.resize(img, (32, 32))
51     img = np.expand_dims(img, 0)
52
53     # Initialized operating environment
54     print('--> Init runtime environment')
55     ret = rknn.init_runtime()
56     if ret != 0:
57         print('Init runtime environment failed!')
58         exit(ret)
59     print('done')
60
61     # Simulation reasoning
62     print('--> Running model')
63     outputs = rknn.inference(inputs=[img])
64     np.save('./pytorch_resnet18_qat_0.npy', outputs[0])
65     #show_outputs(softmax(np.array(outputs[0][0])))
```

(下页继续)

(续上页)

```
66     print(outputs)
67     print('done')
68
69     rknn.release()
```

26.2.6 Board deployment test

26.2.6.1 Simple test

列表 7: rknnlite_inference0.py

```
1  IMG_PATH = '0_125.jpg'
2  RKNN_MODEL = './resnet_18_100.rknn'
3  img_height = 32
4  img_width = 32
5  class_names = ["plane", "car", "bird", "cat", "deer", "dog", "frog", "horse", "ship",
6  ↪ "truck"]
7
8  # Create RKNN object
9  rknn_lite = RKNNLite()
10
11 # load RKNN model
12 print('--> Load RKNN model')
13 ret = rknn_lite.load_rknn(RKNN_MODEL)
14 if ret != 0:
15     print('Load RKNN model failed')
16     exit(ret)
17 print('done')
18
19 # Init runtime environment
```

(下页继续)

(续上页)

```
19 print('--> Init runtime environment')
20 ret = rknn_lite.init_runtime()
21 if ret != 0:
22     print('Init runtime environment failed!')
23     exit(ret)
24 print('done')
25
26 # load image
27 img = cv2.imread(IMG_PATH)
28 img = cv2.resize(img, (32, 32))
29 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
30 img = np.expand_dims(img, 0)
31
32 # runing model
33 print('--> Running model')
34 outputs = rknn_lite.inference(inputs=[img])
35 print("result: ", outputs)
36 print(
37     "This image most likely belongs to {}."
38     .format(class_names[np.argmax(outputs)])
39 )
40 rknn_lite.release()
```

Test Results:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [16:02:15.992] RKNN Runtime Information: librknnrt version: 1.4.0
↳ (a10f100eb@2022-09-09T09:07:14)
I RKNN: [16:02:15.992] RKNN Driver Information: version: 0.7.2
I RKNN: [16:02:15.992] RKNN Model Information: version: 1, toolkit version:
↳ 1.4.0-22dcfef4 (compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35))
↳ target: RKNPU lite, target platform: rk3568, framework name: ONNX,
↳ framework layout: NCHW
```

(续上页)

```
done
--> Running model
result:  [array([[ -2.0566406, -15.234375 ,   6.6835938,  -6.828125 ,  -9.
↪9921875,
          -6.5390625,  -5.671875 , -15.8515625, -17.96875   , -11.90625   ]],
          dtype=float32)]
This image most likely belongs to bird.
```

26.2.6.2 Use test set image test

First change the test set in the .jpg format, and then transmit it to the board for deployment test:

列表 8: cifar10_to_jpg.py

```
1  # The absolute path where the CIFAR-10 dataset is located, modify according_
↪to the specific path
2  base_dir = "/mnt/e/Users/Administrator/Desktop/wsl_user/pytorch/"
3  data_dir = os.path.join(base_dir, "data", "cifar-10-batches-py")
4  test_o_dir = os.path.join( base_dir, "Data", "cifar-10-png", "raw_test")
5
6  # unzip
7  def unpickle(file):
8      with open(file, 'rb') as fo:
9          dict_ = pickle.load(fo, encoding='bytes')
10         return dict_
11
12  # Generate test set pictures
13  if __name__ == '__main__':
14      print("start...")
15      test_data_path = os.path.join(data_dir, "test_batch")
16      test_data = unpickle(test_data_path)
```

(下页继续)

(续上页)

```
17     for i in range(0, 10000):
18         img = np.reshape(test_data[b'data'][i], (3, 32, 32))
19         img = img.transpose(1, 2, 0)
20
21         label_num = str(test_data[b'labels'][i])
22         o_dir = os.path.join(test_o_dir, label_num)
23         if not os.path.isdir(o_dir):
24             os.makedirs(o_dir)
25
26         img_name = label_num + '_' + str(i) + '.jpg'
27         img_path = os.path.join(o_dir, img_name)
28         imwrite(img_path, img)
29     print("done.")
```

Then modify the board test file to add:

列表 9: rknnlite_inference1.py

```
1 def rknn_inference(root):
2     total=0
3     correct=0
4     for path in os.listdir(root):
5         image_filenames = os.listdir(root + '/' + path)
6         for image_filename in image_filenames:
7             img = cv2.imread(root + '/' + path + '/' + image_filename)
8             img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
9             outputs = rknn_lite.inference(inputs=[img])
10            total += 1
11            if np.argmax(outputs) == int(path[:1]) :
12                correct += 1
13        print("corrorect={}, total={}".format(correct,total))
14    print('在{}张测试集图片上的准确率:{:.2f} %'.format(total,100 * correct /
total))
```

(下页继续)

(续上页)

```
#The accuracy rate on the test set picture
```

Simple test results:

```
1 --> Load RKNN model
2 done
3 --> Init runtime environment
4 I RKNN: [10:23:15.384] RKNN Runtime Information: librknrt version: 1.4.0
  ↳(a10f100eb@2022-09-09T09:07:14)
5 I RKNN: [10:23:15.385] RKNN Driver Information: version: 0.7.2
6 I RKNN: [10:23:15.385] RKNN Model Information: version: 1, toolkit version:
  ↳1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
  ↳target: RKNPU lite, target platform: rk3568, framework name: ONNX,
  ↳framework layout: NCHW
7 done
8 --> Running model
9 在 10000 张测试集图片上的准确率:71.21
10 # The accuracy rate on the 10,000 test set pictures: 71.21
11 done
```

26.3 Reference

<https://pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>

https://pytorch.org/hub/pytorch_vision_resnet/

<https://arxiv.org/abs/1512.03385>

Chapter27 YOLOv5

Yolov5 is a target detection algorithm that belongs to a single-stage target detection method. It is an object detection architecture and model series pre-trained on the COCO dataset. It represents Ultralytics' open source research on future visual AI methods. It contains lessons learned and best practices that have been developed through thousands of hours of research and development. The latest YOLOv5 v7.0 includes YOLOv5n, YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x, etc. In addition to target detection, there are also application scenarios such as segmentation and classification.

The basic principle of YOLOv5 is simply: divide the whole picture into several networks, each grid predicts the type and position information of the object in the grid, and then performs the calculation according to the intersection ratio between the predicted frame and the real frame. The screening of the target box, and finally output the prediction box.

This chapter will simply use YOLOv5, and simply deploy and test it on the Lubancat board.

提示: Test environment: Lubancat RK board system is Debian10, PC is WSL2 (ubuntu20.04), PyTorch is 1.10.1, rknn-Toolkit2 version 1.4.0, YOLOv5 v7.0, airockchip/yolov5 v6.2.

27.1 YOLOv5 environment installation

Install python and other environments, as well as related dependent libraries, and then clone the source code of the YOLOv5 warehouse.

```
# Install python and other environments, as well as related dependent_
↳ libraries, and then clone the source code of the YOLOv5 warehouse.
sudo python3 -m venv .yolov5_env
source .toolkit2_env/bin/activate
```

(下页继续)

(续上页)

```
# Pull the latest yolov5 warehouse, pytorch version
git clone https://github.com/ultralytics/yolov5
cd yolov5

# Install dependent libraries
pip3 install -r requirements.txt

# Enter the python command line to detect the installed environment
import torch
import utils
display = utils.notebook_init()

# The test environment here shows:
Checking setup...
YOLOv5 □ 2023-2-20 Python-3.8.10 torch-1.10.1+cpu CPU
Setup complete □ (6 CPUs, 12.4 GB RAM, 77.3/251.0 GB disk)
```

27.2 YOLOv5 is easy to use

27.2.1 Get the pre-trained weights file

Download yolov5s.pt, yolov5m.pt, yolov5l.pt, yolov5x.pt weight files, which can be directly obtained from [here](#). Among them, n, s, m, l, and x represent the width and depth of the network, and the smallest is n, which has the fastest speed and the lowest precision.

27.2.2 YOLOv5 simple test

Enter the yolov5 source code directory, put the previously downloaded weight file in the current directory, and the two test pictures are located in /data/images/

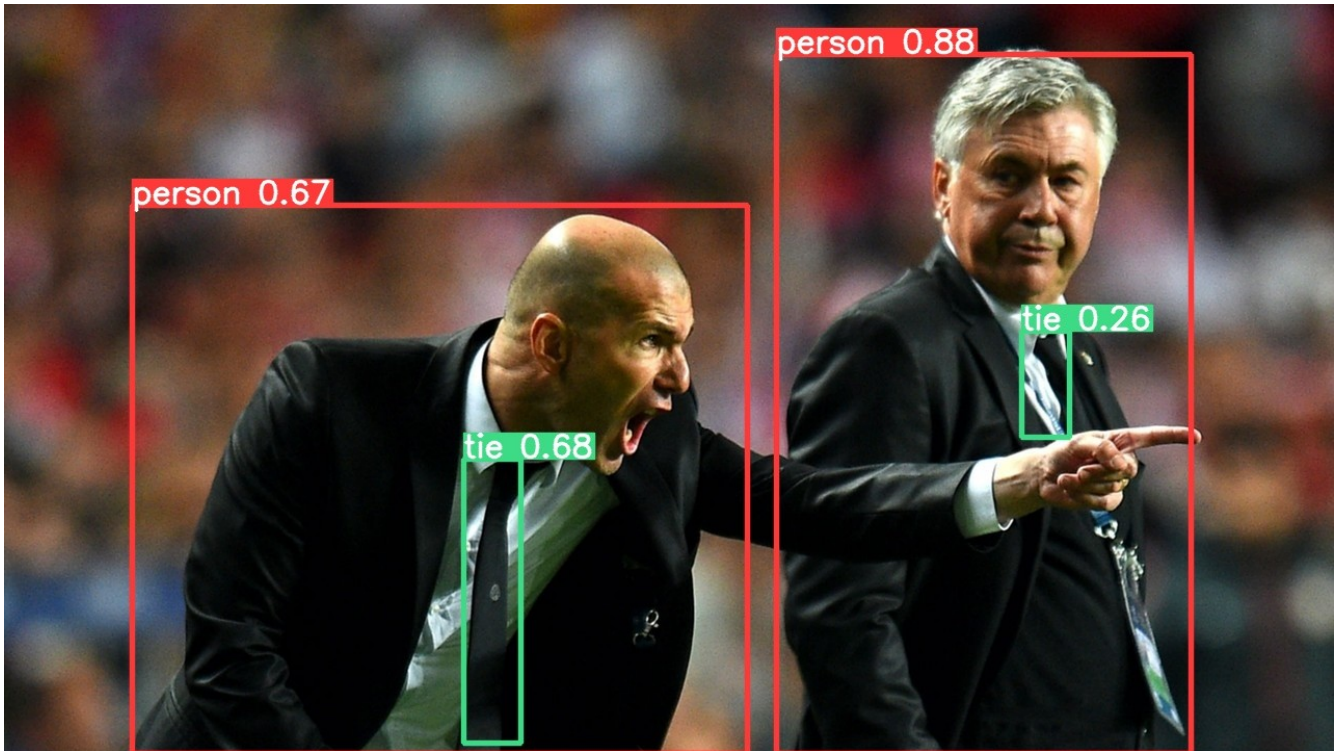
```
# simple test
# --source specifies the test data, which can be pictures or videos, etc.
# --weights specifies the weight file path, which can be trained by
#   ↳ yourself, or directly use the official website.
sudo python3 detect.py --source ./data/images/ --weights yolov5s.pt
```

Test Results:

```
(.yolov5_env) llh@YH-LONG:/mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-7.0$ python detect.py --source ./data/images/zidane.jpg --weights yolov5s.pt
detect: weights=['yolov5s.pt'], source=./data/images/zidane.jpg, data=data/coco128.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 🚀 2023-2-28 Python-3.8.10 torch-1.10.1+cpu CPU

Fusing layers ...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
image 1/1 /mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-7.0/data/images/zidane.jpg: 384x640 2 persons, 2 ties, 193.3ms
Speed: 2.3ms pre-process, 193.3ms inference, 4.0ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs/detect/exp2
```

The above displays in sequence: basic configuration, network parameters, detection results, processing speed, and finally the saved location. We switch to the runs/detect/exp2 directory to view the detection results:



27.2.3 Convert to rknn model

Next, rknn will be exported based on yolov5s.pt:

1、Convert it to yolov5s.onnx (it can also be a model such as torchscript), first install the onnx installation dependent environment:

```
# Install the ambient light of onnx
pip3 install -r requirements.txt coremltools onnx onnx-simplifier
↪onnxruntime

# Obtain weight file, and use yolov5s.pt
# Here we use the following command to export the onnx model
sudo python3 export.py --weights yolov5s.pt --include onnx

# Or use the following command to export Torchscript
```

(下页继续)

(续上页)

```
sudo python3 export.py --weights yolov5s.pt --include torchscript
```

Export display:

```
(.yolov5_env) llh@YH-LONG:/mnt/e/Users/Administrator/Desktop/wsl_user/
↪yolov5/yolov5-7.0$ python export.py --weights yolov5s.pt --include onnx
export: data=data/coco128.yaml, weights=['yolov5s.pt'], imgsz=[640, 640], ↪
↪batch_size=1, device=cpu, half=False, inplace=False, keras=False, ↪
↪optimize=False, int8=False, dynamic=False, simplify=False, opset=12, ↪
↪verbose=False, workspace=4, nms=False, agnostic_nms=False, topk_per_
↪class=100, topk_all=100, iou_thres=0.45, conf_thres=0.25, include=['onnx']
YOLOv5 □ 2023-2-28 Python-3.8.10 torch-1.10.1+cpu CPU

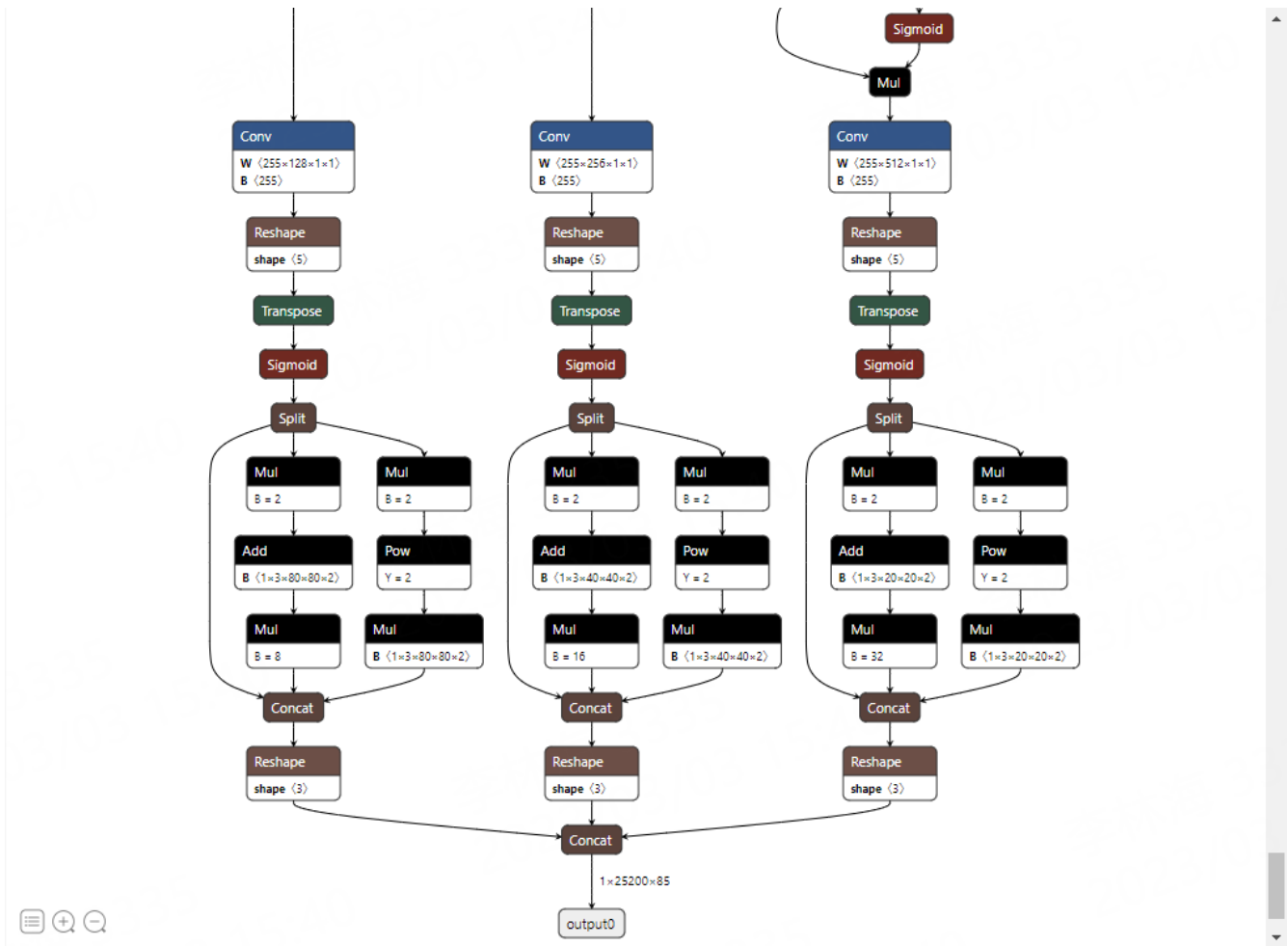
Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs

PyTorch: starting from yolov5s.pt with output shape (1, 25200, 85) (14.1 MB)

ONNX: starting export with onnx 1.13.1...
ONNX: export success □ 1.8s, saved as yolov5s.onnx (28.0 MB)

Export complete (2.3s)
Results saved to /mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-
↪7.0
Detect:          python detect.py --weights yolov5s.onnx
Validate:        python val.py --weights yolov5s.onnx
PyTorch Hub:     model = torch.hub.load('ultralytics/yolov5', 'custom',
↪'yolov5s.onnx')
Visualize:       https://netron.app
```

Yolov5s.onnx files will be generated in the current directory. We can use *Netron* <<https://netron.app/>> _ visible model:



2、Export Looking at the picture above, the rear of the onnx model is finally passed through the DETECT layer to adapt to the RKNN for appropriate processing. Here, remove this network structure (but the Sigmoid function at the rear of the model structure is not deleted), and the three feature maps are directly output. For details, [here](#)

You need to modify the YOLOv5 file source code:

列表 1: models/yolo.py

```
def forward(self, x):
    z = [] # inference output
    for i in range(self.nl):
```

(下页继续)

(续上页)

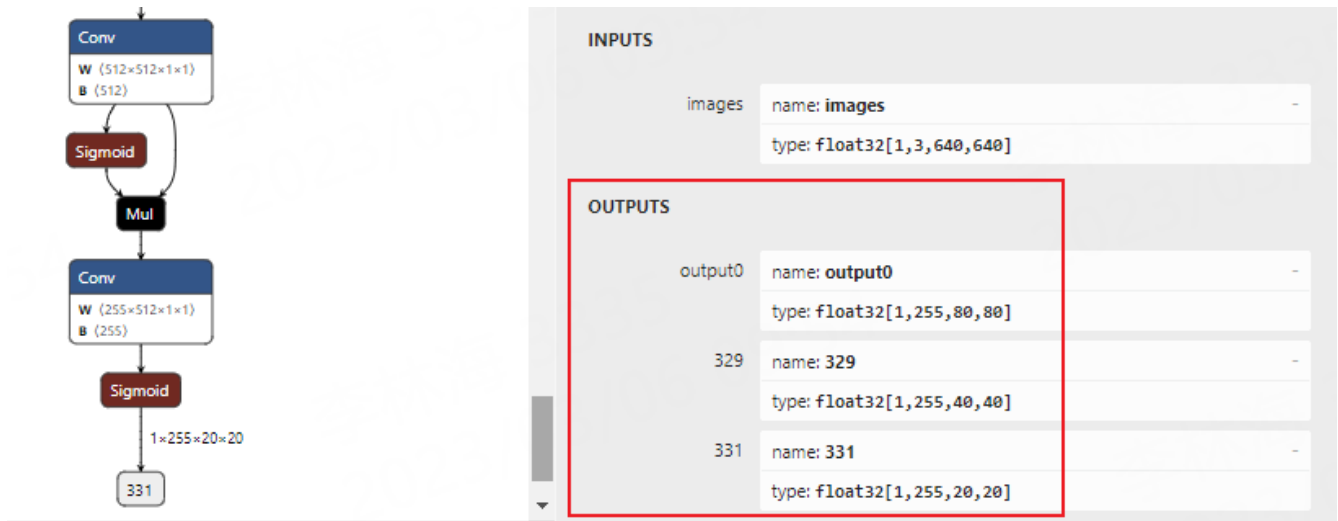
```
z.append(torch.sigmoid(self.m[i](x[i])))

return z
```

列表 2: export.py

```
#shape = tuple((y[0] if isinstance(y, tuple) else y).shape) # model output
↪shape
shape = tuple((y[0] if (isinstance(y, tuple) or (isinstance(y, list))) else
↪y).shape) # model output shape
```

After modification, re -execute the command conversion: “ python export.py -weights yolov5s.pt -found onnx“ Use Netron <<https://netron.app/>> _ visible model:



3、Converted to RKNN model

The front of the Angnx model is converted into the RKNN model. You need to install the environment such as RKNN-Toolkit2, and refer to the chapter of “NPU” .

Use Rknn-Toolkit2 model conversion function:

列表 3: rknn_transfer.py(Part of the code here, please refer to the supporting case)

```
1 if __name__ == '__main__':
2
3     # Create RKNN
4     # If the test encounters a problem, turn on Verbose = TRUE and check
5     ↪ the debugging information.
6     # rknn = RKNN(verbose=True)
7     rknn = RKNN()
8
9     # Enter image processing
10    img = cv2.imread(IMG_PATH)
11    # img, ratio, (dw, dh) = letterbox(img, new_shape=(IMG_SIZE, IMG_SIZE))
12    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
13    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
14
15    # pre-process config
16    print('--> Config model')
17    rknn.config(mean_values=[[0, 0, 0]], std_values=[[255, 255, 255]],
18    ↪ target_platform="rk3568")
19    print('done')
20
21    # If the previous conversion is TORCHScript, renamed .pt files, cancel
22    ↪ the following annotation, and then load the model.
23    #print('--> Loading model')
24    #ret = rknn.load_pytorch(model=PYTORCH_MODEL, input_size_list=[[1, 3, IMG_
25    ↪ SIZE, IMG_SIZE]])
26    #if ret != 0:
27    #    print('Load model failed!')
28    #    exit(ret)
29    #print('done')
```

(下页继续)

(续上页)

```
26
27     # Load the ONNX model
28     print('--> Loading model')
29     ret = rknn.load_onnx(model=ONNX_MODEL)
30     if ret != 0:
31         print('Load model failed!')
32         exit(ret)
33     print('done')
34
35     # Build a model and open the quantification by default
36     print('--> Building model')
37     ret = rknn.build(do_quantization=True, dataset=DATASET)
38     #ret = rknn.build(do_quantization=False)
39     if ret != 0:
40         print('Build model failed!')
41         exit(ret)
42     print('done')
43
44     # Quantitative accuracy analysis. Turn off by default, cancel the_
↪ annotation to open
45     #print('--> Accuracy analysis')
46     #Ret = rknn.accuracy_analysis(inputs=[img])
47     #if ret != 0:
48     #    print('Accuracy analysis failed!')
49     #    exit(ret)
50     #print('done')
51
52     # Export the RKNN model
53     print('--> Export rknn model')
54     ret = rknn.export_rknn(RKNN_MODEL)
55     #if ret != 0:
56     #    print('Export rknn model failed!')
```

(下页继续)



```

57     #         exit(ret)
58     print('done')
59
60     # Initialize the operating environment, the default is not configured.
61     ↪with device_id, etc., you need to turn on when you need to debug.
62     print('--> Init runtime environment')
63     ret = rknn.init_runtime()
64     # ret = rknn.init_runtime(target='rk3568', device_id='192.168.103.
65     ↪115:5555', perf_debug=True)
66     if ret != 0:
67         print('Init runtime environment failed!')
68         exit(ret)
69     print('done')
70
71     # Debug, evaluate the performance of the model, cancel the annotation.
72     ↪and open it
73     # rknn.eval_perf(inputs=[img], is_print=True)
74
75     rknn.release()

```

(下页继续)

(续上页)

(下页继续)



(续上页)

```
D RKNNAPI: API: 1.4.0 (bb6dac9 build: 2022-08-29 16:17:01) (null)
D RKNNAPI: DRV: rknn_server: 1.3.0 (121b661 build: 2022-04-29 11:11:47)
D RKNNAPI: DRV: rknnrt: 1.4.0 (a10f100eb@2022-09-09T09:07:14)
D RKNNAPI: =====
```

done

Performance

```
#### The performance result is just for debugging, ####
#### may worse than actual performance! ####
```

```
Total Weight Memory Size: 7312768
Total Internal Memory Size: 7782400
Predict Internal Memory RW Amount: 138880000
Predict Weight Memory RW Amount: 7312768
```

ID	OpType	DataType	Target	InputShape	OutputShape	DDR Cycles	NPU Cycles	Total Cycles
0	InputOperator	UINT8	CPU	\	(1, 3, 640, 640)	0	0	0
9						1200.00		InputOperator:images
1	Conv	UINT8	NPU	(1, 3, 640, 640), (32, 3, 6, 6), (32)	(1, 32, 320, 320)	687110	691200	691200
						8407	9.14	4409.25
								Conv:Conv_0
2	exSwish	INT8	NPU	(1, 32, 320, 320)	(1, 32, 320, 320)	997336	0	997336
						3737		6400.00
								exSwish:Sigmoid_1_2swish
.....(Omitted)								
145	Sigmoid	INT8	NPU	(1, 255, 80, 80)	(1, 255, 80, 80)	498668	0	498668
						1895		3200.00
								Sigmoid:Sigmoid_199
146	OutputOperator	INT8	CPU	(1, 255, 80, 80), (1, 80, 80, 256)	\	0	0	0
								OutputOperator:output0

(下页继续)

(续上页)

```

147  OutputOperator  INT8      CPU      (1,255,40,40), (1,40,40,256)
      ↪          \              0              0              0
      ↪ 52          \              800.00          OutputOperator:329
148  OutputOperator  INT8      CPU      (1,255,20,20), (1,20,20,256)
      ↪          \              0              0              0
      ↪ 34          \              220.00          OutputOperator:331
Total Operator Elapsed Time(us): 77776
=====

```

The RKNN model will be exported in the current directory. If the EVAL_PERF interface annotation is canceled, the performance evaluation will be performed and simply debug.

27.2.4 Deploy to LubanCat Card

After exporting the RKNN model, use the RKNN Toolkit Lite2 to make a simple deployment on the end of the board.

列表 4: rknnlite_inference.py(Part of the code here, please refer to the supporting case)

```

1  if __name__ == '__main__':
2
3      # Create Rknnlite object
4      rknn_lite = RKNNLite()
5
6      # Import RKNN model
7      print('--> Load RKNN model')
8      ret = rknn_lite.load_rknn(rknn_model)
9      if ret != 0:
10         print('Load RKNN model failed')

```

(下页继续)

(续上页)

```
11         exit(ret)
12     print('done')
13
14     # Initialized operating environment
15     print('--> Init runtime environment')
16     ret = rknn_lite.init_runtime()
17     if ret != 0:
18         print('Init runtime environment failed!')
19         exit(ret)
20     print('done')
21
22     # Enter image processing
23     img = cv2.imread(IMG_PATH)
24     img, ratio, (dw, dh) = letterbox(img, new_shape=(640, 640)) # 灰度填充
25     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
26
27     # reasoning
28     print('--> Running model')
29     outputs = rknn_lite.inference(inputs=[img])
30     print('done')
31
32     # Post -treatment
33     boxes, classes, scores = yolov5_post_process(outputs)
34
35     img_1 = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
36     if boxes is not None:
37         draw(img_1, boxes, scores, classes)
38
39     # Show or save pictures
40     cv2.imwrite("out.jpg", img_1)
41     # cv2.imshow("post process result", img_1)
42     # cv2.waitKey(0)
```

(下页继续)

(续上页)

```
43     # cv2.destroyAllWindows()
44
45     rknn_lite.release()
```

Simple test shows:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [11:59:53.211] RKNN Runtime Information: librknnrt version: 1.4.0
↳(a10f100eb@2022-09-09T09:07:14)
I RKNN: [11:59:53.211] RKNN Driver Information: version: 0.7.2
I RKNN: [11:59:53.213] RKNN Model Information: version: 1, toolkit version:
↳1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳target: RKNPU lite, target platform: rk3568, framework name: PyTorch,
↳framework layout: NCHW
done
--> Running model
done
class: person, score: 0.8889630436897278
box coordinate left,top,right,down: [370, 168, 574, 495]
class: person, score: 0.5832323431968689
box coordinate left,top,right,down: [59, 242, 362, 493]
class: tie, score: 0.668832540512085
box coordinate left,top,right,down: [221, 360, 249, 491]
```

The result will be stored in the out.jpg in the current directory. View the picture:



27.3 airockchip/yolov5 Simple test

The above is a simple test using the official YOLOV5 V7.0. Next, we use AirockChip/YOLOV5. The YOLOV5 of the warehouse is optimized to deploy the RKNPU device. The tutorial test is V6.2.

Pull the latest AirockChip/YOLOV5 warehouse, install similar, refer to the previous front:

```
# The V6.2 version was tested
git clone https://github.com/airockchip/yolov5.git
cd yolov5
```

To obtain a weight document, we need to re-train:

```
# Re-training and obtain the optimized weight file. This is based on weight_
↪ yolov5s.pt, or uses --CFG specified configuration file
sudo python3 train.py --data coco128.yaml --weights yolov5s.pt --img 640
```

(下页继续)

(续上页)

```
# There are a lot of output information after the training, and the output
↪will be out:
...
Optimizer stripped from runs/train/exp/weights/last.pt, 14.9MB
Optimizer stripped from runs/train/exp/weights/best.pt, 14.9MB

Validating runs/train/exp/weights/best.pt...
...
# Training some analysis and weights stored in runs/train/exp
```

It will save files in Runs/Train/EXP/Weights/Save. We renamed yolov5s_relu.pt and copied it to the source code root directory.

Next export TORCHScript and use the command:

```
# --Weights specify the path of the weight file,
# --Rknpu specified platform (RK_Platform supports RK1808, RV1109, RV1126,
↪RK3399Pro, RK3566, RK3568, RK3588, RV1103, RV1106)
# --include specifies the exported onnx model, and the default is
↪TORCHSCRIPT

sudo python3 export.py --weights yolov5s_relu.pt --rknpu rk3568

# Normal conversion will output:
export: data=data/coco128.yaml, weights=['yolov5s.pt'], imgsz=[640, 640],
↪batch_size=1, device=cpu, half=False, inplace=False, train=False,
↪keras=False, optimize=False, int8=False, dynamic=False, simplify=False,
↪opset=12, verbose=False, workspace=4, nms=False, agnostic_nms=False, topk_
↪per_class=100, topk_all=100, iou_thres=0.45, conf_thres=0.25, include=[
↪'torchscript'], rknpu=rk3568
YOLOv5 □ 2022-10-28 Python-3.8.10 torch-1.10.1+cpu CPU
```

(下页继续)

(续上页)

```
Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
--> save anchors for RKNN
[[10.0, 13.0], [16.0, 30.0], [33.0, 23.0], [30.0, 61.0], [62.0, 45.0], [59.
↪0, 119.0], [116.0, 90.0], [156.0, 198.0], [373.0, 326.0]]

PyTorch: starting from yolov5s.pt with output shape (1, 255, 80, 80) (14.1↪
↪MB)

TorchScript: starting export with torch 1.10.1+cpu...
TorchScript: export success, saved as yolov5s.torchscript (27.8 MB)

Export complete (1.92s)
Results saved to /mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-
↪master

Detect:          python detect.py --weights yolov5s.torchscript
Validate:        python val.py --weights yolov5s.torchscript
PyTorch Hub:     model = torch.hub.load('ultralytics/yolov5', 'custom',
↪'yolov5s.torchscript')
Visualize:       https://netron.app

# Or use the following command to export the onnx model to generate the↪
↪YOLOV5S.onnx file in the current directory.
python export.py --weights yolov5s.pt --include onnx --rknpu rk3568
```

Export torchscript, will generate yolov5s_relu.torchscript files in the current directory, and rename it .pt file.

27.3.1 Converted to the RKNN model and deployed to LubanCat RK356X board

Test using the previous conversion file, directly run the program to export the RKNN file, or use the tools [here](#) for model conversion, model evaluation, model deployment, etc.

```
# Run the test, export the RKNN model:
W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
--> Config model
done
--> Loading model
PtParse: 100%|██████████████████████████████████████████████████| 698/698_
↪[00:01<00:00, 602.67it/s]
done
--> Building model
Analysing : 100%|██████████████████████████████████████████████████| 145/145_
↪[00:00<00:00, 381.87it/s]
Quantizing : 100%|██████████████████████████████████████████████████| 145/145_
↪[00:00<00:00, 518.54it/s]
W build: The default input dtype of 'x.1' is changed from 'float32' to 'int8'
↪' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
W build: The default output dtype of '172' is changed from 'float32' to
↪'int8' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
W build: The default output dtype of '173' is changed from 'float32' to
↪'int8' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
W build: The default output dtype of '174' is changed from 'float32' to
↪'int8' in rknn model for performance!
```

(下页继续)

(续上页)

```
Please take care of this change when deploy rknn model.
↪with Runtime API!
done
--> Export rknn model
done
--> Init runtime environment
W init_runtime: Target is None, use simulator!
W init_runtime: Flag perf_debug has been set, it will affect the
↪performance of inference!
I NPUTransfer: Starting NPU Transfer Client, Transfer version 2.1.0
↪(b5861e7@2020-11-23T11:50:36)
D RKNNAPI: =====
D RKNNAPI: RKNN VERSION:
D RKNNAPI:   API: 1.4.0 (bb6dac9 build: 2022-08-29 16:17:01) (null)
D RKNNAPI:   DRV: rknn_server: 1.3.0 (121b661 build: 2022-04-29 11:11:47)
D RKNNAPI:   DRV: rknnrt: 1.4.0 (a10f100eb@2022-09-09T09:07:14)
D RKNNAPI: =====
done
=====
                                Performance
        #### The performance result is just for debugging, ####
        #### may worse than actual performance!                ####
=====
Total Weight Memory Size: 7312768
Total Internal Memory Size: 6144000
Predict Internal Memory RW Amount: 86931200
Predict Weight Memory RW Amount: 7312768
ID   OpType          DataType Target InputShape
↪   OutputShape          DDR Cycles   NPU Cycles   Total Cycles
↪ Time (us)          MacUsage(%)   RW (KB)       FullName
0    InputOperator    UINT8   CPU      \
↪   (1, 3, 640, 640)          0          0          0          (下页继续)
↪ 8 \ 1200.00 InputOperator:x.1
Forum:https://www.firebbs.cn/ 281 Tmall:https://yehuosm.tmall.com
```

(续上页)

```

1      ConvRelu      UINT8      NPU      (1, 3, 640, 640), (32, 3, 6, 6), (32)
→      (1, 32, 320, 320)      687110      691200      691200
→ 8387      9.16      4409.25      Conv:input.4_Conv
2      ConvRelu      INT8      NPU      (1, 32, 320, 320), (64, 32, 3, 3), (64)
→      (1, 64, 160, 160)      750885      921600      921600
→ 2247      45.57      4818.50      Conv:input.6_Conv
..... (Omitted)
80     Conv          INT8      NPU      (1, 512, 20, 20), (255, 512, 1, 1), (255)
→      (1, 255, 20, 20)      66931      102000      102000
→ 180      62.96      429.50      Conv:1189_Conv
81     Sigmoid       INT8      NPU      (1, 255, 20, 20)
→      (1, 255, 20, 20)      31167      0      31167
→ 141      \      200.00      Sigmoid:1190_Sigmoid
82     OutputOperator INT8      CPU      (1, 255, 80, 80), (1, 80, 80, 256)
→      \      0      0      0
→ 161      \      3200.00      OutputOperator:172
83     OutputOperator INT8      CPU      (1, 255, 40, 40), (1, 40, 40, 256)
→      \      0      0      0
→ 46      \      800.00      OutputOperator:173
84     OutputOperator INT8      CPU      (1, 255, 20, 20), (1, 20, 20, 256)
→      \      0      0      0
→ 24      \      220.00      OutputOperator:174
Total Operator Elapsed Time(us): 44412
=====

```

提示: The CPU frequency of the previous test is 1.8GHz, the DDR frequency is 1.056GHz, and the NPU frequency is 900MHz.

The deployment test in the future is not listed as the previous.

27.4 Reference link

<https://github.com/ultralytics/yolov5>

<https://github.com/airockchip/yolov5/tree/master>

https://github.com/airockchip/rknn_model_zoo

Chapter28 Lubancat monitoring and detection

This chapter will briefly introduce an example of camera monitoring and detection. The user logs in to the monitoring page on the browser, and clicks the button after logging in to perform video recording and target detection. The web program uses a python-based flask framework to achieve live streaming. The image is obtained by calling the camera through opencv, and npu is used for image detection and processing.

- Testing platform:lubancat 2
- Board system:Debian10 (xf)
- Python version:Python3.7
- Opencv version:4.7.0.68
- Toolkit Lite2:1.4.0
- Flask:1.0.2

28.1 Dependent tools and library installation

The experimental test uses lubancat 2, the system is Debian10, and some tools and libraries are installed (some tools and systems have already been installed and will not be executed):

```
1 # Installation tool
2 sudo apt update
3 sudo apt -y install git wget
4
5 # Install python related libraries, etc., use python3 by default
6 sudo apt -y install python3-flask python3-pil python3-numpy python3-pip
```

(下页继续)

(续上页)

```
7
8 # Install opencv-python related libraries, the test is to use version 4.7.0.
  ↳ 68, or other versions
9 sudo pip3 install opencv-contrib-python
10
11 # Install rknn-toolkit-lite2, refer to the previous NPU use chapter
```

28.2 Video streaming server and camera get frames

28.2.1 Deploy a video streaming server

In this example, the Flask application framework will be used to build a web page and a live video streaming server.

提示: The simple use of the flask library can refer to the previous [tutorial](#) . Or [Flask official documentation](#) .

Flask returns a Response object with a generator as input via the /video_viewer route. Flask will be responsible for calling the generator, entering a loop, and continuously returning the frame data obtained from the camera as a response block. And send the results of all parts to the client in chunks.

列表 1: sample program ‘lubancat-flask-opencv-rknn/controller/modules/home/views.py’

```
1 from flask import session, render_template, request, redirect, url_for, \
  ↳ Response, jsonify
2 # Import login pages etc.
3 from controller.modules.home import home_blu
4 # Import the VideoCamera class to obtain camera frames or detected frame \
  ↳ streams, etc.
```

(下页继续)

(续上页)

```
5 from controller.utils.camera import VideoCamera
6 import time
7
8 video_camera = None
9 global_frame = None
10
11 # Home page
12 @home_blu.route('/')
13 def index():
14     # Template rendering
15     username = session.get("username")
16     if not username:
17         return redirect(url_for("user.login"))
18     return render_template("index.html")
19
20
21 # Get video stream
22 def video_stream():
23     global video_camera
24     global global_frame
25
26     if video_camera is None:
27         video_camera = VideoCamera()
28
29     while True:
30         # Get a sequence of individual JPEG images
31         frame = video_camera.get_frame()
32         time.sleep(0.01)
33         if frame is not None:
34             global_frame = frame
35             yield (b'--frame\r\n'
36                  b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')
```

(下页继续)

(续上页)

```

37         else:
38             yield (b'--frame\r\n'
39                   b'Content-Type: image/jpeg\r\n\r\n' + global_frame + b'\r\n\r\n')
40
41 # Video stream
42 @home_blu.route('/video_viewer')
43 def video_viewer():
44     # Template rendering
45     username = session.get("username")
46     if not username:
47         return redirect(url_for("user.login"))
48     # Return the Response object of the generator, multipart/x-mixed-
49     ↪replace type, the boundary string is frame
50     return Response(video_stream(),
51                     mimetype='multipart/x-mixed-replace; boundary=frame')

```

In the simple HTML page index.html, one of the image tags ``. It will use `url_for` to point to the route `/video_viewer`, and the browser will automatically display the image from the stream, thus keeping the image element updated:

列表 2: sample program ‘lubancat-flask-opencv-rknn/controller/templates/index.html’ (section)

```

1 <body>
2 <h1 align="center" style="color: whitesmoke;">Flask+OpenCV+Rknn</h1>
3 <div class="top">
4     <div class="recorder" id="recorder" align="center">
5         <button id="record" class="btn">record video</button>
6         <button id="stop" class="btn">pause recording</button>
7         <button id="process" class="btn">Turn on detection</button>

```

(下页继续)

(续上页)

```

8         <button id="pause" class="btn">pause detection</button>
9         <input type="button" class="btn" value="sign out"
10             onclick="javascript:window.location.href='{ url_for('user.
↪logout') }}'">
11         <a id="download"></a>
12         <script type="text/javascript" src="{ url_for('static', filename=
↪'button_process.js') }}"></script>
13     </div>
14 </div>
15 
16 </body>

```

28.2.2 Get frame from camera

Get frames from the camera by importing the packaged VideoCamera class:

列表 3: sample program ‘lubancat-flask-opencv-rknn/controller/utils/camera.py’ (section)

```

1 class VideoCamera(object):
2     def __init__(self):
3         # Use opencv to open the system default camera
4         self.cap = cv2.VideoCapture(0)
5         if not self.cap.isOpened():
6             raise RuntimeError('Could not open camera.')
7
8         # Create RKNNLite object
9         self.rknn_lite = RKNNLite()
10
11        # Set frame width and height
12        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

```

(下页继续)

(续上页)

```
13     self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 640)
14     # .....
15
16     # Exit the program to release resources
17     def __del__(self):
18         self.cap.release()
19         self.rknn_lite.release()
20
21     # Camera capture frame
22     def get_frame(self):
23         ret, self.frame = self.cap.read()
24
25         if ret:
26             # Image detection processing
27             if self.is_process:
28                 #self.image = cv2.cvtColor(self.frame, cv2.COLOR_BGR2RGB)
29                 self.image = cv2.cvtColor(self.frame, cv2.COLOR_BGR2RGB)
30                 self.outputs = self.rknn_lite.inference(inputs=[self.image])
31                 self.frame = process_image(self.image, self.outputs)
32                 #self.rknn_frame = process_image(self.image, self.outputs)
33                 #ret, image = cv2.imencode('.jpg', self.rknn_frame)
34                 #return image.tobytes()
35
36             if self.frame is not None:
37                 # Image coding compression
38                 ret, image = cv2.imencode('.jpg', self.frame)
39                 # Return the image as a bytes object
40                 return image.tobytes()
41         else:
42             return None
43     # .....
```

28.3 NPU processing image

Use the NPU for image detection processing. This example does not have an additional training model. It directly uses the examples/onnx/yolov5 routine in the official Toolkit Lite2 tool, which is only for experimental demonstration. Use RKNN Toolkit Lite2 to deploy the RKNN model on the board system.

提示: For the installation and use of RKNN Toolkit Lite2 on the board, you can refer to the previous [tutorial](#) . Or [official github documentation](#) .

Sample program processing flow:

列表 4: Intercepted from Lubancat monitoring and detection
sample program

```
1  # Create RKNNLite object
2  self.rknn_lite = RKNNLite()
3
4  #.....
5
6  def load_rknn(self):
7      # Import RKNN model
8      print('--> Load RKNN model')
9      ret = self.rknn_lite.load_rknn(RKNN_MODEL)
10     if ret != 0:
11         print('Load RKNN model failed')
12         exit(ret)
13     # Call the init_runtime interface to initialize the running environment
14     print('--> Init runtime environment')
15     ret = self.rknn_lite.init_runtime()
16     if ret != 0:
17         print('Init runtime environment failed!')
```

(下页继续)

(续上页)

```
18         exit(ret)
19
20     #.....
21
22     # Process the pictures acquired by the camera and set the picture size
23     self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
24     self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 640)
25
26     #Convert to RGB format
27     self.image = cv2.cvtColor(self.frame, cv2.COLOR_BGR2RGB)
28
29     # Call the inference interface, perform detection and reasoning on the_
    ↪image, and return the result
30     self.outputs = self.rknn_lite.inference(inputs=[self.image])
31
32     #According to the npu processing results, the image is post-processed and_
    ↪the processed image is returned
33     self.frame = process_image(self.image, self.outputs)
34
35     # Exit the program to release resources
36     def __del__(self):
37         self.cap.release()
38         # Use the release interface to release the RKNNLite object
39         self.rknn_lite.release()
```

Without npu accelerated processing, opencv can be directly used for image detection and recognition processing. Interested students can study the code by themselves, add data sets for training, or replace other models for experiments. Use the opencv library for image processing and digital recognition functions in the experimental code directory: `controller/Utils/opencvTest.py`.

28.4 Adapt to the specific environment

Pull the experimental code:

```
# Enter the following command in the terminal to use the main branch:  
git clone -b main https://gitee.com/LubanCat/lubancat-flask-opencv-rknn.git
```

Modify the IP and port that the server monitors, and modify it according to the actual situation:

```
# Enter the lubancat-flask-opencv-rknn directory  
cd ./lubancat-flask-opencv-rknn  
  
# Modify the startup function parameters in the main.py file  
vim main.py  
  
# The default setting is host="0.0.0.0", which is the ip of the default_  
↪network port  
app.run(threaded=True, host="0.0.0.0", port=5000)  
  
# According to the board environment, set the specific ip  
app.run(threaded=True, host="192.168.103.121", port=5000)  
# Now the IP address of the server listening is 192.168.103.121, and the_  
↪port is 5000.
```

The tutorial test uses ov5648, mipi csi interface. In actual use, you need to confirm the camera number (usually /dev/video0):

1、First confirm the camera number:

```
# Enter the Python3 terminal:  
python3  
  
# Import opencv library package
```

(下页继续)

(续上页)

```
import cv2

# Enter the following command:
cap = cv2.VideoCapture(0)
cap.isOpened()
# If True is printed in the window, the device number is available

# After determining the number, release the camera resource
cap.release()
```

If any error occurs, please increase the number of the camera in turn and try to find the available device number. The upper limit of number increment is the number of video devices, which can be checked by `ls /dev/video*` command.

2、Modify the experimental code to adapt to your own camera:

```
# Enter the following command in the terminal:
# Enter the code directory of the opencv operation camera in the lubancat-
  ↳ flask-opencv-rknn directory:
cd ./controller/utils/

# Modify the VideoCamera(object) function in the camera.py file:
vim camera.py

# Change the function parameter 0 in the self.cap = cv2.VideoCapture(0) ↳
  ↳ statement to the device number corresponding to your camera, or the ↳
  ↳ device file ("/dev/video0")
self.cap = cv2.VideoCapture(0)
```

28.5 Test experiment

1. After modifying the content of the tutorial in the previous section, we can run the experimental code to view the experimental phenomenon.

```
# In the project code directory lubancat-flask-opencv-rknn, execute the  
↪ following command:  
sudo python3 main.py
```

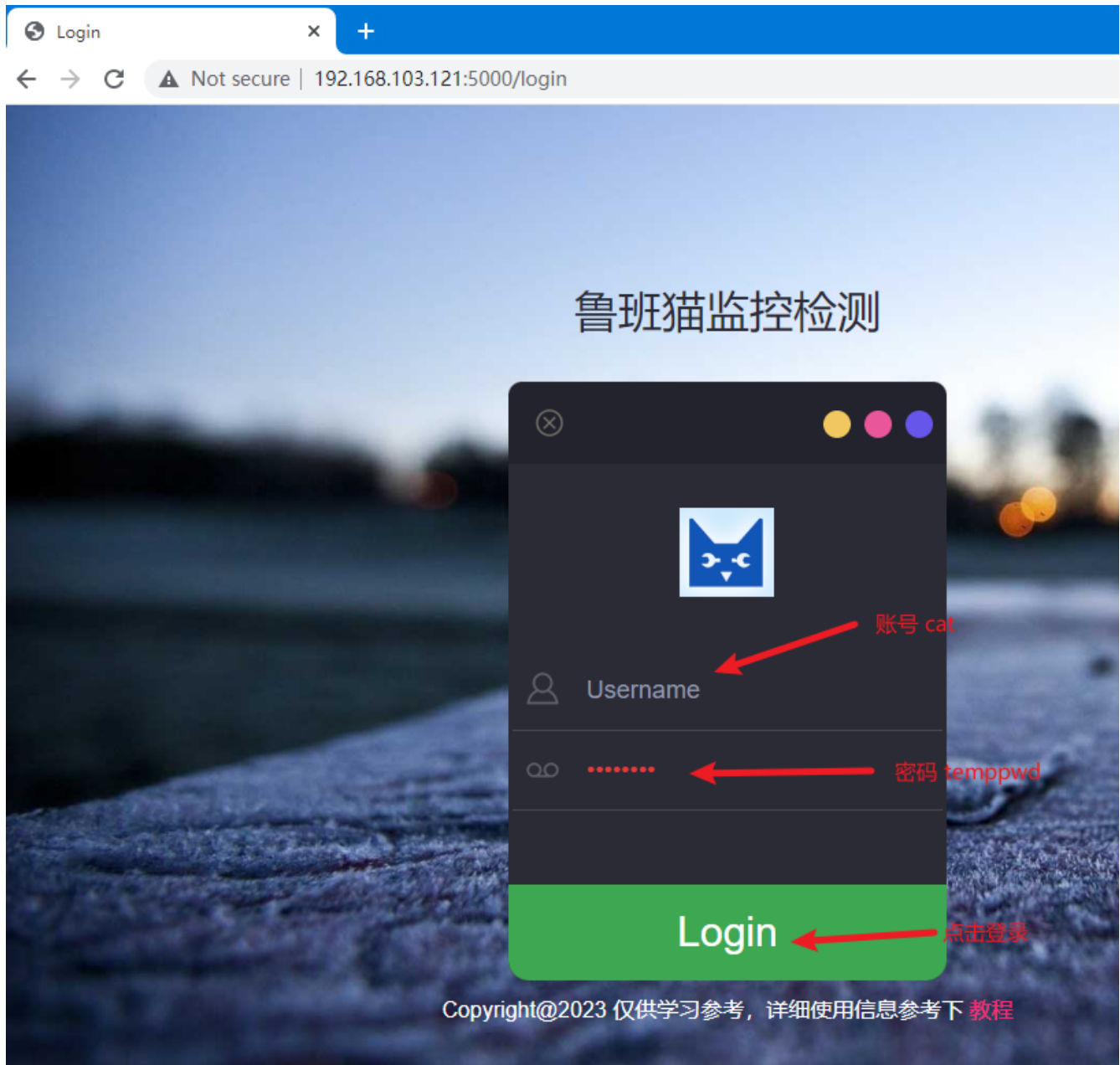
The following experimental phenomena can be seen:

```
cat@lubancat:~/lubancat$ sudo python3 main.py  
* Serving Flask app "controller" (lazy loading)  
* Environment: production  
  WARNING: Do not use the development server in a production environment.  
  Use a production WSGI server instead.  
* Debug mode: on  
I:werkzeug: * Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)  
I:werkzeug: * Restarting with stat  
W:werkzeug: * Debugger is active!  
I:werkzeug: * Debugger PIN: 498-421-198
```

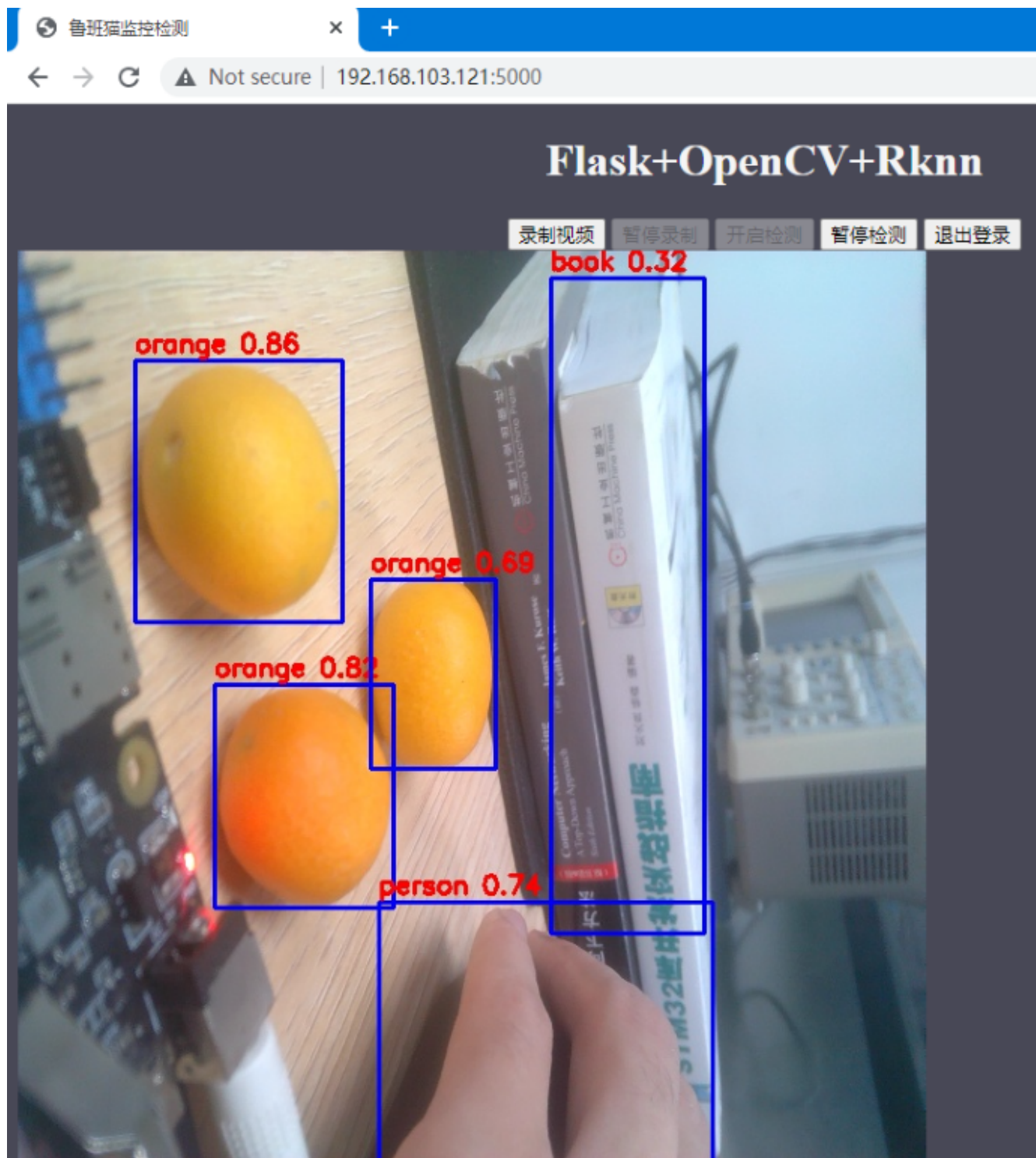
The prompt message printed by the program tells us the address of the server and the default network port ip `http://0.0.0.0:5000` to start monitoring the system. To exit the program, press CTRL+C.

Here, observe the experimental phenomenon by entering the URL in the browser: <http://192.168.103.121:5000>

The experimental phenomenon is shown in the figure:



2. After the login is complete, enter the monitoring interface, click Start Detection to enter the detection state.



3. One frame image acquisition time test.

Use python's time module to calculate the running time of the program to simply test the time required to obtain a frame of image and the time required after npu processing. Add the running time calculation to the program:

列表 5: controller/modules/home/views.py

```
1 # Get video stream
2 def video_stream():
3     global video_camera
4     global global_frame
5
6     if video_camera is None:
7         video_camera = VideoCamera()
8
9     while True:
10         start_time = time.time()
11         frame = video_camera.get_frame()
12         end_time = time.time()
13         print('get_frame cost %f second' % (end_time - start_time))
14         #time.sleep(0.01)
15         if frame is not None:
16             global_frame = frame
17             yield (b'--frame\r\n'
18                   b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')
19         else:
20             yield (b'--frame\r\n'
21                   b'Content-Type: image/jpeg\r\n\r\n' + global_frame + b'\r\n\r\n')
22         ↪r\n')
```

The test environment is as follows: lubancat 2, Lubancat monitoring and detection sample program, using the debian10 system, the default system configuration, using the yolov5 routine model in the Toolkit Lite2 tool, the following test data is for reference only.

When running the program normally without turning on the npu detection image, it takes about 65ms for a

frame of image, of which the fastest is 23ms, and the slowest is 121ms

Turn on npu to process images, NPU defaults to 600Mhz, run: it takes about 345ms to obtain a frame of image (including the time for obtaining camera images, preprocessing images, npu processing images, image post-processing frames, etc.)

Set the NPU frequency to 900Mhz test: it takes about 314ms to acquire a frame of image

```
# lubancat2 npu set 900Mhz:
echo 900000000 > /sys/class/devfreq/fde40000.npu/userspace/set_freq
```

4. Summary

The Lubancat monitoring and detection example implements simple monitoring display and target detection functions, but the effect is not optimal. Only for simple learning reference, possible optimization:

- (1) Adjust the number of frames displayed in the video, how many pictures are processed at a time is more reasonable, and use multi-threading for image processing.
- (2) Confirm the specific usage scenarios, select appropriate models and algorithms, data collection, model adjustment and training.
- (3) Support multiple camera display, etc.

28.6 Reference link

<https://github.com/miguelgrinberg/flask-video-streaming>

<https://gitee.com/Embedfire/flask-video-streaming-recorder>

<https://github.com/rockchip-linux/rknn-toolkit2>

Chapter29 feedback

Due to limited knowledge and time, it is inevitable that there will be mistakes in document editing. If you have any writing suggestions or questions about the document, you are welcome to send your questions and suggestions to zgwinli555@163.com, and we will revise and give feedback in time.

Copyright statement

Embedfire Electronics reserves all copyrights on this project.